

REWIND LANGUAGE / VERSION 2.0.0

REWIND

入門とリファレンス

最初の一行から、巻き戻せる状態と外部処理の設計へ。

32章

60実行例

86モジュール

625標準関数

2026年10月版・オフライン対応・印刷対応

目次

1. 第1章 この本の読み方

2. 第2章 インストールと最初の実行

3. 第3章 変数・値・基本型

4. 第4章 演算と条件分岐

5. 第5章 関数とプログラムの入口

6. 第6章 List・Map・Iterator

7. 第7章 Option・Result・match

8. 第8章 自分の型・generic・trait

9. 第9章 所有権・借用・snapshot

10. 第10章 commit・revert・publishを理解する

11. 第11章 begin・resume・branch

12. 第12章 module・公開範囲・プロジェクト

13. 第13章 効果・許可・外部領域

14. 第14章 コンソール入力と引数

15. 第15章 Fileとpublishの失敗

16. 第16章 文字列・Unicode・正規表現

17. 第17章 非同期処理・cleanup・取消

18. 第18章 ネイティブGUIを作る

19. 第19章 Undo・入力欄・複数画面

20. 第20章 HTTP・TCP・サービス

21. 第21章 SQLite・PostgreSQL・transaction

22. 第22章 JSON・CSV・設定データ

23. 第23章 BigInt・Decimal・日時

24. 第24章 数値配列・線形代数・統計

25. 第25章 自動微分・最適化・モデル保存

26. 第26章 アルゴリズムの実装と選択

27. 第27章 診断・テスト・record / replay

28. 第28章 速度・メモリ・実行予算

29. 第29章 ツールと日々の開発

30. 第30章 実習：小さなアプリケーションへ進む

31. 第31章 構文と組込み操作の早見表

32. 第32章 API編の読み方と保証範囲

標準ライブラリ

std.args	std.autodiff	std.autodiffAsync	std.autodiffForwardAsync
std.bigint	std.bits	std.bitset	std.bytes
std.clock	std.collections	std.config	std.csv
std.csvStream	std.datetime	std.db	std.dbDatetime
std.dbDecimal	std.decimal	std.deque	std.disjointSet
std.distributions	std.dp	std.error	std.external
std.fenwick	std.fft	std.fftAsync	std.flow
std.geometry	std.graph	std.graphLarge	std.graphLargeAsync
std.gui	std.guiDialog	std.guiForm	std.guiMenu

std.guiTable	std.guiWindows	std.heap	std.http
std.httpRouter	std.httpServer	std.integer	std.json
std.jsonStream	std.lazySegment	std.map	std.matching
std.math	std.matrix	std.models	std.modular
std.number	std.numeric	std.numericAsync	std.numericIdentityAsync
std.numericIndex	std.numericInputAsync	std.numericRange	std.numericReduceAsync
std.numericShapeAsync	std.numericTransformAsync	std.optimize	std.option
std.path	std.range	std.regex	std.result
std.rollbackSet	std.scanner	std.search	std.segment
std.sequence	std.sort	std.sparse	std.sparseAsync
std.stream	std.stringSearch	std.suffix	std.task
std.taskError	std.tcp	std.text	std.training
std.trie	std.unicode		

共通型付録・出典と検証

第1章 この本の読み方

REWINDは、計算途中の状態を保存し、戻し、まだ確定していない出力を取り消せるプログラミング言語です。この本はREWIND 2.0.0の入門とリファレンスを兼ねています。将来の草案ではなく、公開済み処理系と同じ標準ライブラリの実装を対象にしています。

最初の数章は、ファイルの一つ作って実行するところから始めます。変数、条件分岐、関数、コレクションを覚えたら、巻き戻し、所有権、エラー処理に進んでください。その後は必要に応じて、文字列、ファイル、非同期処理、GUI、通信、DB、数値計算の章を選べます。最後のAPI編は標準ライブラリの全86公開モジュール・625関数を検索するためのものです。

実行例と説明用断片

「REWIND・実行例」と表示されるコードは、それぞれ独立したプログラムです。必要なimportや補助関数も含めています。コピーして `main.rw` に保存できます。「構文」や「断片」は、文法や一部分を示すもので、そのまま実行する前提ではありません。CLIの例では `rewind` がPATHに入っているものとします。

`assert_eq(a,b)` は等しいことを確認する命令です。失敗すると実行時診断になります。出力がない例も、`assert`が最後まで通れば成功しています。`take` という補助関数は例に定義した関数であり、予約語や組込み関数ではありません。

HTMLの使い方

目次から章へ移動できます。検索欄で日本語、関数名、型名、エラーコードを探せます。このHTMLには本文、スタイル、検索処理をすべて含めているため、ダウンロード後はネット接続なしで読めます。ブラウザの文字サイズ変更や検索も利用できます。

「印刷 / PDF保存」からブラウザの印刷画面を開けます。保存先にPDFを選べば、全章を印刷向けのレイアウトで保存できます。画面での読書には、折り畳めるAPI実装と検索を使えるHTMLがおすすめです。紙面のページ番号に依存せず、章と節の名前で参照します。

本書が説明する保証

「巻き戻せる」とは、VMが管理する状態を巻き戻せるという意味です。すでに表示した文字、送信済みのHTTP要求、DBに確定した書き込み、OSのIME変換途中の状態まで消せるという意味ではありません。本書では、その境界を具体例で繰り返し説明します。

API編のsignature、効果、generic boundはv2.0.0 SDKの機械生成文書から抽出しています。費用・失敗契約がソースに明記されている場合は原文も載せています。記述がない関数の計算量を、名前だけから推測して保証することはしません。必要な場合は、その場で開ける実装を確認してください。

第2章 インストールと最初の実行

SDKを入れる

GitHubのv2.0.0 ReleaseからOSに合ったSDKを選びます。Linux x86_64とWindows x64の配布があります。ソースアーカイブにはソースが入っており、実行バイナリを試す場合はSDKを選んでください。

展開したSDKには `bin/rewind`、`bin/rewindc` があります。Windowsでは `rewind.exe`、`rewindc.exe` です。`bin` をPATHへ追加するか、実行ファイルのフルパスを指定します。

CLIコマンド

```
rewind --version
rewind --help
```

版が2.0.0であることを確認してください。チェックサムはダウンロードしたファイルの破損確認に使います。署名検証では、配布者の公開鍵を信頼できる経路で確認することが別途必要です。同じ配布ページに鍵があるだけでは、配布元を独立に確認したことにはなりません。

Hello, REWIND

作業用フォルダーを作り、次をUTF-8で `main.rw` に保存します。

REWIND・実行例 hello

```
Out.println("Hello, REWIND!");
publish;
```

▼ 標準出力（検証済み）

```
Hello, REWIND!
```

CLIコマンド

```
rewind run main.rw
```

出力は `Hello, REWIND!` です。`Out.println` は改行を付けた出力をVM内で準備します。`publish` で外部へ確定します。プログラム終了を暗黙のpublishとして扱わないことが最初の大切な規則です。

REWIND・実行例 unpublished

```
Out.println("まだ公開していません");
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

この例の標準出力は空です。処理を実行していないわけではなく、pending出力を確定していないためです。何も表示されないなら、publishの位置を確認してください。

単一ファイルのrootを決める

この本の単一ファイル例では、`--root` を付けずに実行します。CLIはsourceの親からrewind.tomlを探し、なければsourceの親をrootとして現行languageのstandalone modeを選びます。明示した `--root` にmanifestがない場合は旧来のproject読み込み経路になり、現行構文やstdが使えないことがあります。rootを明示するprojectではlanguage="2.0.0"のmanifestを用意してください。

コンパイルして渡す

CLIコマンド

```
rewind compile main.rw
rewind run main.rwc
```

標準では同じ場所に `main.rwc` を作ります。 `rewindc main.rw` もコンパイル用の入口です。 `rewind main.rwc` も実行方法です。成果物はソースなしでも実行できますが、runtimeは必要です。OSのネイティブ実行ファイルへ変換するコマンドではありません。

成果物、record、lockは正確なcompiler版と関係します。更新後は再コンパイル・再記録・必要なlock更新を行います。同じREWINDという名前だけで、全版の成果物を相互に実行できる保証はありません。

Windowsで試す

PowerShellで直接指定する場合の形です。パスは実際の展開場所に置き換えます。

PowerShell

```
& 'C:\Tools\rewind\bin\rewind.exe' --version
& 'C:\Tools\rewind\bin\rewind.exe' run .\main.rw
```

CLIへ渡すパスはOSの形式です。一方、File APIに渡すパスはroot相対の `/` 区切りです。この二つを混同しないでください。コンソールプログラムだけならGUI用ディスプレイ環境は不要です。

練習

Helloの文字列を自分の名前に変えて実行してください。次にprintlnを二つ書き、最後に一回publishしてください。最後にpublishを削除し、標準出力が空になることを確認します。「出力を準備する」と「確定する」が別の処理であることが分かります。

第3章 変数・値・基本型

letとvar

`let` はbindingの再代入を禁止し、`var` は許可します。bindingは名前と値の結び付きです。型が推論できる場合、型注釈は省略できます。

REWIND・実行例 variables

```
let initial:Int = 10;
var score = initial;
score += 5;
Out.println(score);
publish;
```

▼ 標準出力（検証済み）

15

出力は15です。`initial=20;` はletへの再代入なので拒否されます。`score` に後でStringを入れることもできません。`var` は型が何にでも変わる変数という意味ではありません。

Listなどはletで保持していても内容を更新できます。`let values=List<Int>(); values.add(1);` は正当です。bindingの再代入と、所有するデータへの変更は別の規則です。変更できないsnapshotにはFrozenを使います。

数値の型

型	表現	注意点
Int	signed 64-bit	範囲外の通常算術は診断になる
Float	IEEE 754 binary64	小数の丸め、NaN、無限大がある
BigInt	std.bigintの任意精度整数	予算と容量の範囲で正確に計算する
Decimal	std.decimalの十進数	precision、scale、丸め方を選ぶ

Intは-9,223,372,036,854,775,808から9,223,372,036,854,775,807までです。大きい数を扱うからといってFloatへ変えると、整数の下位桁が失われる場合があります。正確な整数にはBigInt、十進の丸めを指定する計算にはDecimalを検討してください。

```
let thousand = 1_000;
assert_eq(thousand,1000);
assert_eq(0xff,255);
assert_eq(0b1010,10);
assert_eq(0o755,493);
let fraction:Float=1.25;
assert_eq(fraction,1.25);
assert_eq("\u{754c}", "界");
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

underscoreは数字の間に置きます。型名と識別子は大文字・小文字を区別します。IntとFloatの間に暗黙変換を仮定しないでください。対象APIのchecked変換を使用します。

String、Bytes、Unit

StringはUTF-8文字列、Bytesはbyte列です。Bytes("text") はUTF-8表現を作ります。一文字を必ず一byteと考えてはいけません。Unicodeの章でbyte・scalar・graphemeの違いを扱います。

Unitは値 () を持つ型です。値を返す必要のない関数の戻り値に使います。Boolは true と false です。if 1 のように整数を暗黙に真偽値として扱いません。

tupleには異なる型をまとめられます。(Int,String) の値は (3,"three") です。要素は ._0、._1 で読みます。Listは同じ要素型を持つ列、Mapはキーと値の型を指定する対応表です。

コメントと文字列escape

行コメントは // から行末までです。/* ... */ は入れ子にできます。Stringは二重引用符で囲み、改行 \n、復帰 \r、tab \t、引用符 \"、backslash \\、NUL \0、Unicode scalar \u{HEX} を使えます。Unicode escapeは有効なscalarを1~6桁のhexで指定します。surrogateは有効なscalarではありません。

第4章 演算と条件分岐

演算子の優先順位

弱い順は `||`、`&&`、`== !=`、`< <= > >=`、`+`、`-`、`*`、`/`、`%` です。括弧で意図を明示できます。`&&` と `||` は短絡評価し、必要がない右辺を評価しません。

REWIND・実行例 arithmetic

```
assert_eq(2+3*4,14);
assert_eq((2+3)*4,20);
assert_eq(-7/3,-2);
assert_eq(-7%3,-1);
let divisor=0;
assert_eq(divisor!=0 && 10/divisor>1,false);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

整数除算はゼロ方向へ切り捨てます。余りは被除数の符号に従います。負の剰余を0からm-1に揃えたい場合、通常の `%` の結果をそのまま使わず、`std.modular`の契約を確認します。bit操作は`std.bits`にあります。

ifの書き方

条件の括弧は省略できます。ブロックの波括弧は必要です。

REWIND・実行例 even-odd

```
var num = 13;
commit start;
Out.println("Even");
if num%2 == 1 {
    revert start;
    Out.println("Odd");
}
publish;
drop start;
```

▼ 標準出力 (検証済み)

Odd

出力はOddです。Evenはpublish前にrevertで取り消されています。ifの中から外側のcheckpointへ戻るこの形はv2.0.0で利用できます。revert後もifの実行位置は進み、次のprintlnを実行します。条件を最初から評価する動作ではありません。

通常の分岐なら次の方が簡単です。

REWIND・実行例 plain-if

```
let num=13;
if num%2 == 0 {
    Out.println("Even");
} else {
    Out.println("Odd");
}
publish;
```

▼ 標準出力（検証済み）

Odd

巻き戻しは、分岐をすべて置き換えるためのものではありません。多数の変更を試してからまとめて戻したい場合に便利です。二つの表示を選ぶだけならif/elseで十分です。

反復

`for i in 0..4` は0、1、2、3を順に使います。終端は含みません。whileは条件がtrueの間繰り返します。breakはループを終了し、continueは次の反復へ進みます。

REWIND・実行例 loops

```
var sum=0;
for i in 0..4 {
    if i==2 {continue;}
    sum+=i;
}
while sum<10 {sum+=1;}
assert_eq(sum,10);
Out.println(sum);
publish;
```

▼ 標準出力（検証済み）

10

無限ループでも実行予算は消費します。revertによって予算が回復することはありません。長い反復は停止条件と入力サイズを考えて設計します。

練習

1から100までの和を求めるには `1..101` を使います。3の倍数だけ加算する条件を加えてください。さらに、負の数の商と余りについて、上のassertの除数を変えて確かめてください。

第5章 関数とプログラムの入口

関数を定義する

関数には名前、引数、戻り値の型、効果を宣言します。`effects {}` は外部操作をしない契約です。`return`で結果を返します。通常の間数呼出しには`await`をしません。

REWIND・実行例 factorial

```
fn factorial(n:Int)->Int effects {} {  
  if n<=1 {return 1;}  
  return n*factorial(n-1);  
}  
assert_eq(factorial(5),120);  
Out.println(factorial(5));  
publish;
```

▼ 標準出力（検証済み）

120

再帰では同じ関数を呼びます。停止条件が重要です。この例は5!の計算を確認するためのもので、任意の巨大なnに対応する階乗ではありません。負のnを拒否する検査もしていません。実用関数では定義域、overflow、予算を考え、必要ならResultやBigIntを使います。

引数が少ないからといって効果が少ないとは限りません。出力する関数は `effects {output}` を宣言します。効果のない関数から、出力関数を呼び出すことも効果検査の対象です。

main関数

短いプログラムではトップレベルに文を並べられます。アプリケーションの入口を明示する場合は `fn main()->Int` を使います。

REWIND・実行例 main-function

```
fn main()->Int effects {output} {  
  Out.println("started");  
  publish;  
  return 0;  
}
```

▼ 標準出力（検証済み）

started

0は正常終了です。非0は終了値として扱われます。mainを使っても終了時に出力を自動確定しません。publishを明示してください。外部へ見える結果を最後にまとめて確定する関数なのか、途中でも確定する関数なのかをAPIの設計時に決めると、呼び出し元が扱いやすくなります。

補助関数の責任

文字列を整数に変換する関数は、成功したらInt、失敗したらエラーを返すと自然です。常にpanicする補助関数はテストや短い例には便利ですが、ユーザーの入力を読むアプリケーションでは、呼出し元が再入力やエラー表示を選べるResultの方が適しています。

小さく分ける目安は「この関数が成功したとき何が保証されるか」を一文で説明できることです。計算と入力、モデル更新とGUI表示、SQL構築とDB実行を分ければ、計算部分を外部アクセスなしにテストできます。

第6章 List・Map・Iterator

Listの基本

Listは型付きの可変列です。indexは0始まりです。`get` の範囲外は診断となり、自動でNoneになりません。入力から得たindexでは、先に範囲を確認します。

REWIND・実行例 list

```
let values=List<Int>();
values.add(3);
values.add(5);
values.add(7);
assert_eq(values.len(),3);
values.set(1,9);
var total=0;
for i in 0..values.len() {total+=values.get(i);}
Out.println(total);
publish;
```

▼ 標準出力（検証済み）

19

出力は19です。空のListの最後の要素を読むために `len()-1` を使う場合、先に空かどうかを確認してください。要素型がIntであることと、indexが正当であることは別の問題です。

Listは永続ページを使うVMのデータ構造です。checkpointで保持する過去の版と共有でき、更新された部分は分離されます。普通のメモリ配列の全操作を厳密にO(1)と見積もるのではなく、ページ木の探索、COW、要素の共有・コピー、保持する履歴の費用も含めて考えます。

Mapの基本

Mapのgetは値の有無をOptionで返します。存在しないキーと、値として0が入っていることを区別できます。

```
import std.map as map;
let counts=Map<String,Int>();
counts.set("apple",3);
assert_eq(counts.get("apple"),Some(3));
assert_eq(counts.get("pear"),None);
assert_eq(map.getOr(&counts,"pear",0),0);
map.update(&mut counts,"apple",|old:Option<Int>|->Option<Int>{
  match old {
    Some(n)=>{return Some(n+1);},
    None=>{return Some(1);}
  }
});
assert_eq(counts.get("apple"),Some(4));
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

`std.map.update` のcallbackは、Someを返すと設定し、Noneを返すと削除します。キーはStringです。std.mapのすべての関数が任意のKを受け取るわけではありません。 `entries` もStringキー用です。API編の型を確認してください。

キーにはOrdの契約があります。組込みのキーとuser定義Ordキーでは格納・探索費用が同一とは限りません。文字列をキーにすると、正規化や大文字小文字の同一視は自動ではありません。

Iteratorはsnapshotを読む

Iteratorを使うと、列挙、変換、絞込み、集約をつなげて表現できます。 `collect` は結果をListに集めます。

```
let values=List<Int>();
values.add(1);values.add(2);values.add(3);
let doubled=values.iter()
  .map(|n:Int|->Int{return n*2;})
  .filter(|n:Int|->Bool{return n>2;})
  .collect();
assert_eq(doubled.len(),2);
assert_eq(doubled.get(0),4);
assert_eq(doubled.get(1),6);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

Iteratorにはnext、fold、take、enumerate、zipもあります。元のListを更新しても、既に作ったIteratorのsnapshotに従って変更が混ざることはありません。cursorはcheckpointの対象です。取り出し位置を戻せば同じsnapshotを再度列挙できます。

`iter()` と `freeze` は `native work・history memory` を消費します。深さ64、循環、関数や資源の捕捉にも制限があります。大きいcollectionだから直ちに1万要素で拒否されるという規則ではなく、Mapキーの検査制限とは区別します。列挙してできるListまで含めて必要メモリを見積もってください。

データ構造の選択

必要な操作	候補
順に保持・添字で読む	List
キーで対応を引く	Map、std.map
両端を追加・削除	std.deque
最小値を反復して取り出す	std.heap
同じ集合かを判定	std.disjointSet、std.rollbackSet
区間集約・更新	std.fenwick、std.segment、std.lazySegment
bit集合	std.bitset
staticな区間最小値	std.sparse

heapの比較関数は全操作で一貫させます。disjointSetとrollbackSetは用途が異なり、VMのrevertとは別にデータ構造自身の履歴を操作する場合はrollbackSetを選びます。容量指定やResultの失敗処理を省略しないでください。

第7章 Option・Result・match

値がないことを表現する

`Option<T>` は `Some(value)` か `None` です。例えば「検索が見つからない」を、例外ではなく通常の結果として表現できます。

REWIND・実行例 option

```
let value:Option<Int>=Some(7);
match value {
  Some(n)=>{assert_eq(n,7);},
  None=>{panic("値がありません");}
}
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

`None`を0と同じにするかどうかはアプリケーションの判断です。金額が0であることと金額未入力を区別したい場合、早い段階で`None`を0へ変換すると情報を失います。

成功と失敗を分ける

`Result<T,E>` は `Ok(value)` か `Err(error)` です。EはStringでも、recordでも、enumでも構いません。標準ライブラリの `StdError` は code と offset を持ちますが、すべてのエラーが `StdError` ではありません。

REWIND・実行例 result

```
import std.number as number;
import std.integer as integer;
fn increment(input:String)->Result<Int,String> effects {} {
  match number.decimal(input) {
    Ok(value)=>{return integer.add(value,1);},
    Err(_)=>{return Err("InvalidNumber");}
  }
}
fn twice(input:Int)->Result<Int,String> effects {} {
  let value=integer.add(input,input)?;
  return Ok(value);
}
assert_eq(increment("41"),Ok(42));
assert_eq(increment("x"),Err("InvalidNumber"));
assert_eq(twice(21),Ok(42));
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

`?` は `Result` を返す関数の中で使い、`Err` ならその失敗を早期 `return` します。呼出し先と返却先の失敗型を合わせます。任意の異なるエラー型を自動変換する規則ではありません。

通常算術の `overflow`、範囲外 `get`、`panic`、実行予算超過は、勝手にその関数の `Result` へ変換されません。上の `integer.add` は回復可能な `checked` 加算を選んでいる点が大切です。失敗を受け取って続行したい場所では、検査するか `fallible` API を使います。

matchの網羅性

`enum` を `match` する場合、可能な `variant` を扱います。入れ子の `Option/Result`、`tuple`、`generic enum`、`record` も検査対象です。 `_` は値を捨てる `pattern` であり、「分岐を省略しても何か返す」という命令ではありません。

`guard` は追加条件です。`guard` が付いた分岐だけでは、その `variant` 全体を網羅したことになりません。`guard` の中では `checkpoint` 操作や `publish` を行えません。前の `arm` に完全に含まれる `arm` や不正な `Int` 区間も拒否されます。

所有する接続などを `Result` から取り出す場合は `match move result` が重要です。普通の `match` で共有借用した値に、後から `&mut` を付ければよいとは限りません。所有権の章と `DB・通信` の例を参照してください。

エラーの型を設計する

入力ミス、存在しないデータ、外部通信失敗を同じ文字列へ潰すと、呼出し元が判断できません。`enum` の `variant` や `code` を使って分類し、詳細メッセージは別に持つと扱いやすくなります。特に外部書込みでは「未送信」と「送信後の結果不明」を区別しないと、`retry` で重複する可能性があります。

第8章 自分の型・generic・trait

structとrecord

structは可変データ、recordはShareを満たす不変データの組合せに使います。recordのfieldにはShareな型が必要です。名前付きのデータは、tupleより意味を表しやすくなります。

REWIND・実行例 data-types

```
struct Counter {value:Int}
record Point {x:Int,y:Int}
enum Status {Ready,Failed(String)}
type Score=Int;
let counter=Counter(1);
counter.value=2;
let point=Point(3,4);
assert_eq(point.x+point.y,7);
let status=Status::Failed("missing");
match status {
  Status::Ready=>{},
  Status::Failed(message)=>{assert_eq(message,"missing");}
}
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

struct/record/enumには位置引数のconstructorがあります。named constructorとfield patternも利用できます。private fieldを持つ型は、module外から直接構築・読み書き・分解できません。公開factoryとaccessorを使います。

外部接続やnative配列のような組込み型は、API文書にfieldが見えていても、任意の値をconstructorで作れば正当な接続になるものではありません。opaque値の生成は対応する標準関数を使ってください。

generic

同じ操作を異なる型へ適用するためにtype parameterを使います。**T:Ord+Share** は、そのTに順序と共有の契約を要求します。genericは「型を検査せず何でも入れる」という仕組みではありません。

```

trait Read {fn read(self:&Self)->Int effects {};}
struct Box {value:Int}
impl Read for Box {
    fn read(self:&Box)->Int effects {} {return self.value;}
}
fn read<T:Read>(value:&T)->Int effects {} {return value.read();}
fn compare<T>(a:T,b:T)->Int effects {} where T:Ord+Share {
    return a.cmp(b);
}
let box=Box(7);
assert_eq(read(&box),7);
assert_eq(compare(1,2),-1);

```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

traitは型が提供する操作の契約、implはその実装です。関連型とdefault methodも利用できます。whereを使うと、複数の制約をsignatureから整理して書けます。

generic factoryの型は、戻り値の期待型から全parameterを確定できる場合に推論できます。分からない場合は `List<Int>()` のように明示します。深い型の展開と特殊化は有限のcompiler予算に従うため、無制限の型レベル計算を前提にしないでください。

クロージャー

`|n:Int|->Int{return n*2;}` は引数を取る関数値です。引数がないければ `||->Int{...}` です。関数型にもeffectsがあり、必要に応じて `captures {Send,Share}` の契約が付きます。

捕捉方法はvalue、move、borrowを区別します。借用captureは借用元より長生きしたりtaskへ逃げたりできません。callback引数に関数を渡す場合、「何を計算するか」だけでなく「何を捕捉し、どんな効果を持つか」もsignatureの一部です。API編ではcompilerの関数型表記 `!{...}~{...}` も載せています。`!`の後は効果、`~`の後はcapture契約です。

捕捉を指定する例

value captureは作成時の値を保持します。あとで元bindingを変更しても捕捉値は変わりません。

```

var n=7;
let read=capture value ||->Int{return n;};
n=9;
assert_eq(read(),7);
assert_eq(n,9);

```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

move captureはownerをクロージャーへ移します。元の名前から再利用するのではなく、クロージャーが所有する値として使います。

REWIND・実行例 capture-move

```
let values=List<Int>();values.add(7);
let read=capture move ||->Int{return values.len();};
assert_eq(read(),1);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

borrow captureは借用の有効なscopeへ限定します。scopeを出た後なら元ownerを更新できます。

REWIND・実行例 capture-borrow

```
let values=List<Int>();values.add(7);
{
    let read=capture borrow ||->Int{return values.len();};
    assert_eq(read(),1);
}
values.add(8);
assert_eq(values.len(),2);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

名前付きvariantとmatch式

field名で構築するenumは、引数の順番だけに頼らず値の意味を示せます。matchを式として使う場合、各armの返す値の型を合わせます。

REWIND・実行例 named-enum

```
enum Command {Write {path:String,count:Int},Quit}
let command=Command::Write {count:2,path:"a"};
let text=match command {
    Command::Write {path:value,count:_}=>value,
    Command::Quit=>"quit"
};
assert_eq(text,"a");
let size=match 4 {
    0..3=>"small",
    n if n>3=>"large",
    _=>"middle"
};
assert_eq(size,"large");
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

最後のmatchにはguardで覆えない場合のarmもあります。区間の終端とguardの網羅性を混同しないでください。

練習

座標を持つPointと、二つのPointの距離の二乗を返す関数を作ってください。Intで計算する場合はoverflowの可能性を考えます。成功・失敗をResultへ分ける版と、Floatで計算する版では契約がどう変わるでしょうか。

第9章 所有権・借用・snapshot

何を渡すのかを明示する

可変ownerは自由に複製できません。関数へ値を渡すとき、所有権を移すなら `move`、読むだけなら `&`、変更するなら `&mut` を使います。Int、String、適切なrecordなどのShare値は共有できます。

REWIND・実行例 ownership

```
fn total(values:&List<Int>->Int effects {} {
    var result=0;
    for i in 0..values.len() {result+=values.get(i);}
    return result;
}
fn append(values:&mut List<Int>,n:Int)->Unit effects {} {
    values.add(n);
}
let values=List<Int>();
append(&mut values,7);
assert_eq(total(&values),7);
let snapshot=freeze(values);
let copy=thaw(snapshot);
copy.add(8);
assert_eq(copy.len(),2);
assert_eq(snapshot.len(),1);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

totalはListを読み、appendは変更します。snapshotはその時点の値を保持します。thawで得た可変値の変更はsnapshotへ混ざりません。bindingがletかvarかだけでは、関数に渡せるかどうかは決まりません。

借用が生きている間、元ownerを競合する形で変更・moveできません。借用引数を戻り値や長生きするtaskへ逃がすことにも制限があります。検査に合わせるために無差別にclone相当の処理を追加するのではなく、関数が本当に読むだけか、所有する必要があるかを先に考えます。

ShareとSend

Shareは不変として共有できる契約、Sendはtaskへ渡せる契約です。同じではありません。通信接続はSendでもShareではなく、freezeでsnapshotへ入れられるものではありません。

外部リソースを含むTaskの完了値は一度だけ取り出せます。スコープ所有の接続を別の場所へ移す場合、所有権を明示します。終了済みのphysical接続をcheckpointから復活させることはできません。

freezeの費用

freezeは、対象グラフの検証と必要なコピーを行います。不変ページの共有はできますが、操作全体が無条件に無料になるわけではありません。循環や深さ64を超える構造、関数・資源の捕捉は制限されます。検証のnative workと一時領域も予算対象です。

v2.0.0のnative factoryが返すList/MapとOption/Result内のそれらは、通常の変数所有値として扱えます。例えばnumeric.valuesFloatで得たListをsetしても元配列は変わりません。これは外部接続を共有可能にする変更ではありません。

第10章 commit・revert・publishを理解する

三つの操作を分ける

`commit name;` はVMの作業状態をcheckpointとして保持します。`revert name;` は状態を戻し、現在の実行位置から続けます。`publish;` はpending I/Oを外部へ確定します。この三つは代用できません。

REWIND・実行例 publish-revert

```
var score=10;
commit test1;
score=99;
Out.println(score);
commit test2;
publish;
revert test1;
Out.println(score);
publish;
drop test2;
drop test1;
```

▼ 標準出力（検証済み）

99
10

出力は次の通りです。

表示例

99
10

99のprintlnは最初のpublishで外部へ出ています。revertはscoreを10へ戻しますが、既に出た99を消しません。最後のpublishは新しい10の出力だけを確定します。古い99を再送しません。

publish前なら出力を取り消せる

REWIND・実行例 pending-revert

```
var score=10;
commit test1;
score=99;
Out.println(score);
commit test2;
revert test1;
Out.println(score);
publish;
drop test2;
drop test1;
```

▼ 標準出力（検証済み）

10

出力は10だけです。99のprintlnはpendingであり、test1に戻すことで破棄されています。test2をcommitしていてもpublishにはなりません。

時点	score	pending出力	外部に確定した出力
test1保存直後	10	空	空
99をprintlnした直後	99	99	空
publishした場合	99	確定済み扱い	99
test1へrevert	10	test1の状態	既に確定した99は残る
10をprintln・publish	10	新しい出力を確定	99、10

この表のpublishを省けば、確定した99は存在しません。Gitの履歴を思い浮かべることは役立ちますが、Gitの全コマンドと一対一に対応する仕様ではありません。特にcheckpointは外部ファイルへのcommitではありません。

状態には何が含まれるか

変数、heap、List/Map、仮想file、Iterator cursor、task状態などが対象です。乱数のVM状態も戻せます。一方、消費した実行予算、記録済みのHost観測、確定済みの外部作用は戻りません。

checkpoint後に宣言した変数はrevertで失われる場合があります。戻す前のローカル名を使い続ける設計にしないでください。GUIでもViewとmodelは戻せますが、OSへ確定した画面を変更するには戻したsceneを再度present・publishする必要があります。

dropを忘れたら

publishはcheckpointを自動で捨てません。dropを書かなければ、checkpointが過去の状態を保持し続けます。保持する履歴が予算を圧迫する場合があります。終了時にruntimeが解放されることと、長時間実行中に不要な履歴を解放することは別です。

Undoのために保持したいcheckpointは正当な使用メモリです。不要になったcheckpointはdropしてください。過去のsnapshotを保持しながら変更すると、そのsnapshotが参照する旧ページは回収できません。最後の参照がなくなると回収可能になります。

第11章 begin・resume・branch

自動のbegin

ユーザーコードを実行する前に、`commit begin;` に相当する初期checkpointが自動保存されます。beginは予約名であり、自分でcommit/dropしたり、変数・関数・branchの名前にしたりできません。

REWIND・実行例 begin-reset

```
var score=99;
commit test;
Out.println(score);publish;
Out.println("discarded");
revert begin;
var score=10;
commit test;
Out.println(score);publish;
drop test;
```

▼ 標準出力（検証済み）

```
99
10
```

出力は99、10です。pendingのdiscardedは消えます。ユーザーcheckpointもリセットされるため、testを新しく作れます。revert begin後にscoreを宣言し直している点にも注目してください。

revert beginはmain taskで、active branchの外から使います。変数・heap・未公開I/O・task状態を初期化しますが、実行中の関数やblockの継続枠は保持します。後続のreturnやblock終了を可能にするためです。公開済みの外部結果、観測記録、消費済み予算を消す「OS全体の初期化」ではありません。

revertとresume

revertは状態、resumeは状態と制御位置を復元します。resume beginならユーザーコード先頭へ戻ります。停止条件を外側に持たずにresumeを繰り返すと、予算を消費するループになります。

同じ関数呼出しの中なら、入れ子のifから外側のcheckpointへrevertできます。既に終了した内側スコープや別の関数呼出しのcheckpointへrevertすると、InvalidContinuationになります。checkpointのデータが存在するだけで、任意の継続位置が有効になるわけではありません。

branchの隔離

branchは試行用の状態を作り、終了時に親状態へ戻します。

```
var score=10;
branch trial {
    score=99;
    Out.println("試行の出力");
}
assert_eq(score,10);
Out.println(score);
publish;
```

▼ 標準出力（検証済み）

10

出力は10です。branch内の変更とpending出力は親へ持ち帰りません。branch内でpublishして試行を外部へ確定する使い方には制限があります。採用する結果を返す一般的な関数と、隔離試行のbranchを使い分けてください。

設計の目安

「失敗したら戻す」だけならResultと新しい値の構築で済むこともあります。多数の関連状態をまとめて保持したいならcheckpointが役立ちます。ユーザー操作のUndoでは、どの操作単位にcheckpointを置くか、いくつ保持するか、外部保存の後もUndoを許すかを明確にします。

第12章 module・公開範囲・プロジェクト

標準moduleのimport

REWIND・実行例 import

```
import std.number as number;
assert_eq(number.decimal("42"),0k(42));
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

`std.number` はmodule、`number` はこのsourceで使うaliasです。aliasはmoduleごとの名前です。選択importとaliasも利用できます。単一ファイルで使うstdはcompilerに内蔵されており、最初の一つのプログラムにsdk-installは不要です。

自分のmodule

宣言は原則としてmodule内に属し、外部へ見せるものにpubを付けます。private fieldは型の不変条件を守るために使います。利用者が内部fieldへ直接アクセスする必要があるなら、必要な操作をpublic関数として定義できないかを先に検討します。

相対importはsource directoryを基準に解決します。projectではsource_root、import設定、lock、署名が解決の基準になります。module名の変更は利用側のimportにも影響します。

projectを使う場合

単一ファイルから先へ進み、複数moduleと固定した依存を管理する場合はrewind.tomlを使います。次は基本形です。

設定例

```
language = "2.0.0"
source_root = "."
entry = "main.rw"
effects = "input,output,args,locale,random,tasks"
```

CLIコマンド

```
rewind check --root .
rewind test --root .
rewind run --root .
rewind build --root . --output app.rwc
```

checkは型と効果を調べ、testはprojectのテストを実行します。SDKをprojectへ固定するには、署名を検証してsdk-installします。標準moduleをprojectのvendorへ固定し、lockを更新するため、あとでSDKの場所を動かしても解決できます。

CLIコマンド

```
rewind sdk-verify --sdk SDK_DIR --public-key PUBLIC_KEY_HEX  
rewind sdk-install --root PROJECT_DIR --sdk SDK_DIR --public-key PUBLIC_KEY_HEX
```

PUBLIC_KEY_HEXはplaceholderです。検証済みの鍵へ置き換えます。鍵、SDK、projectが異なるときの失敗を無視して先へ進めないでください。

APIを壊さず変更する

public functionの引数型、戻り型、効果、generic bound、所有権契約は利用者に影響します。単に関数名を残しても、Shareを追加要求したり新しい効果を付けたりすれば呼出し元がコンパイルできなくなる場合があります。本書API編の宣言は、それらを含む契約として読んでください。

第13章 効果・許可・外部領域

宣言と実行許可

効果は、関数が行う操作をsignatureの一部にする仕組みです。 `effects {output}` を宣言しても、あらゆる環境で自動的に操作を許可されるわけではありません。実行者の許可も必要です。

効果	主な用途
input / output	標準入力、stdout / stderr
fileRead / fileWrite	root内の仮想file操作
args / env	引数、許可された環境変数
locale / random	locale、VM乱数
clock	時刻の観測
tasks	task、channel、scheduler
gui	ネイティブ画面とGUI入力
external	即時の外部操作領域
network	HTTP、TCP、listener
db	SQLite、PostgreSQL
live	journalを作らない外部操作

単一sourceの既定許可はinput、output、args、locale、random、tasksです。file、env、clock、gui、external、network、db、liveが必要なら明示します。Envは環境変数名の許可も必要です。

CLIコマンド

```
rewind run main.rw --allow-effects fileRead,fileWrite
rewind run main.rw --allow-effects gui
rewind run main.rw --allow-effects external,network,tasks
rewind run main.rw --allow-effects external,db,tasks
```

効果不足の診断では、呼出し元のeffectsとCLIの許可を両方確認します。効果検査は通信先URLの妥当性やSQLの正しさを保証するものではありません。

externalの境界

File.writeTextやOut.printlnはpublishを待つ仮想I/Oです。通信やDBはVM外の相手とやり取りするため、external領域を使います。領域の終了はpublishではありません。

構文・説明用断片

```
var pending:Option<Task<Result<HttpResponse,HttpError>>>=None;
external {
    pending=Some(http.get(url,10000,65536));
}
// 領域の外でTaskを待つ
```

領域内では操作をsubmitします。await、task切替、checkpoint、publish、領域を抜ける制御フローは拒否されます。Taskを作ったら外へ出てawaitする二段階の形にしてください。Taskの生成後にpublishしても、まだ完了していない通信が完了扱いになることはありません。

通常・fresh・live

通常のexternalは観測結果を保持し、同じcheckpointに戻った場合に再利用します。external freshは新しい操作として扱います。どちらも、実際に起きた外部作用をrevertで消しません。

external liveはreplay用の外部journalを作しません。明示的なlive許可が必要です。記録を残さないから高速・安全・再現可能だと一括して考えないでください。replayに必要な観測を失う境界であり、通常領域とは保証が異なります。

Secret

Secretは診断や記録で値を伏せるための型です。暗号化、メモリ上の完全消去、誤送信防止をすべて自動保証する型ではありません。HTTP credentialやDB credentialsのalias登録では、値をfingerprintに直接入れないためのopaque参照と組み合わせます。

公開ログに接続文字列やtokenを自分でprintlnすれば、Secretを使う設計の意図を壊します。権限、ログ、記録の共有範囲をアプリケーション側でも決めてください。

第14章 コンソール入力と引数

引数を読む

アプリ引数はCLIの `--` より後ろへ渡します。手前はREWINDのオプションです。

REWIND・実行例 arguments

```
let arguments=Args.all();
Out.println(arguments.len());
if arguments.len()>0 {Out.println(arguments.get(0));}
publish;
```

引数: `-- sample`

▼ 標準出力（検証済み）

```
1
sample
```

CLIコマンド

```
rewind run main.rw -- sample
```

この場合は1、sampleを出力します。 `-- --help` もapplication引数であり、REWINDのヘルプを開く指定ではありません。引数の数を先に検査し、getの範囲外診断を利用説明の代わりにしないようにします。

byte scannerを使う

大量の空白区切り整数を読む場合、Stringを全部splitするとtokenごとの中間値が増えます。std.scannerはBytesを保持して位置を進めるparserです。次の例は入力を固定して、その挙動を確認しています。

REWIND・実行例 scanner

```
import std.scanner as scanner;
fn take<T,E>(value:Result<T,E>->T effects {}) {
    match move value {Ok(v)=>{return move v;},Err(_)=>{panic("parse failed");}}
}
let input=Bytes("10 -3 7\n");
let scan=take(scanner.fromBytes(input));
assert_eq(scanner.nextInt(&mut scan),Ok(Some(10)));
assert_eq(scanner.nextInt(&mut scan),Ok(Some(-3)));
assert_eq(scanner.nextInt(&mut scan),Ok(Some(7)));
assert_eq(scanner.nextInt(&mut scan),Ok(None));
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

EOFはOk(None)です。InvalidNumberとOverflowは失敗です。fromBytesの入力容量は1MiBです。空白はASCII spaceと9～13で、任意のUnicode空白を認識するscannerではありません。失敗前にcursorが進む場合があるため、Errだから自動で入力位置が元に戻るとは考えないでください。

増分入力・出力

std.streamはtoken、UTF-8、writerの補助を提供します。In.readChunkとOut.writeBytesを組み合わせれば、全部を一つのStringへ集めることを避けられます。chunkの境目でUTF-8文字やtokenが分割されるため、状態を保持するdecoder/parserを使います。

一度のpublishに巨大な出力を集めることもメモリを使います。適切な単位で確定する設計にするとpending領域を抑えられますが、後のrevertでその公開済み出力を取り消せなくなる点も考慮してください。

整数入力の実用設計

「必要数だけ読めた」「各値が許容範囲」「余分なtokenの扱い」を決めてください。入力が途中で終わったらNoneを正常値として加算せず、不足として扱います。空入力を許すか、空配列を返すか、利用説明を出すかはプログラムの仕様です。

第15章 Fileとpublishの失敗

書込みはまずVMへ

REWIND・実行例 files

```
File.writeText("state.txt","saved");
publish;
assert_eq(File.readText("state.txt"),Ok("saved"));
```

必要な許可: fileRead,fileWrite

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

実行にはfileReadとfileWriteを許可します。この例は実行root内にstate.txtを書きます。writeTextだけでは外部へ確定しません。commit/revertの対象となるpending操作をpublishで反映します。

Fileのsource内パスは相対パス、/区切りです。絶対パス、..、drive指定、root外へのsymlinkを許可しません。rootは単に見やすい表示名ではなく、アクセスの基準です。rootの外へ保存したいから相対パスに../を付ける使い方は拒否されます。

読取りと観測

readTextはResultを返します。ファイルがないこと、読み出せないこと、内容が期待と違うことを区別して処理します。読み取りは外部観測でもあるため、replayでは記録を使って検証できます。replayが無断でHost fileを上書きすることは通常ありません。

入力ファイルを読み、検査し、新しい内容を準備して、最後にpublishする形は扱いやすい設計です。保存直前に外部から変更された場合はExternalStateConflictになることがあります。勝手に上書きして解決するのではなく、利用者へ競合を伝える設計を考えます。

複数操作は一括atomicではない

複数file、directory、stdout、GUIを一つのpublishに含めても、OS全体のtransactionになる保証はありません。適用途中で失敗するとPublishPartiallyAppliedが返り、成功確認済みの操作と失敗phaseを調べる必要があります。

「エラーだから何も起きていない」と判断すると、既に保存したfileや表示した出力を二重に適用する可能性があります。同じruntimeで盲目的な再publishは拒否されます。実際の外部状態と診断のappliedを確認し、再開の方法を判断してください。

DB transactionとの違いにも注意します。FileのpublishはDBのcommitではなく、DBのcommitはVMのcheckpoint作成ではありません。

cleanup

FileHandleにはusingがあり、scope終了時の解放を表現できます。接続やcursorにもscope所有と明示closeがあります。cleanup中の失敗も無視せず、元の診断へcausesとして追加される情報を調べてください。scopeを抜けたことと、外部書込みがpublishされたことも別です。

第16章 文字列・Unicode・正規表現

一文字の単位を選ぶ

操作	index・長さの単位
String.byteLen、String.find	UTF-8 byte
String.charLen、std.text.length	Unicode scalar
String.utf16Len	UTF-16 code unit
String.slice	byte半開区間、UTF-8境界を検査
std.text.slice / find	scalar区間 / scalar offset
std.unicode.grapheme*	extended grapheme cluster
regex span、unicode offsets	UTF-8 byte offset

REWIND・実行例 text

```
import std.text as text;
assert_eq("a👤界".byteLen(),8);
assert_eq("a👤界".charLen(),3);
assert_eq("a👤界".utf16Len(),4);
assert_eq(text.slice("a👤界",1,3),Ok("👤界"));
assert_eq("a👤界".slice(1,5),Ok("👤"));
```

▼ 標準出力なし・正常終了を検証済み

空（assertを含む処理は正常終了）

byte、scalar、graphemeを混ぜると、日本語や絵文字で不具合が出ます。regexのspanをstd.text.sliceへそのまま渡すことも間違いになり得ます。単位を関数の名前やデータ型の説明へ残してください。

graphemeと正規化

見た目一文字の「e+結合アクセント」は二つのscalarで一つのgraphemeになり得ます。国旗や家族の絵文字も同様です。GUIのbackspaceを人が見た一文字単位にしたいならgrapheme編集を選びます。

```
import std.unicode as unicode;
fn take<T,E>(value:Result<T,E>->T effects {} {
    match move value {Ok(v)=>{return move v;},Err(_)=>{panic("unicode failed");}}
}
let source="é🇯🇵";
assert_eq(take(unicode.graphemeCount(&source)),3);
assert_eq(take(unicode.graphemeSlice(&source,1,2)),"🇯🇵");
let normalized=take(unicode.normalize(&source,unicode.Form::NFC));
assert_eq(take(unicode.isNormalized(&normalized,unicode.Form::NFC)),true);
let clusters=take(unicode.graphemes(&source));
clusters.add("!");
assert_eq(clusters.len(),4);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

StringやMapキーを自動正規化する規則はありません。NFC/NFD/NFKC/NFKDを明示して使います。互換正規化は見た目や意味に影響する文字も変えるため、ユーザーが入力した原文を無条件にNFKCへ書き換えるべきかは用途で判断します。データ版はunicode.versionsで確認できます。

正規表現

std.regexでcompileしたRegexを再利用します。findは見つからなければNoneを返し、spanはbyte区間です。groupはそのcaptureが参加しなければNoneになる場合があります。続きの検索にはAPIのcursorを使います。

対応する文法はAPI契約に従います。すべての他言語のregex拡張と同じとは限りません。特に任意のbackreferenceやlookaroundを前提にせず、compile結果を検査してください。regex自体と入力にも容量・work予算があります。

正規化と検索の順番

正規化後のStringで得たoffsetを元のStringへ戻して使うと、byte列が変わっているため間違えます。原文を表示する検索では、正規化後の結果と原文の対応を別途管理するか、どの文字列のoffsetかを明確にしてください。

第17章 非同期処理・cleanup・取消

Taskの二段階

async fnの呼出しはTaskを作り、spawnで実行へ進め、awaitで結果を待ちます。awaitは `Result<T,TaskError>` を返します。

REWIND・実行例 async

```
async fn compute(n:Int)->Int effects {} {return n*2;}
let task=spawn compute(21);
match await task {
  Ok(value)=>{assert_eq(value,42);Out.println(value);},
  Err(_)=>{panic("task failed");}
}
publish;
```

▼ 標準出力（検証済み）

42

TaskのT自体がResultなら、awaitの結果は二重のResultです。外側はtaskの待機失敗や取消、内側はそのAPI自身の失敗です。通信やDBの例で `take(take(await task))` になる理由はこの二段階です。

REWINDのVMは決定的な協調schedulerを使います。asyncを書くことがOS thread上のCPU並列計算を意味するわけではありません。外部I/OのworkerとVM taskの実行は別の層です。

timeoutとisDone

Taskのlogical timeoutはwall clockの秒数ではありません。HTTPのtimeoutMillisは実時間の期限です。両者を同じ意味で扱わないでください。 `task.isDone()` は待機せず完了を調べますが、完了を確認するループだけで他taskへ適切に制御を渡せるとは限りません。

ChannelとTaskGroupは、結果の通知や複数taskの寿命管理に使います。待機関係に循環があり進行できなければTaskDeadlockになり、waitGraphを調べます。取消したこととcleanupが完了したことを区別し、cancelAndJoinやjoinなどで終了を確認します。

defer

deferはscope終了、return、失敗のcleanupを登録します。次は出力順を確認する例です。

```
fn work()->Unit effects {output} {  
    defer ||->Unit{Out.println("cleanup");};  
    Out.println("work");  
}  
work();  
publish;
```

▼ 標準出力（検証済み）

```
work  
cleanup
```

出力はwork、cleanupです。deferもprintlnをpublishへ自動変換しません。cleanupで失敗した場合、元の失敗が主診断に残り、cleanupの失敗はcausesへ追加されます。エラーの原因が二つあるとき、最後の一つだけを見ると問題を見失います。

数値計算と応答性

同期native kernelの実行中は、任意の命令位置でtaskを切り替えるとは限りません。GUIと重い計算を組み合わせるなら、numericTransformAsync、numericReduceAsync、numericShapeAsync、numericInputAsync、numericAsync、graphLargeAsync、autodiffAsyncなど、必要な協調APIを選びます。

例えば要素変換と集約の対応するAPIは4096論理要素以下のstepへ分けます。これは全native APIが一律に同じstep幅で動くという保証ではありません。各APIの費用契約を確認します。async版が作る途中状態も履歴・予算の対象です。取消で部分結果を完成済み配列として返すことはしません。

第18章 ネイティブGUIを作る

model、View、OSの画面

GUIの状態は三層に分けると理解しやすくなります。modelは数値や入力内容、Viewは部品と配置、OSの画面はpublish済みsceneです。modelとViewを変更しても、presentとpublishをしなければ画面へ確定しません。

最初の例はウィンドウにラベルとボタンを表示し、閉じる入力を待ちます。実行にはgui許可とネイティブ表示環境が必要です。

REWIND・実行例 gui-minimal

```
import std.gui as gui;
fn checked(value:Result<Unit,gui.GuiError>->Unit effects {} {
    match value {Ok(_)=>{},Err(e)=>{panic(e.code);}}
}
match gui.window("Hello GUI",320,180) {
    Err(e)=>{panic(e.code);},
    Ok(view)=>{
        checked(gui.label(&mut view,"message","Hello, REWIND",20,20,280,40));
        checked(gui.button(&mut view,"close","Close",20,90,120,40));
        checked(gui.present(&view));
        publish;
        var running=true;
        while running {
            match gui.nextEvent() {
                Err(e)=>{panic(e.code);},
                Ok(event)=>{
                    match gui.dispatch(&mut view,event) {
                        Some(gui.Action::Close)=>{running=false;},
                        Some(gui.Action::Activate(id))=>{if id=="close"{running=false;}},
                        _=>{}
                    }
                }
            }
        }
        checked(gui.close());
        publish;
    }
}
```

必要な許可: gui / GUI検証は入力fixtureを使用。通常実行にはnative表示環境が必要。

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

windowはViewを作るfactoryです。OSのウィンドウ表示はpresentでsceneを準備してpublishした時点で行われます。nextEventは入力を待ち、dispatchは座標やキーを部品に対応するActionへ変換します。

GUIの容量

一画面の各軸は64～4096 client pixelsです。部品は最大2048、idは空でなく128 UTF-8 bytes以内、captionは4096 bytes以内です。sceneは1MiB以内です。idの重複や部品の範囲外配置はResultで拒否されます。

Window APIの複数画面は最大16、合計画素は16,777,216までです。単一画面用とnamed-window用の入力・closeを混ぜず、対象を明示してください。ネイティブのOS標準部品すべてを自動で提供するAPIではなく、canvas sceneを中心に構成します。

OSの条件

WindowsはWin32/GDI、LinuxはX11を使います。LinuxではDISPLAY、対応する24bit画面、libX11、font環境が必要です。Wayland環境ではXwaylandなどの条件を確認します。GUIを使わないCLI実行のためにX11を必ず起動する必要はありません。

日本語入力にはOS側のIME/XIMサービスも必要です。Linuxのnative file dialogはGTK3の条件があります。両OSで実装された機能と、すべてのOS設定・font・入力法で完全に同じ見た目になる保証は別です。

fixtureで検証する

GUIの入力は記録可能です。`--gui-events EVENTS.json`を使う試験では、OSウィンドウを開かずにevent列を与えます。これはmodel更新と終了の回帰試験に便利ですが、fixtureが通るだけでネイティブIMEや画面の描画が試験できたことにはなりません。

第19章 Undo・入力欄・複数画面

Undoの順番

Undoではcheckpointへmodel/Viewを戻し、既に受信した入力を読み飛ばし、戻した画面をpresent・publishします。単にrevertするだけではOSの画面は変わりません。

構文・説明用断片

```
revert ready;
gui.continueInput();
// 復元したmodelに基づくViewを準備
checked(gui.present(&view));
publish;
```

continueInputは、この実行ですでに届けた入力だけを読み飛ばします。replayでも同じ契約です。これを省くと、巻き戻した入力cursorが古いクリックをもう一度読む設計になり得ます。ユーザーが「Undo」を押した直後に古い「Add」が再度効く動作を避けるための区別です。

入力欄の編集

textBoxは単一行、textAreaは複数行の入力です。selectionはscalar offsetでcursorとanchorを持ちます。graphemeEditingをtrueにすると、結合文字や絵文字をcluster単位で編集できます。位置の表現はscalarのままです。

Ctrl+C/V/Xによるclipboard操作が使えます。コピーは公開したsceneの選択範囲から行います。貼り付けの確定text入力は記録されますが、OSのclipboard自体をrevertするものではありません。

IMEの未確定文字列はOS側の変換状態です。確定した文字だけがVMのtext入力になります。未確定状態をsnapshotへ含めません。変換中に異なる入力状態をpublishすると、旧contextをキャンセルする場合があります。日本語入力を扱うときは、確定textとcomposition中の表示を混同しないでください。

複数画面と他task

std.guiWindowsで名前付き画面を扱います。同期のnextEventAnyは入力待ち中にVM taskを進めません。他taskと組み合わせる現在のAPIにはnextEventAnyAsyncがあります。古い案内の「handoffは今後追加」は現在の制限ではありません。

GUI更新とpublishはapplication側で行い、通信・DB処理taskからはChannel等で結果を通知する構成にすると、UIと外部作用の境界を保てます。同期的な重い計算をイベントハンドラーへ直接入れると、async入力を使っても応答性を失う可能性があります。

form、table、menu、dialog

guiFormは入力項目と検査、guiTableは表の表示、guiMenuはメニューとshortcut、guiDialogはnative file選択の補助です。gui.overlayは同じ寸法と編集modeのsceneを合成し、下の画面のfocusとscrollを保持します。

file選択はgui、external、tasksの操作で、領域内でTaskを作り外でawaitします。選んだパスにアクセスするFile権限は別途必要です。dialogでcancelしたことはfileへの書込みではありません。選択結果は観測として再利用し、replayでOS dialogを開き直しません。

練習

Counterアプリケーションを作り、AddとUndoの二つのボタンを置いてください。Undo後にmodelは戻るが画面が戻らない場合、presentとpublishを確認します。保存ボタンを追加したら、「Undoは保存済みfileも戻すのか」をアプリケーションの仕様として明示してください。

第20章 HTTP・TCP・サービス

HTTP client

`std.http.get`は`Task<Result<HttpResponse,HttpError>>`を返します。領域内で送信を開始し、外側でTaskとHTTP結果の両方を処理します。`timeout`、`response`容量、`statusCode`を明示します。

本文末尾の実習では、URLを引数で受け取る完全なHTTP clientを掲載します。`get`は`redirect`や`retry`を自動で行いません。HTTP 404などの`status`と、接続・TLS・期限による失敗は分けて扱います。サーバーの`status`が200だったことと、`body`が正しい形式であることも別です。

HTTPSでは証明書`chain`と`hostname`を検証します。CA指定や`credential`は設定用APIを使います。診断を避けるためにTLSを検証しない接続へ変える設計にしないでください。

逐次送受信

`download`と`read`で`body`をchunkごとに受け取れます。`read`のEOFはNoneです。`upload`、`write`、`finish`は送信側を分けます。接続は`Send`で`Share`ではなく、`scope`終了や`close`で解放します。`Result`から取り出す所有権にも注意してください。

buffered HTTP要求の`maxBytes`は最大4MiB、`timeout`は最大120,000msです。`download`の`read chunk`は1~65,536 bytes、全体は最大1GiBです。期限は将来の`read`も含む`transfer`全体へかかります。`native`同時操作数も有限です。API契約の上限以内でも、`work/memory`予算によって失敗する場合があります。

`write`の受付成功が、相手`application`で処理済みという意味になるとは限りません。送信後`timeout`になった場合は結果が不明かもしれません。`revert`で送信を消して`retry`することはできません。

TCPとHTTP server

`std.tcp`はbyte streamを扱います。`read`はmessage境界を保証しません。アプリケーションの`protocol`で長さや区切りを決め、分割・結合された`read`を処理します。HTTP用途には`httpClient`と`httpRouter`を選べます。

`httpRouter`は`method`と`path`を対応させる補助です。認証、認可、入力`schema`、業務`transaction`まで自動で完成するframeworkではありません。外部から受ける入力の上限と検査をアプリケーションに設けてください。

未応答`request`を捨てたとき、`listener`が生きていれば`oneshot`の破棄により空の503応答となります。`listener`自体を先に閉じる場合は接続終了になります。どちらも成功した200応答として扱いません。取消の後に不要な`request resource`を残さない設計が重要です。

recordとretry

記録付き実行では観測を保持し、`replay`で送信を再実行しません。`replay`が成功したことは、実サーバーへもう一度送信できたという意味ではありません。通信をやり直す操作は`fresh`で明示し、相手の`idempotency key`などの重複防止も検討します。

同期client、外部worker、VM task、GUIのevent loopの責任を分けると、途中取消やshutdownを扱いやすくなります。
閉じたnative socketはrevertで復活しません。

第21章 SQLite・PostgreSQL・transaction

接続と所有権

std.dbはSQLiteとPostgreSQLを提供します。db、external、tasksを許可し、external内で接続Taskを作り、外でawaitします。DbConnection、DbStatement、DbCursorはscope所有のaffine resourceです。

SQLiteのパスはroot相対です。`:memory:`は独立したmemory DBを作ります。終了すればその内容は保持されません。file名を指定したDBは、REWINDを終了しても保存した内容が残ります。SQLiteへ書くことはFile.writeTextのpending操作ではありません。

実習のSQLite例ではtable作成、parameter付きINSERT、cursorでSELECT、cleanupの待機までを一つのプログラムにまとめています。二重のResult、所有権を取り出すtake、externalの外でのawaitを順に追ってください。

parameter binding

SQLを文字列連結して値を埋め込むのではなく、DbValueの列をparameterとして渡します。SQLiteとPostgreSQLではplaceholderの書き方などをbackendに合わせます。型付きbindingでも、任意のSQL文が業務上安全になるわけではありません。table名や列名の動的指定など、値bindingで代替できない場所は別途検査します。

NULLはDbValue::Nullです。整数0や空Stringと同じではありません。db.integerはInt cell、db.floatingはFloat cellを要求し、暗黙にStringをparseしません。型が違えばDbTypeになります。

列名で読む場合、存在しない列や重複した名前を勝手に選びません。cellByNameとcolumnの失敗を処理します。大量の行ではcursorとbounded batchを使い、全件をListへ集める必要があるか考えます。

VMとDBのcommitは別

VMのcommitはcheckpoint保存です。DB transactionのcommitはDBへの確定です。revertはDB transactionのrollbackではありません。DBで保存済みの行をVMのrevertで削除しようとしてはいけません。

操作	対象
commit label	VMの作業状態
revert label	VMの保存状態への復元
publish	pending仮想I/O
DB begin / commit / rollback API	DBのtransaction
cursor / connection close	physical資源の終了

DB transactionを明示して複数SQLをまとめます。executeManyは暗黙のtransactionを作らないため、途中失敗前の行が適用済みの場合があります。atomicにしたい処理はtransactionへ入れ、成功時にcommit、失敗時にrollbackしてください。

既に閉じた接続はrevertで復活しません。保存していたtokenのVM情報が戻ることと、physical接続が生きていることを混同しないでください。

PostgreSQLとcredential

postgres接続では、Secretの接続文字列をcredentialsでopaque aliasへ登録し、aliasで開きます。TLSのchainとhostnameを検証します。certificate指定の形式はDB APIに従い、HTTPのCA指定と同じ形式だと仮定しないでください。

DbErrorはcode、phase、sqlCode、sqlStateを持ちます。SQLSTATEを残せば、制約違反と接続失敗を区別できます。自動reconnect、failover、書き込みretryを勝手に行うAPIではありません。送信後の結果不明では外部状態の確認が必要です。

cleanupを確認する

scope終了で資源の解放を開始できます。waitForCleanupでnative cleanup完了とrollback/closeの失敗を待ちます。cursorを明示closeし、完了を確認してからconnectionを再利用する設計も重要です。cancelしたからDBで一切処理されていないとは判断しません。

第22章 JSON・CSV・設定データ

JSONの型

JsonはNull、Bool、Int、Float、Text、Array、Objectのvariantを持ちます。parseはResultを返します。JSONとして正しいことと、アプリケーションが期待するschemaであることを分けて検査します。

REWIND・実行例 json

```
import std.json as json;
fn take<T,E>(value:Result<T,E>)->T effects {} {
  match move value {Ok(v)=>{return move v;},Err(_)=>{panic("json failed");}}
}
let value=take(json.parse("{\"name\":\"Alice\",\"score\":42}"));
match json.get(value,"score") {
  Some(item)=>{assert_eq(json.integer(item),Some(42));},
  None=>{panic("score is required");}
}
assert_eq(json.get(value,"absent"),None);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

json.integerはInt variantを返し、数値に見えるStringを自動parseしません。json.floatingもFloat variantを区別します。型が違う場合をNoneとして扱う便利さと、詳細なschemaエラーを返したい要求を区別してください。

大きなJSONとCSV

std.jsonStreamはchunkを供給してeventを読みます。構造開始・終了、key、text、numberなどのeventを扱います。number tokenを必要なexact型へ変換する段階は別に設計します。

std.csvはboundedなデータの変換、std.csvStreamは増分処理に使えます。CSVの引用符や改行を考えず単純にcommaでsplitする方法は、一般的なCSV parserではありません。chunkの境目に引用符やUTF-8文字が来ても状態を保持するparserを選びます。

増分処理にしても、最後に全行をListへ保持すれば全体の保存メモリが必要です。一行ずつ検査・集約・書込みできる設計では、その分を抑えられます。途中の結果をpublishするなら、後続の失敗で取り消せる範囲を明示します。

exactな値を保存する

BigIntやDecimalをFloatへ変換してからJSON numberへ保存すると、精度を失う場合があります。対応するtoJson/fromJsonの契約に従い、正確なtext表現で往復します。日時もtimezoneと精度を明示して保存してください。

config、path、argsは設定や名前の処理を補助します。pathを組み立てたことが、File APIのroot外アクセス許可を与えるわけではありません。検査と権限は別です。

第23章 BigInt・Decimal・日時

大きな整数

BigIntはFloatを介さず正確な整数を扱います。parse、format、算術、bit操作などはstd.bigintを使います。native値は不変で、Map keyやcheckpointにも対応します。容量、work、memory予算は適用されます。

REWIND・実行例 bigint

```
import std.bigint as big;
fn take<T,E>(value:Result<T,E>->T effects {} {
  match move value {0k(v)=>{return move v;},Err(_)=>{panic("bigint failed");}}
}
let two=take(big.fromInt(2));
let value=take(big.pow(&two,100));
assert_eq(take(big.format(&value,10)), "1267650600228229401496703205376");
Out.println(take(big.format(&value,10)));
publish;
```

▼ 標準出力（検証済み）

1267650600228229401496703205376

BigIntは無限メモリを提供する型ではありません。非常に大きいべき乗の結果を求めるより、剰余だけが必要なならmodular演算を選ぶ方が適切な場合があります。

十進の丸め

Decimalは十進数としてprecisionとscaleを管理し、丸め方を明示します。Floatの0.1+0.2が必ず十進の0.3と同じ表現になるわけではないため、金額などの用途では丸め仕様を先に決めます。

REWIND・実行例 decimal

```
import std.decimal as decimal;
fn take<T,E>(value:Result<T,E>->T effects {} {
  match move value {0k(v)=>{return move v;},Err(_)=>{panic("decimal failed");}}
}
let a=take(decimal.parse("0.1"));
let b=take(decimal.parse("0.2"));
let sum=take(decimal.add(&a,&b,28,DecimalRounding::Exact));
assert_eq(take(decimal.format(&sum)), "0.3");
let tie=take(decimal.parse("2.5"));
let rounded=take(decimal.quantize(&tie,0,28,DecimalRounding::HalfEven));
assert_eq(take(decimal.format(&rounded)), "2");
```

▼ 標準出力なし・正常終了を検証済み

空（assertを含む処理は正常終了）

HalfEvenはtieを偶数側へ丸めます。Exactは必要な丸めを黙って行うmodeではありません。Decimalの数値比較とscaleを含む保存表現は別なので、`1.0` と `1.00` の表示桁を保存したい用途では契約を確認します。dbDecimalはDBとの精度境界を扱います。

時刻と暦

Instantは時点、Durationは時間差です。datetimeの変換・算術はpureなResult APIです。現在時刻を観測するclock.nowはexternal、clockを要求します。「日時をformatする」と「現在時刻を読む」は効果が違います。

timezoneを明示します。DSTの戻りで同じ現地時刻が二度現れる場合はEarlier/Later/Rejectを選び、存在しない現地時刻のgapはエラーです。Calendarへ年・月・日を入れればあらゆるzoneで一意にInstantへ変換できるわけではありません。

SDK同梱IANAデータ版はdatetime.databaseVersionで確認できます。DBに保存する場合、相手のtimestamp精度で丸められるか、timezone情報をどこへ保持するかを決めます。dbDatetimeの変換契約も参照してください。

第24章 数値配列・線形代数・統計

Listとnative配列

List<Float>は汎用collectionです。FloatArray/IntArrayはshape、rank、strideを持つtyped native storageです。大量の数値を処理するにはnumericの配列APIを使います。

REWIND・実行例 numeric-owner

```
import std.numeric as numeric;
fn take<T,E>(value:Result<T,E>)->T effects {} {
  match move value {Ok(v)=>{return move v;},Err(_)=>{panic("numeric failed");}}
}
let shape=List<Int>();shape.add(3);
let zero=take(numeric.zerosFloat(&shape));
let data=take(numeric.affine(&zero,1.0,0.25));
let values=take(numeric.valuesFloat(&data));
values.set(0,1.0);
assert_eq(values.get(0),1.0);
let index=List<Int>();index.add(0);
assert_eq(take(numeric.getFloat(&data,&index)),0.25);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

v2.0.0で返却Listは更新できます。元配列は0.25のままです。valuesFloatは明示的にListを作る境界であり、native配列のまま計算が続けられるなら不要な変換を避けます。

shapeとview

reshape、transpose、slice、broadcastはshapeとstrideの契約を持ちます。viewは生成時の版を保持します。broadcastはreadonlyです。withFloat/withIntは更新された新しい値を返すので、利用したい変数へ代入します。元変数を変更する副作用関数だと思って結果を捨てないでください。

numericIndexはlogicalな行優先の一つのindexで要素へアクセスします。転置や逆順sliceでもlogical順で読むため、physical storageの並びをそのままindexと考えないようにします。

線形代数

matmul、LU solve、列pivot QR、最小二乗、対称固有値分解などを提供します。戻るshape、rank、収束、許容誤差を調べて使います。単にResultがOkだから、入力行列が望んだconditionを持つと保証されるわけではありません。

Floatの加算順序は結果の末尾のbitへ影響します。実数の数学的等式とFloatの完全一致を混同しないでください。誤差判定はabsoluteとrelativeを用途に応じて選び、NaNやInfを含めて入力を検査します。

統計と分布

sum、mean、variance、共分散、相関、分位点、histogramがあります。varianceのddofや分位点の規則はAPI契約へ合わせます。空配列、定数列、非finite値の扱いを確認してください。

OnlineMomentsは全サンプルを保存せずに集約します。random効果のdistributionsはVMのcheckpointed generatorから分布を生成します。再現可能な乱数状態と外部から観測したデータを混同しないでください。

sparse・FFT・協調処理

sparseは疎行列、fftは変換の補助です。計算量だけでなくデータの表現変換と一時領域も費用に含めます。大きい計算をGUIと同時に行うなら、対応するAsync moduleを選び、TaskErrorも処理します。同期版がすべて廃止されたわけではありません。

第25章 自動微分・最適化・モデル保存

計算をtapeへ記録する

`std.autodiff`はtape上へparameterと演算を記録し、lossからgradientを求めます。constantとparameterを区別し、backward後に必要なNodeのgradientを照会します。数学的に微分可能なことと、任意のREWINDコードをそのまま自動微分できることは違います。対応する演算をtape APIで表現します。

同じparameter名を使う場合の契約、shape、broadcast軸のgradient集約を確認します。shapeが合っているだけで、データの行・列の意味まで検査されるわけではありません。

最適化の状態

`std.optimize`はSGD/Adam等の更新を補助します。Adamの状態にはstepとmomentが含まれます。重みだけを戻してoptimizer状態を戻さないUndoは、同じ学習状態へ戻ったことになりません。必要なら両方をcheckpointで保持します。

`std.training`はsample数で重み付けしたbatch meanやglobal-norm clippingを組み合わせます。大きさが異なるbatchのmeanを単に等重みで平均すると、sample単位のmeanとは違う結果になります。

同期と協調

`autodiff.backward`は同期です。`autodiffAsync.backward`はtapeを所有して協調的に逆伝播します。Nodeを保持し、返却Gradientsから照会できます。準備、走査、重いkernelの途中で制御を渡す契約があります。所有するtapeをmoveするので、同じ可変tapeを別taskから更新する設計ではありません。

forward側にも`autodiffForwardAsync`があります。Asyncという名前だけで全演算が別OS threadへ並列化されるとは考えないでください。取消、部分状態、history memory、native workを含めて設計します。

モデル保存

`std.models`で名前付き重みをまとめ、encode/decode、save/loadします。model.rwmのようなfileへのsaveは仮想File操作とpublishの境界を持ちます。保存形式の検証を省略せず、読み込んだ重みの名前とshapeを利用側でも確認します。

毎反復のtapeと途中gradientを不要になったら手放します。全epochでcheckpointを保持すれば、正当に全過去状態のメモリが残ります。保持したい最良モデルと、毎回の途中計算を区別して管理すると必要領域を減らせます。

小さな学習から始める

最初は既知の小さな線形関係で、lossが下がること、求めた係数が期待に近いこと、保存して再読み込みしても同じ重みであることを確認します。次にbatch分割、丸め、clip、optimizer状態の復元を追加します。結果だけでなく、使ったデータと設定も残すと調査しやすくなります。

第26章 アルゴリズムの実装と選択

sortとsearch

`std.sort.stable`は比較が負/0/正を返す安定merge sortです。入力を借用して新しいListを返します。`inPlace`は更新する版です。`tryStable`はcallbackのErrを返し、入力を保持します。どのAPIでも途中失敗時に入力完全に戻ると一括保証するものではありません。

REWIND・実行例 sort

```
import std.sort as sort;
let values=List<Int>();
values.add(8);values.add(2);values.add(5);values.add(2);
let ordered=sort.integers(&values);
assert_eq(ordered.get(0),2);
assert_eq(ordered.get(1),2);
assert_eq(ordered.get(2),5);
assert_eq(ordered.get(3),8);
assert_eq(values.get(0),8);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

比較の整合性が必要です。`a<b` と `b<a` を両方trueにする比較関数では正しい順序が作れません。`search`の二分探索を使う場合、同じ比較規則で整列済みであることを事前条件にします。

最短経路

`graph.create`は頂点と辺容量を指定します。`add`は有向辺を追加します。無向graphなら必要な向きの辺を明示します。

REWIND・実行例 graph

```
import std.graph as graph;
fn take<T,E>(value:Result<T,E>->T effects {}) {
  match move value {Ok(v)=>{return move v;},Err(_)=>{panic("graph failed");}}
}
let g=take(graph.create(4,4));
take(graph.add(&mut g,0,1,5));
take(graph.add(&mut g,1,2,2));
take(graph.add(&mut g,0,2,10));
let distance=take(graph.dijkstra(&g,0));
assert_eq(distance.get(0),Some(0));
assert_eq(distance.get(2),Some(7));
assert_eq(distance.get(3),None);
let hops=take(graph.bfs(&g,0));
assert_eq(hops.get(2),1);
assert_eq(hops.get(3),-1);
```

▼ 標準出力なし・正常終了を検証済み

空 (assertを含む処理は正常終了)

Dijkstraは重みを使い、BFSは辺数を使います。ここでは0→2の直接辺があるため、BFSの距離は1です。Dijkstraの未到達はNone、BFSの未到達は-1です。戻り値の単位と表現が違います。

graphは最大65,536頂点/辺、ancestorsは最大4096頂点の根付き木など、関数ごとの上限があります。大きいgraphにはnative IntArrayを使うgraphLargeがあり、頂点/辺は最大1,048,576です。上限内でも予算は別に必要です。

graphの他の処理

topologicalはcycleをErrにします。bellmanFordは到達可能な負cycleを検査します。componentsは強連結成分の密なidを返します。minimumForestは各有向辺を無向候補として解釈するKruskalです。二重に入れた辺も候補です。ancestors/lcaは根付き木的前提を検査します。

最大流はflow、二部matchingはmatchingです。flowのresidual状態はVMのcheckpoint対象です。最大matchingの結果には最小vertex coverも含まれます。目的が違うため、単純なunion-findで代替できるとは限りません。

文字列・区間・幾何

stringSearchにはKMP/Z、trieにはprefix検索、suffixにはsuffix関連処理があります。fenwick、segment、lazySegmentは更新と区間queryの違いに合わせて選びます。geometryでは座標範囲とoverflow、境界点の扱いを確認します。dp、matrix、number、modularも必要なprimitiveを提供します。

教科書上の $O(n \log n)$ は主に比較やprimitive操作数です。VM collectionのページ探索・COW費用までゼロになるわけではありません。大きいデータでは入力scanner、native配列、不要なfreezeやcollectの回避も効果があります。

第27章 診断・テスト・record / replay

エラーはまずcodeと位置

診断にはcode、source、line、column、taskId、causes、frames、hintsなどがあります。textでは内側から外側へcallerを表示します。位置は1始まり、columnはUnicode scalarです。LSPへはUTF-16位置へ変換されます。

callerは最大32件で、省略を示します。local値、引数、closure captureをbacktraceへ追加しません。source-free成果物でもsource名と位置は保持します。native内部のすべての関数呼出しまで完全なstackを保証するものではありません。

code	最初に確認する点
DivisionByZero	divisor
IntegerOverflow	Intの範囲、checked API
IndexOutOfBounds	負index、len以上、空collection
UnknownName	scope、綴り、import
MissingEffect / EffectMissing	関数宣言と実行許可
InvalidContinuation	checkpointと関数・scopeの寿命
ExecutionBudgetExceeded / NativeWorkBudgetExceeded	停止条件、入力量、費用
TaskDeadlock	waitGraph、通知・close・joinの順
ExternalStateConflict	Host fileが途中で変更されたか
PublishPartiallyApplied	phaseとapplied、実際の外部状態

JSON診断はstderrへ出し、applicationのstdoutと分けます。

CLIコマンド
<pre>rewind run main.rw --diagnostic-format json</pre>

診断JSONにはexit_status、message、diagnostic、publish_failure、retryable、retry_hintなどがあります。過去のrecordでは追加fieldが省略される場合があります。codeを見ずにmessageの一部だけで分類しないようにします。

子taskの失敗

std.taskError.describeは元の診断とtyped budget、wait graphを所有するFailure envelopeへ変換します。診断を持つFailedは元codeを保持し、取消、timeout、native work超過を区別します。

asStdはcodeと失敗診断の行をStdErrorへ投影します。全文と原因treeを保存する投影ではありません。詳細を必要とするなら先にdescribeしてください。具体的なbudget失敗をすべて「NumericTask」として処理する旧前提は使わないでください。

小さなテスト

assertとassert_eqでpureな関数を確認します。正常な入力だけでなく、空、最小値、最大値、境界の一つ外、負数、Unicode、失敗時に入力が保持されるかを考えます。可変処理では「戻り値が正しい」と「入力を壊していない」を分けて検査します。

projectならrewind testを使えます。recordは入力・scheduler選択・観測・失敗位置などを再現する調査手段です。unit testをすべてrecordへ置き換える必要はありません。

CLIコマンド

```
rewind run main.rw --record trace.json
rewind run main.rw --record trace.json --record-mode compact
rewind replay trace.json --root .
```

既定debug記録はstep移動に対応し、compactは詳細step履歴を保持せず命令数・順序digest・観測・最終状態を照合します。長い計算でdebug indexの上限に当たる場合、compactの保証を理解して選びます。

replayは外部処理の再実行ではない

通信とDBの記録済み観測を使い、送信やDB書込みを再実行しません。fixture GUIもOS画面を開きません。secret入力は再注入が必要な場合があります。実サービスとの接続が不要な再現は有用ですが、実サービスの現在の状態を検査する試験とは区別します。

第28章 速度・メモリ・実行予算

二種類の仕事量

VM命令のstepsと、native処理のworkは別です。既定はそれぞれ1,000,000、task-stepsは100,000です。実行量に合わせて明示します。

CLIコマンド

```
rewind run main.rw --steps 10000000 --native-work 100000000 --task-steps 1000000
rewind run main.rw --history-memory 128MiB --history-storage 512MiB --spill-threshold 32MiB
```

historyの値はbytes、KiB、MiB、GiBを指定できます。spill thresholdの0は強制spill用です。source内のruntime設定もありますが、処理済みの累積予算をrevertや設定変更で回復できません。

予算はlogical work/allocationの契約で、物理CPU時間やRSSを厳密に測る値ではありません。Linuxではcompiler/runtimeへ既定2GiBのaddress-space上限がありmemory-mibで指定できます。WindowsではそのOS上限を提供せず、明示memory-mibはunsupportedです。両OSでVMのlogical予算は使えます。

履歴とGC

checkpointが保持するメモリは生きたデータです。GCは、利用者が保存した過去状態を勝手に捨ててメモリを減らしません。不要なcheckpointをdropし、現在の変数やTaskにも不要な値を残さないようにします。

List/Map/heapは永続構造で、共有部分はruntime内で重複計上を抑え、変更部分と独立した所有元は計上します。GCは参照を含まない共有部分を省略し、同じ共有nodeを一回の探索で重複走査しません。checkpointのheapはそれぞれ保持されます。

Bytesはshared native bufferの実際のcapacityと管理情報をruntimeごとに計上します。sliceが小さくても大きいbufferの参照を残す場合があります。弱参照索引はbufferを保持せず、GC・profile・上限判定で解放済み項目を整理します。外部workerがまだ保持するbufferも生きています。

何から最適化するか

まず入力量とアルゴリズム、次に中間collectionの数、snapshotと履歴、native変換境界、最後に小さな式を調べます。何回もvaluesFloatへ変換するよりnative配列のまま計算する、全行のCSVを保持せず集約する、不要なcheckpointを落とす、といった変更は効果が分かりやすくなります。

速さのためにすべてをexternal liveへ出すと、再現性の境界が変わります。性能変更も効果・所有権・失敗契約を守ってください。測定時は同じ入力・予算・記録modeで比較します。

compilerにも上限がある

sourceは1MiB/module、module数は256などの上限があります。generic展開、type展開、dependency解決、effect推論にも有限予算が適用されます。allocatorやOS強制終了をResultへ必ず変換できる保証はありません。大量sourceを一つのmoduleへ詰めれば問題が消えるとも限りません。

第29章 ツールと日々の開発

よく使うコマンド

目的	コマンド
版・総合help	rewind --version / rewind --help
source実行	rewind run main.rw
compile	rewind compile main.rw / rewindc main.rw
artifact実行	rewind run main.rwc / rewind main.rwc
project検査	rewind check --root DIR
project試験	rewind test --root DIR
project成果物	rewind build --root DIR --output app.rwc
整形	rewind fmt main.rw
整形検査	rewind fmt main.rw --check
module API生成	rewind doc FILE --root DIR --output FILE
エディタ接続	rewind lsp --root DIR
記録再生	rewind replay TRACE --root DIR

formatterは既存改行を保ちつつindentを整えます。任意の一行コードをすべて文単位に分割するformatterではありません。docには対象sourceを指定します。全subcommandで `--help` が同じように受理されると仮定せず、総合helpと主要commandのhelpを利用します。

LSP

stdio LSP clientをエディタへ設定します。診断、補完、hover、定義・参照、rename、signature help、quick fix、整形などを提供します。エディタ側でstdio serverを起動できる設定が必要です。runtime診断と編集中の診断は、利用できる情報とタイミングが違います。

rename後にCLIでcheckし、公開APIへ影響した場合は利用側も検査します。quick fixは提案であり、アプリケーションの目的まで保証しません。

配布前の確認

source-freeで実行するなら、ソースがなくても動くか、必要なeffectを許可しているか、application引数を受け取れるか、診断位置が残るかを確認します。記録も実行するならexact compilerを揃えます。

GUI fixtureとnative画面、DB memoryと実file、local HTTPとTLSなど、異なる層の試験を混同しないでください。公開したartifactを更新した場合、元sourceとcompiler版、API契約の対応を残すと問い合わせに答えやすくなります。

第30章 実習：小さなアプリケーションへ進む

実習の順序

最初は計算とassertだけで作り、次に入力、エラー表示、保存、GUIを加えます。一度に通信とDBとGUIを導入するより、失敗の原因を絞れます。以下の完全な実習コードは、それぞれ独立して試せます。

カウンターGUIでは、AddとUndoを繰り返し、戻したmodelをpublishします。SQLiteではparameter付きINSERTとcursorを使います。HTTPではURLを引数に取り、二重のResultを検査します。線形方程式ではnative配列を作り、結果とcheckpointによる復元を確認します。

例を実用化するとき

例中のtakeは失敗するとpanicします。実用アプリではErrを分類して表示し、必要なcleanupやrollbackを行う関数へ変えてください。HTTPの引数がない場合、利用説明を出します。DBのCREATE TABLEはfile DBで再実行すると既存tableに対して失敗するため、migrationや初期化規則を別に設けます。

GUIのUndoはfileやDBに保存済みの結果を自動で戻しません。保存後に編集内容を戻した場合、表示、現在model、保存済み状態の違いを利用者に伝える必要があります。保存をUndoしたいなら、DBの新しいtransaction等で補償する処理を明示します。

数値処理では、例の小さい行列から、大きいshape、悪条件、非finite、work不足へ試験を増やします。エラーを隠して0や空配列に変えると、呼出し元が誤った結果を正常と扱う可能性があります。

実習：Undo付きカウンター

Add、Undo、入力cursor、再表示の完全なevent loop。gui許可が必要です。

```

import std.gui as gui;
import std.number as number;
fn decimal(value:Int)->String effects {} {match number.formatInt(value,10){Ok(text)=>{return
text;},Err(e)=>{panic(e.code);}}}
fn checked(result:Result<Unit,gui.GuiError>)->Unit effects {} {
  match result {Ok(_)=>{},Err(e)=>{panic(e.code);}}
}
match gui.window("REWIND counter",360,220) {
  Err(e)=>{panic(e.code);},
  Ok(view)=>{
    var count=0;
    checked(gui.label(&mut view,"count","0",20,20,300,40));
    checked(gui.button(&mut view,"add","Add",20,80,140,40));
    checked(gui.button(&mut view,"undo","Undo all",180,80,140,40));
    checked(gui.checkbox(&mut view,"enabled","Enable counting",true,20,140,300,40));
    var running=true;
    commit ready;
    checked(gui.present(&view));publish;
    while running {
      match gui.nextEvent() {
        Err(e)=>{panic(e.code);},
        Ok(event)=>{
          match gui.dispatch(&mut view,event) {
            Some(gui.Action::Activate(id))=>{
              if id=="add" {
                if gui.checked(&view,"enabled")==Ok(true) {count+=1;}
              } else if id=="undo" {
                revert ready;
                gui.continueInput();
              }
              checked(gui.setText(&mut view,"count",decimal(count)));
            },
            Some(gui.Action::Close)=>{running=false;},
            _=>{}
          }
        }
      }
      if running {checked(gui.present(&view));publish;}
    }
    checked(gui.close());Out.println(count);publish;
  }
}

```

必要な許可: gui / GUI検証は入力fixtureを使用。通常実行にはnative表示環境が必要。

▼ 標準出力（検証済み）

1

実習：SQLiteを読み書きする

独立したmemory DBへparameter付きINSERTを行い、cursorを読み、cleanup完了を待ちます。external,db,tasksを許可します。


```

import std.db as db;
fn take<T,E>(value:Result<T,E>)->T effects {} {
  match move value {Ok(item)=>{return move item;},Err(_)=>{panic("database operation failed");}}
}
var opening:Option<Task<Result<DbConnection,DbError>>>=None;
external {opening=Some(db.sqlite(":memory:",false,5000));}
match opening {
  None=>{panic("missing connection");},
  Some(task)=>{
    let connection=take(take(await task));
    var writing:Option<Task<Result<Int,DbError>>>=None;
    external {writing=Some(db.execute(&mut connection,"CREATE TABLE scores (name TEXT, score
INTEGER)",freeze(List<DbValue>()),5000));}
    match writing {Some(task)=>{take(take(await task));},None=>{panic("missing write");}}
    let values=List<DbValue>
();values.push(DbValue::Text("Alice"));values.push(DbValue::Int(42));
    external {writing=Some(db.execute(&mut connection,"INSERT INTO scores VALUES (?1, ?
2)",freeze(move values),5000));}
    match writing {Some(task)=>{Out.println(take(take(await task)));},None=>{panic("missing
insert");}}
    var querying:Option<Task<Result<DbCursor,DbError>>>=None;
    external {querying=Some(db.query(&mut connection,"SELECT name, score FROM
scores",freeze(List<DbValue>()),5000));}
    match querying {
      None=>{panic("missing query");},
      Some(task)=>{
        let cursor=take(take(await task));
        var reading:Option<Task<Result<DbBatch,DbError>>>=None;
        external {reading=Some(db.next(&mut cursor,64,1048576,5000));}
        match reading {None=>{panic("missing read");},Some(task)=>{let
batch=take(take(await
task));Out.println(take(db.text(batch.rows.get(0),0)));Out.println(take(db.integer(batch.rows.get(0
),1)));}}
      }
    }
  }
}
var cleanup:Option<Task<Result<Unit,DbError>>>=None;
external {cleanup=Some(db.waitForCleanup(5000));}
match cleanup {None=>{panic("missing cleanup");},Some(task)=>{take(take(await task));}}
publish;

```

必要な許可: external,db,tasks

▼ 標準出力（検証済み）

```

1
Alice
42

```

実習：HTTP client

URLを--以降の引数に渡します。external,network,tasksを許可してください。通常の外部送信は取り消せません。

```

import std.http as http;
import std.bytes as bytes;
let arguments=Args.all();
if arguments.len()!=1 {Err.println("usage: rewind run main.rw --allow-effects
external,network,tasks -- URL");publish;}
else {
  var pending:Option<Task<Result<HttpResponse,HttpError>>>=None;
  external {pending=Some(http.get(arguments.get(0),10000,65536));}
  match pending {
    None=>{panic("missing HTTP task");},
    Some(task)=>{
      match await task {
        Err(_)=>{Err.println("HTTP task cancelled");},
        Ok(response)=>{
          match response {
            Err(e)=>{Err.println(e.code+" (" +e.phase+""));},
            Ok(r)=>{
              Out.println(r.status);
              match bytes.decode(r.body) {Ok(text)=>{Out.println(text);},Err(e)=>
{Err.println(e.code);}}
            }
          }
        }
      }
    }
  }
  publish;
}
}

```

必要な許可: external,network,tasks / 引数: -- URL (検証時はローカルHTTP server)

▼ 標準出力 (検証済み)

```

200
book test

```

実習：連立方程式とcheckpoint

native行列と右辺を作りsolveします。更新は返却値を代入し、checkpointへ戻した値も確認します。

```
import std.numeric as numeric;
fn take<T,E>(value:Result<T,E>->T effects {} {
  match move value {Ok(item)=>{return move item;},Err(_)=>{panic("numeric operation failed");}}
}
let shape=List<Int>();shape.push(2);shape.push(2);
let values=List<Float>();values.push(0.0);values.push(2.0);values.push(1.0);values.push(3.0);
var matrix=take(numeric.fromFloat(&shape,&values));
let rhsShape=List<Int>();rhsShape.push(2);
let rhsValues=List<Float>();rhsValues.push(4.0);rhsValues.push(7.0);
let rhs=take(numeric.fromFloat(&rhsShape,&rhsValues));
let answer=take(numeric.solve(&matrix,&rhs,0.000000000000001));
let index=List<Int>();index.push(0);
Out.println(take(numeric.getFloat(&answer,&index)));
index.set(0,1);
Out.println(take(numeric.getFloat(&answer,&index)));
publish;
commit saved;
let cell=List<Int>();cell.push(0);cell.push(0);
matrix=take(numeric.withFloat(&matrix,&cell,99.0));
Out.println(take(numeric.getFloat(&matrix,&cell)));
publish;
revert saved;
let oldCell=List<Int>();oldCell.push(0);oldCell.push(0);
Out.println(take(numeric.getFloat(&matrix,&oldCell)));
publish;
```

▼ 標準出力（検証済み）

```
1
2
99
0
```

実習：Regexのcaptureと継続検索

byte区間をtextへ変換し、captureと次のcursorを使います。Regexをcheckpointで戻します。

```
import std.regex as regex;
fn take<T,E>(value:Result<T,E>)->T effects {} {match move value{Ok(item)=>{return move
item;},Err(_)=>{panic("regex operation failed");}}}
var pattern=take(regex.compile("(?P<name>[A-Za-z]+)-([0-9]+)"));
let input="Alice-42 Bob-7";
match take(regex.find(&pattern,&input)){None=>{panic("missing match");},Some(found)=>{
  let span=regex.span(&found);Out.println(take(regex.text(&input,span)));
  match take(regex.group(&found,1)){None=>{panic("missing name");},Some(name)=>
{Out.println(take(regex.text(&input,name)));}}
  match regex.next(&found){None=>{panic("missing cursor");},Some(cursor)=>{match
take(regex.findFrom(&pattern,&input,cursor)){None=>{panic("missing next match");},Some(next)=>
{Out.println(take(regex.text(&input,regex.span(&next))))};}}}
}}
commit original;
pattern=take(regex.compile("never"));Out.println(take(regex.isMatch(&pattern,&input)));publish;
revert original;Out.println(take(regex.isMatch(&pattern,&input)));
let names=take(regex.groupNames(&pattern));Out.println(names.len());
let copy=take(regex.compile("(?P<name>[A-Za-z]+)-([0-9]+)"));let index=Map<Regex,Int>
();index.set(copy,1);match index.get(pattern){Some(value)=>{Out.println(value==1);},None=>
{Out.println(false);}}publish;
```

▼ 標準出力（検証済み）

```
Alice-42
Alice
Bob-7
false
true
3
true
```

実習：DSTの二つの時点

同じ現地時刻をEarlier/Laterで解決し、差分と明示timezone変換を確認します。

```
import std.datetime as datetime;
fn take<T,E>(value:Result<T,E>->T effects {}) {match move value{Ok(item)=>{return move
item;},Err(_)=>{panic("datetime operation failed");}}}
let wall=datetime.Calendar(2024,11,3,1,30,0,123456789);
var first=take(datetime.resolve(move wall,"America/New_York",datetime.Ambiguity::Earlier));
let
second=take(datetime.resolve(datetime.Calendar(2024,11,3,1,30,0,123456789),"America/New_York",datet
ime.Ambiguity::Later));
let distance=take(datetime.difference(&second,&first));
Out.println(take(datetime.durationSeconds(&distance)));
commit original;
first=take(datetime.add(&first,&distance));
Out.println(take(datetime.format(&first)));
publish;
revert original;
Out.println(take(datetime.format(&first)));
let local=take(datetime.calendar(&first,"Asia/Tokyo"));
Out.println(local.offsetSeconds);
let index=Map<Instant,String>();index.set(first,"same instant");
match index.get(take(datetime.parse("2024-11-03T05:30:00.123456789Z"))){Some(text)=>
{Out.println(text);},None=>{panic("missing instant");}}
publish;
```

▼ 標準出力（検証済み）

```
3600
2024-11-03T06:30:00.123456789Z
2024-11-03T05:30:00.123456789Z
32400
same instant
```

実習：最大流と二部matching

residual状態の変更をrevertしてから再計算します。matchingは別の問題として解きます。

```
import std.flow as flow;
import std.matching as matching;
fn take<T,E>(value:Result<T,E>->T effects {}) {
  match move value {Ok(item)=>{return move item;},Err(_)=>{panic("algorithm failed");}}
}
let network=take(flow.create(3,3));
take(flow.add(&mut network,0,1,4));
take(flow.add(&mut network,1,2,3));
take(flow.add(&mut network,0,2,2));
commit saved;
assert_eq(flow.augment(&mut network,0,2,3),Ok(3));
revert saved;
Out.println(take(flow.augment(&mut network,0,2,99)));
drop saved;
let pairs=List<Tuple<Int,Int>>();pairs.push((0,0));pairs.push((0,1));pairs.push((1,0));
let result=take(matching.maximum(2,2,&pairs));
Out.println(result.size);
publish;
```

▼ 標準出力（検証済み）

```
5
2
```

実習：完了したTaskを選ぶ

selectReadyは完了indexを返します。結果そのものは元Taskをawaitして取り出します。

```
import std.task as task;
async fn number()->Int effects {tasks} {task.yieldNow();return 42;}
async fn text()->String effects {} {return "ready";}
let left=number();let right=text();
assert_eq(await task.selectReady(left,right),Ok(1));
match await right {Ok(value)=>{Out.println(value);},Err(_)=>{panic("text task");}}
match await left {Ok(value)=>{Out.println(value);},Err(_)=>{panic("number task");}}
publish;
```

▼ 標準出力（検証済み）

```
ready
42
```

実習：協調的なforward計算

同期と協調版の演算結果、checkpointとTaskの結果を確認します。

```
import std.numeric as n;import std.autodiff as ad;import std.autodiffForwardAsync as forward;import
std.task as task;
fn take<T,E>(r:Result<T,E>->T effects {}) {match move r{Ok(v)=>{return move v;},Err(_)=>
{panic("failed");}}}
let dims=List<Int>();dims.add(65537);let array=take(n.zerosFloat(&dims));
let expectedTape=ad.create();let expectedNode=take(ad.parameter(&mut expectedTape,"x",array));
let work=spawn forward.parameter(ad.create(),"x",array);for i in
0..3{task.yieldNow();}assert(!work.isDone());commit running;
let first=take(take(await
work));assert_eq(first._1,expectedNode);assert_eq(take(ad.value(&first._0,first._1)),array);Out.pri
ntln("finished");publish;
revert running;assert(!work.isDone());let second=take(take(await
work));assert_eq(second._1,expectedNode);assert_eq(take(ad.value(&second._0,second._1)),array);drop
running;Out.println("restored");publish;
for i in 0..8{let pending=spawn
forward.parameter(ad.create(),"cancel",array);task.yieldNow();assert(!pending.isDone());pending.can
cel();match await pending{Err(TaskError::Cancelled)=>{},_=>{panic("partial tape");}}}
Out.println("cancelled");publish;
```

検証予算: steps=10000000, native-work=100000000

▼ 標準出力（検証済み）

```
finished
restored
cancelled
```

実習：logical indexによる数値アクセス

transposeなどのviewでもlogical順で読み、更新した新しい配列を利用します。

```

import std.numeric as n;
import std.numericIndex as flat;
fn take<T,E>(r:Result<T,E>->T effects {}) {match move r{Ok(v)=>{return move v;},Err(_)=>
{panic("numeric failure");}}}
let shape=List<Int>();shape.add(3);shape.add(2);
let values=List<Int>();for i in 0..6{values.add(i);}
let original=take(n.fromInt(&shape,&values));
let axes=List<Int>();axes.add(1);axes.add(0);
var view=take(n.transposeInt(&original,&axes));
assert_eq(take(flat.lengthInt(&view)),6);
assert_eq(take(flat.getInt(&view,1)),2);
assert_eq(take(flat.getInt(&view,4)),3);
commit unchanged;
view=take(flat.withInt(&view,1,99));
assert_eq(take(flat.getInt(&original,2)),2);
assert_eq(flat.getInt(&view,-1),Err(StdError("NumericIndex",0)));
assert_eq(flat.withInt(&view,6,0),Err(StdError("NumericIndex",0)));
Out.println(take(flat.getInt(&view,1)));publish;
revert unchanged;
Out.println(take(flat.getInt(&view,1)));publish;
drop unchanged;
let reversed=take(n.sliceInt(&view,1,2,3,-1));
assert_eq(take(flat.getInt(&reversed,0)),4);
assert_eq(take(flat.getInt(&reversed,5)),1);
let scalarShape=List<Int>();let floats=List<Float>();floats.add(3.5);
let scalar=take(n.fromFloat(&scalarShape,&floats));
assert_eq(take(flat.lengthFloat(&scalar)),1);
let broad=take(n.broadcastFloat(&scalar,&shape));
assert_eq(take(flat.getFloat(&broad,5)),3.5);
assert_eq(flat.withFloat(&broad,0,8.0),Err(StdError("NumericReadOnly",0)));
let changed=take(flat.withFloat(&scalar,0,8.0));
assert_eq(take(flat.getFloat(&scalar,0)),3.5);
assert_eq(take(flat.getFloat(&changed,0)),8.0);

```

▼ 標準出力（検証済み）

99

2

実習：Decimalの比較と保存表現

exactな加算、HalfEven、scaleの異なる値の比較、JSON textへの変換を確認します。


```
import std.decimal as decimal;
import std.json as json;
fn take<T,E>(value:Result<T,E>->T effects {}) {match move value{Ok(item)=>{return move
item;},Err(_)=>{panic("decimal operation failed");}}}
var amount=take(decimal.parse("0.1"));let change=take(decimal.parse("0.2"));
commit
saved;amount=take(decimal.add(&amount,&change,28,DecimalRounding::Exact));Out.println(take(decimal.
format(&amount)));publish;
revert saved;Out.println(take(decimal.format(&amount)));publish;
let tie=take(decimal.parse("2.5"));let
even=take(decimal.quantize(&tie,0,28,DecimalRounding::HalfEven));Out.println(take(decimal.format(&
even)));
let a=take(decimal.parse("1.0"));let b=take(decimal.parse("1.00"));Out.println(a==b);let
values=Map<Decimal,Int>();values.set(a,1);values.set(b,2);Out.println(values.len());
let encoded=take(decimal.toJson(&amount));Out.println(take(json.stringify(encoded)));publish;
```

▼ 標準出力（検証済み）

```
0.3
0.1
2
true
1
"0.1"
```

実習：Unicodeの正規化と履歴

grapheme、NFC/NFD、互換正規化、full casingとcheckpointを確認します。

```
import std.unicode as unicode;
fn take<T,E>(value:Result<T,E>->T effects {}) {match move value{Ok(item)=>{return move
item;},Err(_)=>{panic("unicode operation failed");}}}
let source="é🇯🇵🇸🇯";
Out.println(take(unicode.graphemeCount(&source)));
Out.println(take(unicode.graphemeSlice(&source,1,2)));
var text=take(unicode.normalize(&source,unicode.Form::NFC));
commit composed;
text=take(unicode.normalize(&text,unicode.Form::NFD));
Out.println(take(unicode.isNormalized(&text,unicode.Form::NFC)));
publish;
revert composed;
Out.println(take(unicode.isNormalized(&text,unicode.Form::NFC)));
let compatibility="A①ffi";Out.println(take(unicode.normalize(&compatibility,unicode.Form::NFKC)));
let word="straße";Out.println(take(unicode.upper(&word)));
let clusters=take(unicode.graphemes(&source));Out.println(clusters.len());
publish;
```

▼ 標準出力（検証済み）

```
3
🇯🇵
```

```
false
true
Alffi
STRASSE
3
```

実習：CSVをchunk単位で読む

parserのcursorとchunkを保持し、引用符やUTF-8を含む入力を増分処理します。

REWIND・実行例 csv-stream

```
import std.csvStream as csv;
fn take<T,E>(value:Result<T,E>->T effects {}) {match move value{Ok(item)=>{return move
item;},Err(_)=>{panic("CSV operation failed");}}}
let reader=take(csv.reader());
let input=take(stdEncode("name,note\r\n日本語,\"a\" \"b\" 日 \"r\""));
var offset=0;
while offset<stdBytesLength(input){
  let chunk=take(stdBytesSlice(input,offset,offset+1));
  take(csv.feed(&mut reader,chunk,false));offset+=1;
}
take(csv.finish(&mut reader));
commit rows;
match take(csv.next(&mut reader)){None=>{panic("header missing");},Some(row)=>
{Out.println(row.get(0));Out.println(row.get(1));}}
match take(csv.next(&mut reader)){None=>{panic("data missing");},Some(row)=>
{Out.println(row.get(0));Out.println(row.get(1));}}
publish;
revert rows;
match take(csv.next(&mut reader)){None=>{panic("restored row missing");},Some(row)=>
{Out.println(row.get(0));}}publish;
```

▼ 標準出力（検証済み）

```
name
note
日本語
"a"b
日
name
```

実習：JSONをeventとして読む

構造と値のevent、number token、checkpointによるparser状態の復元を確認します。

```
import std.jsonStream as json;
import std.bigint as bigint;
fn take<T,E>(value:Result<T,E>->T effects {}) {match move value{Ok(item)=>{return move
item;},Err(_)=>{panic("JSON stream operation failed");}}}
let reader=take(json.reader());
let input=take(stdEncode("{\"items\":[1267650600228229401496703205376,2,3],\"name\":"\u672c\"}"));
var offset=0;
while offset<stdBytesLength(input){take(json.feed(&mut
reader,take(stdBytesSlice(input,offset,offset+1)),false));offset+=1;}
take(json.finish(&mut reader));
commit events;
var reading=true;var count=0;
while reading{match take(json.next(&mut reader)){None=>{reading=false;},Some(event)=>{
match event{JsonStreamEvent::Number(raw,_)=>{let
number=take(bigint.parse(raw,10));Out.println(take(bigint.format(&number,10)));count+=1;},JsonStrea
mEvent::Text(value,_)=>{Out.println(value);},_=>{}}
}}}
Out.println(count);publish;
revert events;
match take(json.next(&mut reader)){None=>{panic("checkpoint lost event");},Some(event)=>{match
event{JsonStreamEvent::ObjectStart(position)=>{Out.println(position);},_=>{panic("expected opening
event");}}}
publish;
```

▼ 標準出力（検証済み）

```
1267650600228229401496703205376
2
3
日本
3
0
```

実習：疎行列とFFT

typed配列と疎行列表現を使う計算例です。shapeと戻り値を確認してください。

```

import std.numeric as num;
import std.fft as fft;
import std.sparse as sparse;
import std.sparseAsync as cg;
fn take<T,E>(r:Result<T,E>->T effects {}) {match move r{0k(v)=>{return move v;},Err(_)=>
{panic("unexpected error");}}}
fn real(values:List<Float>->FloatArray effects {}) {let shape=List<Int>
();shape.add(values.len());return take(num.fromFloat(&shape,&values));}
fn ints(values:List<Int>->IntArray effects {}) {let shape=List<Int>
();shape.add(values.len());return take(num.fromInt(&shape,&values));}
fn f2(a:Float,b:Float)->FloatArray effects {} {let l=List<Float>();l.add(a);l.add(b);return
real(move l);}
let cooRows=List<Int>();cooRows.add(0);cooRows.add(0);cooRows.add(1);cooRows.add(1);
let cc=List<Int>();cc.add(0);cc.add(1);cc.add(0);cc.add(1);
let vv=List<Float>();vv.add(4.0);vv.add(1.0);vv.add(1.0);vv.add(3.0);
let ro=ints(move cooRows);let co=ints(move cc);let va=real(move vv);let
matrix=take(sparse.create(2,2,&ro,&co,&va));
let rhs=f2(1.0,2.0);let future=cg.conjugateGradient(matrix,rhs,0.000000001,10);let
solved=take(take(await future));
Out.println(solved.converged);Out.println(solved.iterations);Out.println(solved.residual<0.00000000
1);
let input=f2(1.0,2.0);let imaginary=f2(0.0,0.0);let
spectrum=take(fft.transform(&input,&imaginary,false));let
restored=take(fft.transform(&spectrum.real,&spectrum.imag,true));let
values=take(num.valuesFloat(&restored.real));assert_eq(values.get(0),1.0);assert_eq(values.get(1),2
.0);
let other=f2(3.0,4.0);let convolution=take(fft.convolve(&input,&other));let
cells=take(num.valuesFloat(&convolution));assert_eq(cells.len(),3);Out.println(take(stdNumericMath(
"abs",cells.get(1)-10.0))<0.000000001);publish;

```

▼ 標準出力（検証済み）

```

true
2
true
true

```

実習：協調型の逆伝播

tapeをtaskへ所有させ、返却gradientをNodeで照会します。

```
import std.autodiff as ad;import std.autodiffAsync as jobs;import std.numeric as n;import
std.numericIndex as index;import std.task as task;
fn take<T,E>(r:Result<T,E>)->T effects {} {match move r{0k(v)=>{return move v;},Err(_)=>
{panic("backward failed");}}}
fn model(x:FloatArray)->(ad.Tape,ad.Node,ad.Node) effects {} {var tape=ad.create();let
parameter=take(ad.parameter(&mut tape,"x",x));let square=take(ad.unary(&mut
tape,"square",parameter));let loss=take(ad.sum(&mut tape,square));return (move
tape,parameter,loss);}
async fn marker()->Int effects {tasks}{return 7;}
let one=List<Int>();one.add(1);let values=List<Float>();values.add(0.5);let
scalar=take(n.fromFloat(&one,&values));let shape=List<Int>();shape.add(32769);let
x=take(n.broadcastFloat(&scalar,&shape));
let graph=model(x);let expected=take(ad.backward(&graph._0,graph._2));let work=spawn
jobs.backward(move graph._0,graph._2);let fast=spawn marker();assert_eq(take(await
fast),7);assert(!work.isDone());
commit halfway;
let answer=take(take(await
work));assert_eq(take(ad.gradient(&answer,graph._1)),take(ad.gradient(&expected,graph._1)));Out.pri
ntln(true);publish;
revert halfway;
let restored=take(take(await
work));assert_eq(take(ad.gradient(&restored,graph._1)),take(ad.gradient(&expected,graph._1)));Out.p
rintln(true);drop halfway;publish;
let another=model(x);let cancelled=spawn jobs.backward(move
another._0,another._2);task.yieldNow();assert(!cancelled.isDone());cancelled.cancel();match await
cancelled{Err(TaskError::Cancelled)=>{},_=>{panic("partial gradient escaped");}}
Out.println("backward done");publish;
```

検証予算: steps=10000000, native-work=100000000

▼ 標準出力（検証済み）

```
true
true
backward done
```

実習：協調型の統計集約

同期と協調版のsum/mean/variance、taskの待機とcheckpointを確認します。

```

import std.numeric as numeric;
import std.numericReduceAsync as jobs;
import std.task as task;
fn take<T,E>(value:Result<T,E>->T effects {}) {
  match move value {Ok(v)=>{return move v;},Err(_)=>{panic("numeric operation failed");}}
}
async fn marker()->Int effects {tasks} {return 7;}
let one=List<Int>();one.add(1);
let values=List<Float>();values.add(0.5);
let scalar=take(numeric.fromFloat(&one,&values));
let shape=List<Int>();shape.add(32769);
let repeated=take(numeric.broadcastFloat(&scalar,&shape));
let reversed=take(numeric.sliceFloat(&repeated,0,32768,32769,-1));
let work=spawn jobs.sum(reversed);
let fast=spawn marker();assert_eq(take(await fast),7);assert(!work.isDone());
commit halfway;
assert_eq(take(take(await work)),16384.5);Out.println(true);publish;
revert halfway;
assert_eq(take(take(await work)),take(numeric.sum(&reversed)));Out.println(true);
drop halfway;publish;
assert_eq(take(take(await spawn jobs.mean(reversed))),0.5);
assert_eq(take(take(await spawn jobs.variance(reversed,1))),0.0);
let cancelled=spawn jobs.variance(reversed,0);task.yieldNow();assert(!cancelled.isDone());
cancelled.cancel();match await cancelled{Err(TaskError::Cancelled)=>{},{_=>{panic("partial result
escaped");}}
Out.println("reductions done");publish;

```

検証予算: steps=10000000, native-work=100000000

▼ 標準出力（検証済み）

```

true
true
reductions done

```

実習：native配列のgraph

辺をIntArrayへまとめ、大きいgraphのnative表現で探索します。

```

import std.graphLarge as graph;
import std.numericRange as range;
import std.numericIndex as index;
fn take<T,E>(result:Result<T,E>)->T effects {} {match move result{Ok(value)=>{return move
value;},Err(_)=>{panic("graph failure");}}}
let froms=take(range.integers(0,1,5));
var tos=take(range.integers(1,1,5));
let weights=take(range.integers(1,0,5));
let chain=take(graph.fromEdges(6,&froms,tos,weights));
assert_eq(graph.vertices(&chain),6);assert_eq(graph.edges(&chain),5);
commit saved;
tos=take(index.withInt(&tos,0,5));
let distances=take(graph.bfs(&chain,0));
assert_eq(take(index.getInt(&distances,5)),5);
assert_eq(take(graph.dijkstra(&chain,0)).get(5),Some(5));
assert_eq(graph.add(&mut chain,0,1,0),Err("Capacity"));
Out.println(take(index.getInt(&distances,5)));publish;
revert saved;
assert_eq(take(index.getInt(&tos,0)),1);
let reversed=take(graph.bfs(&chain,5));
assert_eq(take(index.getInt(&reversed,0)),-1);
drop saved;
Out.println(take(index.getInt(&reversed,0)));publish;

```

▼ 標準出力（検証済み）

```

5
-1

```

次に読む場所

目的のmoduleをAPI編で検索し、signatureの引数名と型、所有権、effectsを確認してください。失敗・費用の契約がある場合はそれを読み、source実装で境界の動作を確かめます。本書の文法編とAPI編を行き来して、自分の入力・容量・外部作用の条件に合わせて組み立ててください。

第31章 構文と組み込み操作の早見表

文の骨格

次は構文の対応表です。NAME、TYPE、EXPR等は説明用placeholderです。全体を実行するプログラムではありません。

用途	構文の形
binding	let NAME:TYPE=EXPR; / var NAME=EXPR;
代入	NAME=EXPR; / NAME+=EXPR;
条件	if EXPR {...} else {...}
反復	while EXPR {...} / for NAME in START..END {...}
分岐	match EXPR {PATTERN=>{...},...}
関数	fn NAME(PARAMS)->TYPE effects {...} {...}
非同期関数	async fn NAME(PARAMS)->TYPE effects {...} {...}
型	struct NAME {...} / record NAME {...} / enum NAME {...}
alias	type NAME=TYPE;
generic	fn NAME<T:BOUND>(...)->TYPE effects {...} {...}
module	import std.number as number;
公開	pub fn / pub struct / pub record / pub enum / pub type
共有借用	&EXPR
可変借用	&mut EXPR
所有権移動	move EXPR
snapshot	freeze(EXPR) / thaw(EXPR)
終了	return EXPR; / break; / continue;
履歴	commit NAME; / revert NAME; / resume NAME; / drop NAME;
隔離	branch NAME {...}
公開	publish;
即時外部	external {...} / external fresh {...} / external live {...}
cleanup	defer EXPR; / using NAME=EXPR;

用途	構文の形
task	spawn EXPR / await EXPR

文末にはセミicolon、blockには波括弧を使います。beginは予約名です。API文書の型表記には `Tuple<A,B>` 等が現れますが、本文のtuple注釈では `(A,B)` も使います。関数型の効果表記 `!{}` とcapture表記 `~{Send+Share}` はcompilerのcanonicalな表記です。

よく使う組み込み関数

操作	意味・失敗
assert(Bool)	falseでAssertionFailed
assert_eq(a,b)	不一致でAssertionFailed
panic(String)	通常復帰しない診断
freeze(owner)	Frozen snapshotを作る。深さ・循環・資源制限あり
thaw(snapshot)	独立した可変値を作る
secret(value)	Secretで包む
reveal(secret)	秘密値を取り出す。出力・送信は利用者の責任

freezeで渡す所有権の規則は式と型に応じて検査されます。任意の資源や関数をfreeze可能と解釈しないでください。assertやpanicはResultを返す通常の入力検査APIではありません。

String・数値のmember

呼出し	戻り型
String.byteLen / charLen / utf16Len	Int
String.encodeUtf8()	Bytes
Bytes.decodeUtf8()	Result<String,String>
String.contains / startsWith / endsWith(String)	Bool
String.find(String)	Option<Int>、byte offset
String.split(String)	List<String>
String.slice(Int,Int)	Result<String,String>、byte区間
Bytes.slice(Int,Int)	Result<Bytes,String>

呼出し	戻り型
String.parseInt / parseFloat	Result<Int,String> / Result<Float,String>
Int.format / Float.format	String
Int.toFloatChecked()	Result<Float,String>
Float.toIntChecked()	Result<Int,String>
Float.isFinite / isNaN	Bool

組込みmemberとstd.text/std.bytesのerror型が同じとは限りません。例えばBytes.decodeUtf8はString error、std.bytes.decodeはStdErrorです。変換を混ぜる関数ではEを合わせるか、明示的にerrorを変換します。

REWIND・実行例 builtin-conversion

```

assert_eq("42".parseInt(),0k(42));
assert_eq("abc".parseInt(),Err("InvalidIntOrOverflow"));
assert_eq(Bytes("hello").decodeUtf8(),0k("hello"));
assert_eq(42.toFloatChecked(),0k(42.0));
assert_eq(3.0.toIntChecked(),0k(3));
assert_eq(1.5.isFinite(),true);

```

▼ 標準出力なし・正常終了を検証済み

空（assertを含む処理は正常終了）

コンソール・環境・file

呼出し	戻り型と境界
Out.println(value) / Err.println(value)	Unit、pending
Out.writeBytes(Bytes)	Unit、pending
Out.flush() / Err.flush()	Unit、publishとは区別する
In.readLine()	Option<String>、EOFはNone
In.readSecretLine()	Option<Secret<String>>
In.readChunk(Int)	Option<Bytes>
Args.all()	List<String>、起動引数
Env.get(String) / getSecret(String)	Option<String> / Option<Secret<String>>
Locale.current()	String
Time.now() / Random.next()	Int、観測 / VM乱数

呼出し	戻り型と境界
File.readText(String) / readBytes(String)	Result<String,FileError> / Result<Bytes,FileError>
File.open(String) / openSnapshot(String)	FileHandle
File.writeText(String,String) / writeBytes(String,Bytes)	Unit、 pending
Directory.entries(String)	List<String>、 観測

Time.nowのIntをそのまま任意の単位のtimestampと見なさず、型付き日時にはstd.clock/std.datetimeを利用します。Fileにはcreate、delete、truncate、append、copy、move、Directoryにはcreate、delete、moveもあります。書込みはpublish境界を持ちます。読み取り失敗とfatal診断の扱いは操作ごとに確認します。

Iterator・Result・Task

Iterator adapterのmap/filter/foldは同期callbackです。taskやpublishを伴うcallbackを渡せません。collectはList、enumerateはindexとのtuple、zipは二つの列のtupleを作ります。

Resultにはmap、mapErr、andThen、flattenがあります。andThenは同じerror型のResultを返すcallback、flattenは内外で同じerror型のResultを要求します。すべてのResultを任意のerror型へ自動統合するわけではありません。

async fnはcold Task、外部通信・DBの開始APIはhot Taskです。cold Taskはspawn/await等で開始します。Channelはcapacityを持ち、0ならrendezvousです。send/receiveをawaitし、完了値や閉鎖の失敗を処理します。

第32章 API編の読み方と保証範囲

signatureから呼び方を読む

`pub fn getOrd<K:Ord,V:Share>(values:&Map<K,V>,key:K,fallback:V)->V effects {}` なら、Mapを読む借用を一つ、キーとfallbackを渡し、Vを受け取ります。KにはOrd、VにはShareを要求し、外部効果はありません。

`&mut` の引数があれば変更を許す借用、借用のないowner引数なら所有権の受渡しを確認します。Task<Result<T,E>>を返す外部APIは、呼出し場所とawait場所、外側TaskErrorと内側Eを区別します。

canonicalな返却型と表示用宣言のaliasが違う場合があります。例えばGuiErrorはStdErrorへのaliasです。canonical型は解決後の契約、宣言は利用する名前と引数名を読むために載せています。

モジュールの型と共通型

API編は全moduleの公開関数と、module固有の公開type/alias/traitを掲載します。各moduleへ共通に現れるbuiltin型は共通型付録へまとめ、同じ定義を86回繰り返しません。

機械生成schemaのconstructor fieldはexport検査用の情報です。DbConnection、HttpDownload、TcpSocket等のopaque native resourceについて、constructor fieldだけを根拠に任意の正当な接続を自作できるとは解釈しません。生成・所有・closeは専用APIとchecker/runtimeの契約に従います。内部のprivate fieldの型が表示されても、直接アクセスはできません。

費用と失敗

costとfailureは `@api` 注釈の原文です。英語の原文も保つことで、容量、単位、失敗時の入力保持、fatal budgetとの区別を省略しません。注釈がない場合、独自の保証を補っていません。moduleの説明、signature、実装、本文の対応する章と一緒に読んでください。

実装表示でstdNumeric等のnative入口が現れる場合、その一行だけでnative内部の全処理量が説明されるわけではありません。公開注釈と本文の数値・外部・予算契約も適用されます。

現在の提供範囲

Linux x86_64 / Windows x64 SDK、ネイティブGUI、HTTP/HTTPS、TCP、HTTP server、SQLite/PostgreSQL、typed数値配列、linear algebra、統計、exact整数・十進数、日時、Unicode、regex、増分データ処理、自動微分・optimizer・model保存を提供します。

OS上の任意の外部作用をrevertすること、JVM互換、任意の子process実行、GPU/device I/O、全OS/architecture向けバイナリ、版をまたぐartifact/record形式の全面互換は提供範囲に含めません。外部作用は明示的な領域に置き、VM状態と区別します。

用語

用語	本書での意味
owner	可変値や資源の所有元
binding	名前と値の結び付き
snapshot	ある時点の不変状態
checkpoint	名前付きのVM保存状態
pending	まだ外部へ確定していない操作
publish	pending I/Oの確定
observation / journal	Hostから得た観測と記録
replay	同じcompilerで記録を照合して再現
COW	共有中のデータを変更時に分離する方式
affine	資源を自由に複製せず所有・解放を管理する契約
logical budget	VM契約上の仕事量・領域の上限
fatal diagnostic	通常のResultへ自動変換しない実行失敗

標準ライブラリ API編

全86公開モジュール・625関数。各宣言には引数名、generic bound、戻り型、効果を含めています。共通型は末尾へまとめています。

std.args	std.autodiff	std.autodiffAsync	std.autodiffForwardAsync
std.bigint	std.bits	std.bitset	std.bytes
std.clock	std.collections	std.config	std.csv
std.csvStream	std.datetime	std.db	std.dbDatetime
std.dbDecimal	std.decimal	std.deque	std.disjointSet
std.distributions	std.dp	std.error	std.external
std.fenwick	std.fft	std.fftAsync	std.flow
std.geometry	std.graph	std.graphLarge	std.graphLargeAsync
std.gui	std.guiDialog	std.guiForm	std.guiMenu
std.guiTable	std.guiWindows	std.heap	std.http
std.httpRouter	std.httpServer	std.integer	std.json
std.jsonStream	std.lazySegment	std.map	std.matching
std.math	std.matrix	std.models	std.modular
std.number	std.numeric	std.numericAsync	std.numericIdentityAsync
std.numericIndex	std.numericInputAsync	std.numericRange	std.numericReduceAsync
std.numericShapeAsync	std.numericTransformAsync	std.optimize	std.option
std.path	std.range	std.regex	std.result
std.rollbackSet	std.scanner	std.search	std.segment
std.sequence	std.sort	std.sparse	std.sparseAsync
std.stream	std.stringSearch	std.suffix	std.task
std.taskError	std.tcp	std.text	std.training

std.trie

std.unicode

std.args

起動引数の取得と検査の補助。CLIの--以降がapplication引数です。Args.allの結果を利用し、必要な個数と値を検査します。

```
import

import std.args as args;
```

module固有の公開型

型 (公開field型)

```
record ArgError {
  code: String
}
```

型 (公開field型)

```
record OptionSpec {
  name: String
  short: String
  takesValue: Bool
  required: Bool
}
```

共通型のfield・variant / このmoduleの実装を読む

std.args.help

公開宣言

```
pub fn help(schema: &List<OptionSpec>) -> Result<String,ArgError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<String,ArgError>**

std.args.parse

公開宣言

```
pub fn parse(values: &List<String>, names: &List<String>) -> Result<Map<String,String>,ArgError>
effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Map<String,String>,ArgError>

std.args.parseOptions

公開宣言

```
pub fn parseOptions(values: &List<String>, schema: &List<OptionSpec>) ->
Result<Map<String,String>,ArgError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Map<String,String>,ArgError>

std.autodiff

native配列の演算をtapeへ記録し、逆伝播でgradientを求めます。constantとparameter、Nodeとtapeの所有関係を区別します。同期版です。

```
import
```

```
import std.autodiff as autodiff;
```

module固有の公開型

型（公開field型）

```
record BackwardStep {
  kind: String
  a: Int
  b: Int
  left: FloatArray
  right: FloatArray
  value: FloatArray
  needLeft: Bool
  needRight: Bool
  gradient: Option<FloatArray>
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct BackwardWork {
}
```

型（公開field型）

```
record ForwardInputs {
  kind: String
  left: FloatArray
  right: FloatArray
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct ForwardWork {
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Gradients {  
  
}
```

型（公開fieldなし。専用APIで生成・操作）

```
record Node {  
  
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Tape {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.autodiff.backward

公開宣言

```
pub fn backward(tape: &Tape, loss: Node) -> Result<Gradients,StdError> effects {}
```

効果	なし (effects {})
canonical戻り型	Result<Gradients,StdError>
費用契約・原文	sum of the forward/backward native kernel costs; O(nodes + retained intermediate gradients) storage. Synchronous, individual kernels are not preemptible.
失敗・境界契約・原文	AutodiffNode/Loss, NumericShape/Domain/NonFinite/Overflow; requires a one-element loss, no mutation of tape; fatal work/memory budgets apply. Gradients retains intermediate native arrays until dropped.

std.autodiff.backwardAccumulated

公開宣言

```
pub fn backwardAccumulated(work: &BackwardWork, index: Int) -> Result<Option<FloatArray>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<FloatArray>, StdError>`

失敗・境界契約・原文 AutodiffState/Node; constants return None and must not receive allocated gradients.

std.autodiff.backwardDone

公開宣言

```
pub fn backwardDone(work: &BackwardWork) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 `Bool`

std.autodiff.backwardWork

公開宣言

```
pub fn backwardWork(tape: Tape, loss: Node) -> Result<BackwardWork, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BackwardWork, StdError>`

費用契約・原文 O(rank) validation and empty metadata allocation; consumes tape, no full metadata freeze.

失敗・境界契約・原文 AutodiffNode/Loss; requires a one-element loss. Existing Node storage keys are preserved.

std.autodiff.binary

公開宣言

```
pub fn binary(tape: &mut Tape, operation: String, left: Node, right: Node) -> Result<Node, StdError>
effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Node, StdError>**

失敗・境界契約・原文 NumericDomain/Shape/NonFinite/Overflow, AutodiffNode/Size; add/sub/mul only, shapes must match, broadcasting is explicit.

std.autodiff.broadcast

公開宣言

```
pub fn broadcast(tape: &mut Tape, input: Node, dimensions: &List<Int>) -> Result<Node, StdError>
effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Node, StdError>**

std.autodiff.constant

公開宣言

```
pub fn constant(tape: &mut Tape, array: FloatArray) -> Result<Node, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Node, StdError>**

std.autodiff.create

公開宣言

```
pub fn create() -> Tape effects {}
```

効果 なし (effects {})

canonical戻り型 `Tape`

費用契約・原文 $O(1)$; VM-owned tape, checkpoint/revert uses COW metadata and shared native pages.

std.autodiff.finishBackward

公開宣言

```
pub fn finishBackward(work: BackwardWork) -> Result<Gradients, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Gradients, StdError>`

失敗・境界契約・原文 AutodiffState until traversal completes; consumes work and releases tape metadata, returns existing Gradients.

std.autodiff.finishForward

公開宣言

```
pub fn finishForward(work: ForwardWork, output: FloatArray) -> Result<Tuple<Tape, Node>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Tuple<Tape, Node>, StdError>`

費用契約・原文 append's existing conservative full-storage identity admission; actual identity is cached when the async companion prepared output first. Bridge callers supply a fully computed validated output.

失敗・境界契約・原文 AutodiffSize, NumericType/Domain; consumes work and returns tape/node only after successful append, never accepts a caller-supplied node key.

std.autodiff.forwardInputs

公開宣言

```
pub fn forwardInputs(work: &ForwardWork) -> ForwardInputs effects {}
```

効果 なし (effects {})

canonical戻り型 `ForwardInputs`

std.autodiff.forwardWork

公開宣言

```
pub fn forwardWork(tape: Tape, kind: String, left: Option<Node>, right: Option<Node>, name: String, array: Option<FloatArray>) -> Result<ForwardWork, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<ForwardWork, StdError>`

費用契約・原文 bounded metadata validation and shared input descriptors, no numeric payload scan; consumes tape.

失敗・境界契約・原文 AutodiffName/Node/Size, NumericDomain; parameter/constant, binary:add/sub/mul, unary:relu/sigmoid/tanh/exp/log/square, matmul, reshape/transpose/broadcast/sum/mean.

std.autodiff.gradient

公開宣言

```
pub fn gradient(gradients: &Gradients, node: Node) -> Result<Option<FloatArray>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<FloatArray>, StdError>`

失敗・境界契約・原文 AutodiffNode; None means constant or unreachable from the selected loss, not an allocated zero array.

std.autodiff.matmul

公開宣言

```
pub fn matmul(tape: &mut Tape, left: Node, right: Node) -> Result<Node, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Node, StdError>

std.autodiff.mean

公開宣言

```
pub fn mean(tape: &mut Tape, input: Node) -> Result<Node, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Node, StdError>

std.autodiff.nextBackward

公開宣言

```
pub fn nextBackward(work: &mut BackwardWork) -> Result<BackwardStep, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<BackwardStep, StdError>

失敗・境界契約・原文 AutodiffState before preparation or after completion; returns immutable views, no tensor computation.

std.autodiff.parameter

公開宣言	
<pre>pub fn parameter(tape: &mut Tape, name: String, array: FloatArray) -> Result<Node, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<Node, StdError>
費用契約・原文	O(elements) finite validation, no copy of numeric pages; name bounded to 128 UTF-8 bytes, at most 100000 operations per tape.
失敗・境界契約・原文	AutodiffName/Size, NumericNonFinite; fatal work/memory budgets apply. Tape changes only on success.

std.autodiff.prepareBackward

公開宣言	
<pre>pub fn prepareBackward(work: &mut BackwardWork) -> Result<Bool, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<Bool, StdError>
費用契約・原文	At most 64 metadata entries per call; scalar loss seed only after complete initialization.

std.autodiff.reshape

公開宣言	
<pre>pub fn reshape(tape: &mut Tape, input: Node, dimensions: &List<Int>) -> Result<Node, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<Node, StdError>

std.autodiff.storeBackward

公開宣言

```
pub fn storeBackward(work: &mut BackwardWork, index: Int, incoming: FloatArray) ->
Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

失敗・境界契約・原文 AutodiffState/Node, NumericShape; accepts a fully computed gradient for a needed parent.

std.autodiff.sum

公開宣言

```
pub fn sum(tape: &mut Tape, input: Node) -> Result<Node, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Node, StdError>

std.autodiff.transpose

公開宣言

```
pub fn transpose(tape: &mut Tape, input: Node) -> Result<Node, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Node, StdError>

std.autodiff.unary

公開宣言	
<pre>pub fn unary(tape: &mut Tape, operation: String, input: Node) -> Result<Node,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<Node,StdError></code>
失敗・境界契約・原文	NumericDomain/NonFinite/Overflow, AutodiffNode/Size; relu/sigmoid/tanh/exp/log/square, relu derivative at zero is zero.

std.autodiff.value

公開宣言	
<pre>pub fn value(tape: &Tape, node: Node) -> Result<FloatArray,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<FloatArray,StdError></code>
失敗・境界契約・原文	AutodiffNode for a foreign or replaced node; structurally identical graphs are interchangeable.

std.autodiffAsync

tapeを所有する協調型の逆伝播。awaitにはTaskError、計算結果にはStdErrorがあります。同期backwardの代替が必要な場面で選びます。

```
import

import std.autodiffAsync as autodiffAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.autodiffAsync.backward

公開宣言

```
pub async fn backward(tape: $import$autodiff$Tape, loss: $import$autodiff$Node) ->
  Result<$import$autodiff$Gradients,StdError> effects {tasks}
```

効果	tasks
canonical戻り型	Result<\$import\$autodiff\$Gradients,StdError>
費用契約・原文	Owned tape with shared native pages; metadata in batches of 64 nodes, traversal yields between nodes, tensor derivatives use bounded cooperative kernels. No host worker or external observation.
失敗・境界契約・原文	Same node/loss/numeric validation as synchronous autodiff.backward; child task errors preserve diagnostic/budget codes and failed diagnostic line, cancellation exposes no partial gradients. Fatal budgets apply. Consumes tape; checkpoint roots retain their own snapshots.

std.autodiffForwardAsync

自動微分のforward側を協調処理する補助。途中状態・取消・入力的所有権をsignatureと契約で確認してください。

```
import

import std.autodiffForwardAsync as autodiffForwardAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.autodiffForwardAsync.binary

公開宣言

```
pub async fn binary(tape: $import$autodiff$Tape, operation: String, left: $import$autodiff$Node,
right: $import$autodiff$Node) ->
Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError>`

費用契約・原文 bounded zip and identity preparation; matching shapes, explicit broadcasting. Consumes tape and returns the same graph/storage identities as synchronous forward operations.

失敗・境界契約・原文 NumericDomain/Shape/NonFinite/Overflow, AutodiffNode/Size/Task; no partial output or tape on failure/cancel, fatal work/memory limits apply.

std.autodiffForwardAsync.broadcast

公開宣言

```
pub async fn broadcast(tape: $import$autodiff$Tape, input: $import$autodiff$Node, dimensions:
List<Int>) -> Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError>`

費用契約・原文 O(rank) shared view, reuses parent storage identity; conservative append admission remains.

失敗・境界契約・原文 NumericShape/Type, AutodiffNode/Size/Task; consumes tape, no partial result on cancellation.

std.autodiffForwardAsync.constant

公開宣言	
<pre>pub async fn constant(tape: \$import\$autodiff\$Tape, array: FloatArray) -> Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError></code>
費用契約・原文	bounded finite validation and storage identity preparation; shares input pages, consumes tape.
失敗・境界契約・原文	NumericNonFinite, AutodiffSize/Task; no partial tape on cancellation, fatal budgets apply.

std.autodiffForwardAsync.matmul

公開宣言	
<pre>pub async fn matmul(tape: \$import\$autodiff\$Tape, left: \$import\$autodiff\$Node, right: \$import\$autodiff\$Node) -> Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError></code>
費用契約・原文	$O(m*n*k)$ scalar work in bounded chunks, shared inputs and COW output, then bounded identity preparation.
失敗・境界契約・原文	NumericShape/Type/NonFinite/Overflow, AutodiffNode/Size/Task; fatal budgets apply, no partial tape.

std.autodiffForwardAsync.mean

公開宣言

```
pub async fn mean(tape: $import$autodiff$Tape, input: $import$autodiff$Node) ->
Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError>`

費用契約・原文 bounded logical chunks preserving synchronous accumulation and final division, scalar output identity preparation.

失敗・境界契約・原文 NumericEmpty/NonFinite/Overflow, AutodiffNode/Size/Task; no partial tape on failure/cancel.

std.autodiffForwardAsync.parameter

公開宣言

```
pub async fn parameter(tape: $import$autodiff$Tape, name: String, array: FloatArray) ->
Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Tuple<$import$autodiff$Tape,$import$autodiff$Node>,StdError>`

費用契約・原文 bounded finite validation and storage-identity preparation; owns tape, shares numeric pages and returns tape/node on success. Existing conservative append identity work admission remains.

失敗・境界契約・原文 same name/finite/size contract as synchronous autodiff.parameter; typed child task errors preserve diagnostic/budget codes and failed diagnostic line, no partial tape/node on cancellation, fatal limits apply. Kept checkpoints retain their own tape versions.

std.autodiffForwardAsync.reshape

公開宣言	
<pre>pub async fn reshape(tape: \$import\$autodiff\$Tape, input: \$import\$autodiff\$Node, dimensions: List<Int>) -> Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError></code>
費用契約・原文	O(rank) shared view for contiguous input; otherwise bounded logical copy and new storage identity preparation.
失敗・境界契約・原文	NumericShape/Type/Size, AutodiffNode/Size/Task; no partial tape on failure/cancel.

std.autodiffForwardAsync.sum

公開宣言	
<pre>pub async fn sum(tape: \$import\$autodiff\$Tape, input: \$import\$autodiff\$Node) -> Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError></code>
費用契約・原文	bounded logical chunks preserving synchronous Neumaier order, scalar output identity preparation.
失敗・境界契約・原文	NumericNonFinite/Overflow, AutodiffNode/Size/Task; no partial tape on failure/cancel.

std.autodiffForwardAsync.transpose

公開宣言	
<pre>pub async fn transpose(tape: \$import\$autodiff\$Tape, input: \$import\$autodiff\$Node) -> Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError></code>
費用契約・原文	O(rank) shared rank-two view, reuses parent storage identity; conservative append admission remains.
失敗・境界契約・原文	NumericShape/Type, AutodiffNode/Size/Task; consumes tape, no partial result on cancellation.

std.autodiffForwardAsync.unary

公開宣言	
<pre>pub async fn unary(tape: \$import\$autodiff\$Tape, operation: String, input: \$import\$autodiff\$Node) -> Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Tuple<\$import\$autodiff\$Tape,\$import\$autodiff\$Node>,StdError></code>
費用契約・原文	bounded chunks for relu/sigmoid/tanh/exp/log/square and output identity preparation; consumes tape.
失敗・境界契約・原文	NumericDomain/NonFinite/Overflow, AutodiffNode/Size/Task; no partial tape on failure/cancel.

std.bigint

Floatを介さない正確な任意精度整数。parse/format、算術、bit操作、JSON変換を提供します。
有限の容量・native work・memory予算が適用されます。

```
import
```

```
import std.bigint as bigint;
```

共通型のfield・variant / このmoduleの実装を読む

std.bigint.add

公開宣言

```
pub fn add(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.bitAnd

公開宣言

```
pub fn bitAnd(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.bitNot

公開宣言

```
pub fn bitNot(value: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.bitOr

公開宣言

```
pub fn bitOr(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.bitXor

公開宣言

```
pub fn bitXor(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.bits

公開宣言

```
pub fn bits(value: &BigInt) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.compare

公開宣言

```
pub fn compare(left: &BigInt, right: &BigInt) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.divide

公開宣言

```
pub fn divide(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.format

公開宣言

```
pub fn format(value: &BigInt, radix: Int) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<String, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.fromInt

公開宣言

```
pub fn fromInt(value: Int) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.fromJson

公開宣言

```
pub fn fromJson(value: Json) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.gcd

公開宣言

```
pub fn gcd(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.modPow

公開宣言

```
pub fn modPow(value: &BigInt, exponent: &BigInt, modulus: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.modulo

公開宣言

```
pub fn modulo(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.multiply

公開宣言

```
pub fn multiply(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.negate

公開宣言

```
pub fn negate(value: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.parse

公開宣言

```
pub fn parse(value: String, radix: Int) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.pow

公開宣言

```
pub fn pow(value: &BigInt, exponent: Int) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.remainder

公開宣言

```
pub fn remainder(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BigInt, StdError>**

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.shift

公開宣言

```
pub fn shift(value: &BigInt, left: Bool, count: Int) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.subtract

公開宣言

```
pub fn subtract(left: &BigInt, right: &BigInt) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.toInt

公開宣言

```
pub fn toInt(value: &BigInt) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bigint.toJson

公開宣言

```
pub fn toJson(value: &BigInt) -> Result<Json, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json, StdError>`

費用契約・原文 native work is precharged from input/output size; quadratic kernels and powers have additional work.

失敗・境界契約・原文 StdError BigIntSyntax/Radix/Size/Domain/Overflow/DivisionByZero/JsonType; fatal VM work and memory budgets apply.

std.bits

Intのbit操作。shift幅や符号の扱いを確認し、算術の剰余と混同しないでください。

```
import
```

```
import std.bits as bits;
```

共通型のfield・variant / このmoduleの実装を読む

std.bits.and

公開宣言

```
pub fn and(a: Int, b: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.bits.count

公開宣言

```
pub fn count(a: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.bits.not

公開宣言

```
pub fn not(a: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.bits.or

公開宣言

```
pub fn or(a: Int, b: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.bits.shiftLeft

公開宣言

```
pub fn shiftLeft(a: Int, n: Int) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,StdError>`

std.bits.shiftRight

公開宣言

```
pub fn shiftRight(a: Int, n: Int) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,StdError>`

std.bits.shiftUnsigned

公開宣言

```
pub fn shiftUnsigned(a: Int, n: Int) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,StdError>`

std.bits.xor

公開宣言

```
pub fn xor(a: Int, b: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 **Int**

std.bitset

boundedなbit集合。indexと容量を検査して使います。集合演算や数え上げに、BoolのList以外の表現を選べます。

```
import
```

```
import std.bitset as bitset;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct BitSet {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.bitset.count

公開宣言

```
pub fn count(set: &BitSet) -> Int effects {}
```

効果 なし（effects {}）

canonical戻り型 **Int**

std.bitset.create

公開宣言

```
pub fn create(length: Int) -> Result<BitSet,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<BitSet,String>**

std.bitset.get

公開宣言

```
pub fn get(set: &BitSet, index: Int) -> Result<Bool,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool,String>`

std.bitset.intersection

公開宣言

```
pub fn intersection(a: &BitSet, b: &BitSet) -> Result<BitSet,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BitSet,String>`

std.bitset.length

公開宣言

```
pub fn length(set: &BitSet) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.bitset.set

公開宣言

```
pub fn set(set: &mut BitSet, index: Int, value: Bool) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,String>`

std.bitset.shifted

公開宣言

```
pub fn shifted(input: &BitSet, distance: Int) -> Result<BitSet,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BitSet,String>`

std.bitset.union

公開宣言

```
pub fn union(a: &BitSet, b: &BitSet) -> Result<BitSet,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BitSet,String>`

std.bytes

UTF-8 encode/decodeとbyte列の読み取り・slice。ResultのerrorはStdErrorです。組み込みdecodeUtf8のString errorと区別します。

```
import

import std.bytes as bytes;
```

共通型のfield・variant / このmoduleの実装を読む

std.bytes.decode

```
公開宣言

pub fn decode(value: Bytes) -> Result<String,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,StdError>`

std.bytes.encode

```
公開宣言

pub fn encode(value: String) -> Result<Bytes,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bytes,StdError>`

std.bytes.get

公開宣言

```
pub fn get(value: Bytes, index: Int) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

std.bytes.length

公開宣言

```
pub fn length(value: Bytes) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.bytes.slice

公開宣言

```
pub fn slice(value: Bytes, start: Int, end: Int) -> Result<Bytes, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bytes, StdError>`

std.clock

型付きInstantとして現在時刻を観測します。externalとclockが必要です。日時のpureな変換とは効果が違います。

```
import
```

```
import std.clock as clock;
```

共通型のfield・variant / このmoduleの実装を読む

std.clock.now

公開宣言

```
pub fn now() -> Result<Instant, StdError> effects {clock, external}
```

効果	clock, external
----	-----------------

canonical戻り型	<code>Result<Instant, StdError></code>
--------------	--

費用契約・原文	$O(1)$; seconds and nanoseconds are one atomic recorded external observation.
---------	--

失敗・境界契約・原文	ExternalFailure/DateTimeRange; requires external region; rewind reuses the observed value, never rewinds the physical clock.
------------	--

std.collections

Listなどのcollection操作の補助。借用と所有値の返却、境界条件をsignatureで確認してください。

```
import

import std.collections as collections;
```

共通型のfield・variant / このmoduleの実装を読む

std.collections.all

公開宣言

```
pub fn all<T: Share>(values: &List<T>, predicate: fn(T)->Bool!{}~{Send+Share}) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 Bool

generic契約 T: Share

std.collections.any

公開宣言

```
pub fn any<T: Share>(values: &List<T>, predicate: fn(T)->Bool!{}~{Send+Share}) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 Bool

generic契約 T: Share

std.collections.count

公開宣言

```
pub fn count<T: Share>(values: &List<T>, predicate: fn(T)->Bool!{}~{Send+Share}) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

generic契約 T: Share

std.collections.filter

公開宣言

```
pub fn filter<T: Share>(values: &List<T>, predicate: fn(T)->Bool!{}~{Send+Share}) -> List<T> effects {}
```

効果 なし (effects {})

canonical戻り型 List<T>

generic契約 T: Share

std.collections.find

公開宣言

```
pub fn find<T: Share>(values: &List<T>, predicate: fn(T)->Bool!{}~{Send+Share}) -> Option<T> effects {}
```

効果 なし (effects {})

canonical戻り型 Option<T>

generic契約 T: Share

std.collections.fold

公開宣言

```
pub fn fold<T: Share, U: Share>(values: &List<T>, initial: U, combine: fn(U,T)->U!{}~{Send+Share})  
-> U effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	U
--------------	---

generic契約	T: Share, U: Share
-----------	--------------------

std.collections.map

公開宣言

```
pub fn map<T: Share, U: Share>(values: &List<T>, transform: fn(T)->U!{}~{Send+Share}) -> List<U>  
effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	List<U>
--------------	---------

generic契約	T: Share, U: Share
-----------	--------------------

std.config

設定値の解析・取得を補助します。形式の解析が成功しても、業務上の値の検査は別に必要です。

```
import
```

```
import std.config as config;
```

module固有の公開型

型 (公開field型)

```
record ConfigError {  
  code: String  
  field: String  
}
```

型 (公開field型)

```
record Field {  
  name: String  
  kind: String  
  required: Bool  
  fallback: Json  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.config.resolve

公開宣言

```
pub fn resolve(file: Json, environment: Json, arguments: Json, fields: &List<Field>) ->  
Result<Json,ConfigError> effects {}
```

効果

なし (effects {})

canonical戻り型

Result<Json,ConfigError>

std.csv

boundedなCSVの解析・生成。引用符や改行を含むCSVを、単純なcomma splitと区別して処理します。

```
import

import std.csv as csv;
```

module固有の公開型

型（公開field型）

```
record CsvError {
  code: String
  position: Int
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Table {

}
```

共通型のfield・variant / このmoduleの実装を読む

std.csv.format

公開宣言

```
pub fn format(table: &Table) -> Result<Bytes,CsvError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Bytes,CsvError>**

std.csv.get

公開宣言

```
pub fn get(table: &Table, row: Int, column: Int) -> Result<String,CsvError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,CsvError>`

std.csv.parse

公開宣言

```
pub fn parse(input: Bytes) -> Result<Table,CsvError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Table,CsvError>`

std.csv.rows

公開宣言

```
pub fn rows(table: &Table) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.csv.width

公開宣言

```
pub fn width(table: &Table, row: Int) -> Result<Int,CsvError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,CsvError>`

std.csvStream

chunk境界をまたぐCSV解析の状態を保持します。入力 of 終了と途中token、error offset、容量を扱います。全行を保存する必要がない処理に向きます。

```
import

import std.csvStream as csvStream;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Reader {

}
```

共通型のfield・variant / このmoduleの実装を読む

std.csvStream.cancel

公開宣言

```
pub fn cancel(reader: &mut Reader) -> Result<Unit,StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Unit,StdError>**

std.csvStream.feed

公開宣言

```
pub fn feed(reader: &mut Reader, chunk: Bytes, finish: Bool) -> Result<Unit,StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Unit,StdError>**

std.csvStream.finish

公開宣言

```
pub fn finish(reader: &mut Reader) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.csvStream.next

公開宣言

```
pub fn next(reader: &mut Reader) -> Result<Option<List<String>>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<List<String>>, StdError>`

std.csvStream.position

公開宣言

```
pub fn position(reader: &Reader) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

std.csvStream.reader

公開宣言

```
pub fn reader() -> Result<Reader, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Reader, StdError>`

std.datetime

Instant/Durationとcalendar、明示timezoneの変換・算術。DSTのoverlapは選択、gapは失敗です。現在時刻を読むAPIとは別です。

```
import
```

```
import std.datetime as datetime;
```

module固有の公開型

variantとpayload型

```
enum Ambiguity {  
    Earlier,  
    Later,  
    Reject  
}
```

型（公開field型）

```
struct Calendar {  
    year: Int  
    month: Int  
    day: Int  
    hour: Int  
    minute: Int  
    second: Int  
    nanosecond: Int  
}
```

型（公開field型）

```
struct LocalTime {  
    calendar: Calendar  
    offsetSeconds: Int  
    weekday: Int  
    dayOfYear: Int  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.datetime.add

公開宣言	
<pre>pub fn add(value: &Instant, delta: &Duration) -> Result<Instant, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<Instant, StdError></code>

std.datetime.addDurations

公開宣言	
<pre>pub fn addDurations(left: &Duration, right: &Duration) -> Result<Duration, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<Duration, StdError></code>

std.datetime.calendar

公開宣言	
<pre>pub fn calendar(value: &Instant, zone: String) -> Result<LocalTime, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<LocalTime, StdError></code>
費用契約・原文	bounded IANA lookup; weekday Monday=1..Sunday=7, dayOfYear=1..366; historical offsets retain seconds.

std.datetime.calendarOffset

公開宣言

```
pub fn calendarOffset(value: &Instant, offsetSeconds: Int) -> Result<LocalTime, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<LocalTime, StdError>`

std.datetime.databaseVersion

公開宣言

```
pub fn databaseVersion() -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.datetime.difference

公開宣言

```
pub fn difference(left: &Instant, right: &Instant) -> Result<Duration, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Duration, StdError>`

std.datetime.duration

公開宣言

```
pub fn duration(seconds: Int, nanoseconds: Int) -> Result<Duration, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Duration, StdError>`

失敗・境界契約・原文 DateTimeRange; normalizes signed nanoseconds with checked overflow.

std.datetime.durationFromJson

公開宣言

```
pub fn durationFromJson(value: Json) -> Result<Duration, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Duration, StdError>`

std.datetime.durationNanoseconds

公開宣言

```
pub fn durationNanoseconds(value: &Duration) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

std.datetime.durationSeconds

公開宣言

```
pub fn durationSeconds(value: &Duration) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

std.datetime.durationToJson

公開宣言

```
pub fn durationToJson(value: &Duration) -> Result<Json, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json, StdError>`

std.datetime.format

公開宣言

```
pub fn format(value: &Instant) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.datetime.formatOffset

公開宣言

```
pub fn formatOffset(value: &Instant, offsetSeconds: Int) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

失敗・境界契約・原文 DateTimeOffset/Range; RFC3339 requires a whole-minute offset strictly inside +/-24 hours.

std.datetime.fromJson

公開宣言

```
pub fn fromJson(value: Json) -> Result<Instant, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Instant, StdError>`

std.datetime.instant

公開宣言

```
pub fn instant(seconds: Int, nanoseconds: Int) -> Result<Instant, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Instant, StdError>`

失敗・境界契約・原文 DateTimeRange; UTC calendar years 1..9999, nanoseconds 0..999999999.

std.datetime.nanoseconds

公開宣言

```
pub fn nanoseconds(value: &Instant) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

std.datetime.negateDuration

公開宣言

```
pub fn negateDuration(value: &Duration) -> Result<Duration, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Duration, StdError>`

std.datetime.parse

公開宣言

```
pub fn parse(text: String) -> Result<Instant, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Instant, StdError>`

失敗・境界契約・原文 DateTimeSyntax/Calendar/Range; explicit RFC3339 offset, up to nine fractional digits, no leap seconds.

std.datetime.resolve

公開宣言

```
pub fn resolve(value: Calendar, zone: String, ambiguity: Ambiguity) -> Result<Instant, StdError>
effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Instant, StdError>

失敗・境界契約・原文 DateTimeCalendar/Zone/Ambiguous/Gap/Range; gaps always fail, overlaps require an explicit policy.

std.datetime.resolveOffset

公開宣言

```
pub fn resolveOffset(value: Calendar, offsetSeconds: Int) -> Result<Instant, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Instant, StdError>

std.datetime.seconds

公開宣言

```
pub fn seconds(value: &Instant) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Int, StdError>

std.datetime.subtractDurations

公開宣言

```
pub fn subtractDurations(left: &Duration, right: &Duration) -> Result<Duration, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Duration, StdError>`

std.datetime.toJson

公開宣言

```
pub fn toJson(value: &Instant) -> Result<Json, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json, StdError>`

std.db

SQLite/PostgreSQL、parameter binding、statement/cursor、transaction、cleanup。領域内でsubmitし外でawaitします。DB書込みはVMのrevertで取り消しません。

```
import
```

```
import std.db as db;
```

共通型のfield・variant / このmoduleの実装を読む

std.db.beginTransaction

公開宣言

```
pub fn beginTransaction(connection: &mut DbConnection, timeoutMillis: Int) ->
Task<Result<Unit, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Unit, DbError>>

費用契約・原文 O(DB transaction setup); independent of VM commit labels.

std.db.boolean

公開宣言

```
pub fn boolean(row: Frozen<DbRow>, index: Int) -> Result<Bool, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Bool, DbError>

費用契約・原文 O(1); SQLite integer 0/1 is converted explicitly, other integers return DbType.

std.db.bytes

公開宣言

```
pub fn bytes(row: Frozen<DbRow>, index: Int) -> Result<Bytes, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bytes, DbError>`

費用契約・原文 O(1) shared bytes; requires a Bytes cell.

std.db.cell

公開宣言

```
pub fn cell(row: Frozen<DbRow>, index: Int) -> Result<DbValue, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<DbValue, DbError>`

費用契約・原文 O(value bytes); bounds failure is DbColumnRange, SQL NULL is DbValue::Null.

std.db.cellByName

公開宣言

```
pub fn cellByName(row: Frozen<DbRow>, columns: Frozen<List<String>>, name: String) ->  
Result<DbValue, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<DbValue, DbError>`

費用契約・原文 O(column names + value bytes); use a cursor's immutable column metadata.

std.db.close

公開宣言

```
pub fn close(connection: &mut DbConnection, timeoutMillis: Int) -> Task<Result<Unit, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Unit, DbError>>

費用契約・原文 O(DB cleanup); awaits rollback and connection close, preserving SQL failure codes.

std.db.closeCursor

公開宣言

```
pub fn closeCursor(cursor: &mut DbCursor, timeoutMillis: Int) -> Task<Result<Unit, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Unit, DbError>>

費用契約・原文 O(statement cleanup); explicit close leaves the parent connection reusable.

std.db.closeStatement

公開宣言

```
pub fn closeStatement(statement: &mut DbStatement, timeoutMillis: Int) -> Task<Result<Unit, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Unit, DbError>>

費用契約・原文 O(1); invalidates the affine handle, cache capacity remains bounded.

std.db.column

公開宣言

```
pub fn column(columns: Frozen<List<String>>, name: String) -> Result<Int,DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,DbError>`

費用契約・原文 `O(column names)`; rejects absent or duplicated names instead of guessing.

std.db.commitTransaction

公開宣言

```
pub fn commitTransaction(connection: &mut DbConnection, timeoutMillis: Int) ->  
Task<Result<Unit,DbError>> effects {db, external, tasks}
```

効果 `db, external, tasks`

canonical戻り型 `Task<Result<Unit,DbError>>`

費用契約・原文 `O(DB commit)`; missing acknowledgement leaves outcome Unknown.

std.db.credentials

公開宣言

```
pub fn credentials(alias: String, connectionString: Secret<String>, caCertificate: Bytes) ->  
Result<Unit,DbError> effects {db, external, tasks}
```

効果 `db, external, tasks`

canonical戻り型 `Result<Unit,DbError>`

費用契約・原文 `O(connection-string and certificate bytes)`; immutable alias outside VM checkpoints.

失敗・境界契約・原文 `DbError`; remote connections require verified TLS; certificate is DER, empty Bytes uses OS roots.

std.db.execute

公開宣言

```
pub fn execute(connection: &mut DbConnection, sql: String, parameters: Frozen<List<DbValue>>,
timeoutMillis: Int) -> Task<Result<Int, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Int, DbError>>

費用契約・原文 O(SQL work + parameter bytes); binds values without SQL interpolation.

失敗・境界契約・原文 DbError; DB writes are external effects and are not undone by VM revert.

std.db.executeMany

公開宣言

```
pub fn executeMany(connection: &mut DbConnection, sql: String, parameters:
Frozen<List<Frozen<List<DbValue>>>>, timeoutMillis: Int) -> Task<Result<Int, DbError>> effects {db,
external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Int, DbError>>

費用契約・原文 O(SQL work + parameter bytes); prepares once for at most 1024 bounded parameter rows.

失敗・境界契約・原文 DbError; does not create an implicit transaction, earlier rows may have been applied.

std.db.executeStatement

公開宣言

```
pub fn executeStatement(statement: &mut DbStatement, parameters: Frozen<List<DbValue>>,
timeoutMillis: Int) -> Task<Result<Int, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Int, DbError>>

費用契約・原文 O(SQL work + parameter bytes); statement belongs to its original connection.

std.db.floating

公開宣言

```
pub fn floating(row: Frozen<DbRow>, index: Int) -> Result<Float, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, DbError>`

費用契約・原文 $O(1)$; requires a Float cell, does not silently coerce strings or integers.

std.db.integer

公開宣言

```
pub fn integer(row: Frozen<DbRow>, index: Int) -> Result<Int, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, DbError>`

費用契約・原文 $O(1)$; requires an Int cell, NULL and other types return DbType.

std.db.next

公開宣言

```
pub fn next(cursor: &mut DbCursor, maxRows: Int, maxBytes: Int, timeoutMillis: Int) ->  
Task<Result<DbBatch, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 `Task<Result<DbBatch, DbError>>`

費用契約・原文 $O(\text{batch bytes} + \text{SQL work})$; at most 64 rows and 4 MiB per batch.

失敗・境界契約・原文 DbError; maxBytes is 1048576..4194304, deadline also respects the original query deadline.

std.db.postgres

公開宣言

```
pub fn postgres(alias: String, timeoutMillis: Int) -> Task<Result<DbConnection, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<DbConnection, DbError>>

費用契約・原文 O(DNS + one TLS connection); no reconnect, failover or write retry.

失敗・境界契約・原文 DbError; uses a registered opaque alias, never stores a connection string in trace fingerprints.

std.db.prepare

公開宣言

```
pub fn prepare(connection: &mut DbConnection, sql: String, timeoutMillis: Int) -> Task<Result<DbStatement, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<DbStatement, DbError>>

費用契約・原文 O(SQL preparation); reuses a bounded per-connection statement cache.

std.db.privateParameter

公開宣言

```
pub fn privateParameter(alias: String, value: Secret<DbValue>) -> Result<DbValue, DbError> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Result<DbValue, DbError>

費用契約・原文 O(value bytes); immutable opaque alias outside VM checkpoints.

失敗・境界契約・原文 DbError; values are bound privately and never included in request fingerprints.

std.db.query

公開宣言

```
pub fn query(connection: &mut DbConnection, sql: String, parameters: Frozen<List<DbValue>>,
timeoutMillis: Int) -> Task<Result<DbCursor, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<DbCursor, DbError>>

費用契約・原文 O(SQL prepare); rows are read only by next.

失敗・境界契約・原文 DbError; one active cursor per SQLite connection.

std.db.queryStatement

公開宣言

```
pub fn queryStatement(statement: &mut DbStatement, parameters: Frozen<List<DbValue>>,
timeoutMillis: Int) -> Task<Result<DbCursor, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<DbCursor, DbError>>

費用契約・原文 O(SQL preparation); rows remain lazy until next.

std.db.rollbackTransaction

公開宣言

```
pub fn rollbackTransaction(connection: &mut DbConnection, timeoutMillis: Int) ->
Task<Result<Unit, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Unit, DbError>>

費用契約・原文 O(DB rollback); no VM state is reverted.

std.db.sqlite

公開宣言

```
pub fn sqlite(path: String, readOnly: Bool, timeoutMillis: Int) ->
  Task<Result<DbConnection, DbError>> effects {db, external, tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<DbConnection, DbError>>

費用契約・原文 O(connection setup); at most eight native connections.

失敗・境界契約・原文 DbError; path is relative to root, :memory: is an isolated in-memory database.

std.db.text

公開宣言

```
pub fn text(row: Frozen<DbRow>, index: Int) -> Result<String, DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<String, DbError>

費用契約・原文 O(UTF-8 bytes); requires a Text cell.

std.db.waitForCleanup

公開宣言

```
pub fn waitForCleanup(timeoutMillis: Int) -> Task<Result<Unit, DbError>> effects {db, external,
  tasks}
```

効果 db, external, tasks

canonical戻り型 Task<Result<Unit, DbError>>

費用契約・原文 O(closed resources); waits without blocking the VM on native cleanup.

失敗・境界契約・原文 DbError; preserves rollback/close failures from scope-owned resources.

std.dbDatetime

型付き日時とDB表現の境界。backendのtimestamp精度とtimezone情報の保持を確認します。

```
import  
  
import std.dbDatetime as dbDatetime;
```

共通型のfield・variant / このmoduleの実装を読む

std.dbDatetime.cell

```
公開宣言  
  
pub fn cell(row: Frozen<DbRow>, index: Int) -> Result<Instant,DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Instant,DbError>`

std.dbDatetime.parameter

```
公開宣言  
  
pub fn parameter(value: &Instant) -> Result<DbValue,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<DbValue,StdError>`

失敗・境界契約・原文 DateTimeRange if nanos are not whole microseconds; PostgreSQL timestampz is an instant, not a local calendar or zone name.

std.dbDatetime.sqliteParameter

公開宣言

```
pub fn sqliteParameter(value: &Instant) -> Result<DbValue, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<DbValue, StdError>`

std.dbDecimal

exact十進数とDB表現の境界。Floatへ黙って変換せず、precision/scale/NULL/型を確認します。

```
import  
  
import std.dbDecimal as dbDecimal;
```

共通型のfield・variant / このmoduleの実装を読む

std.dbDecimal.cell

公開宣言

```
pub fn cell(row: Frozen<DbRow>, index: Int) -> Result<Decimal,DbError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Decimal,DbError>**

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.dbDecimal.parameter

公開宣言

```
pub fn parameter(value: &Decimal) -> Result<DbValue,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<DbValue,StdError>**

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.dbDecimal.sqliteParameter

公開宣言	
<pre>pub fn sqliteParameter(value: &Decimal) -> Result<DbValue,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<DbValue,StdError>
費用契約・原文	native work and scratch precharged from coefficient, scale and output size.
失敗・境界契約・原文	typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal

precision・scale・roundingを明示する十進数。数値比較と保存するscaleを区別し、JSON変換でもexact表現を使います。

```
import
```

```
import std.decimal as decimal;
```

共通型のfield・variant / このmoduleの実装を読む

std.decimal.add

公開宣言

```
pub fn add(left: &Decimal, right: &Decimal, precision: Int, rounding: DecimalRounding) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.coefficient

公開宣言

```
pub fn coefficient(value: &Decimal) -> Result<BigInt, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<BigInt, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.compare

公開宣言

```
pub fn compare(left: &Decimal, right: &Decimal) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.divide

公開宣言

```
pub fn divide(left: &Decimal, right: &Decimal, scale: Int, precision: Int, rounding: DecimalRounding) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.format

公開宣言

```
pub fn format(value: &Decimal) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.fromCoefficient

公開宣言

```
pub fn fromCoefficient(value: &BigInt, scale: Int) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.fromJson

公開宣言

```
pub fn fromJson(value: Json) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.multiply

公開宣言

```
pub fn multiply(left: &Decimal, right: &Decimal, precision: Int, rounding: DecimalRounding) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.parse

公開宣言

```
pub fn parse(value: String) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.quantize

公開宣言

```
pub fn quantize(value: &Decimal, scale: Int, precision: Int, rounding: DecimalRounding) ->  
Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.representation

公開宣言

```
pub fn representation(value: &Decimal) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.scale

公開宣言

```
pub fn scale(value: &Decimal) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.subtract

公開宣言

```
pub fn subtract(left: &Decimal, right: &Decimal, precision: Int, rounding: DecimalRounding) -> Result<Decimal, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Decimal, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.decimal.toJson

公開宣言

```
pub fn toJson(value: &Decimal) -> Result<Json, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json, StdError>`

費用契約・原文 native work and scratch precharged from coefficient, scale and output size.

失敗・境界契約・原文 typed StdError/DbError; no implicit Float conversion; fatal VM work and memory budgets apply.

std.deque

両端を操作するqueue。capacity、空の取り出し、indexの扱いを検査します。通常のListの先頭削除とは費用が違います。

```
import
```

```
import std.deque as deque;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Deque<T> {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.deque.create

公開宣言

```
pub fn create<T: Share>(capacity: Int) -> Result<Deque<T>,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Deque<T>,String>**

generic契約 **T: Share**

std.deque.length

公開宣言

```
pub fn length<T: Share>(d: &Deque<T>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

generic契約 T: Share

std.deque.popBack

公開宣言

```
pub fn popBack<T: Share>(d: &mut Deque<T>) -> Option<T> effects {}
```

効果 なし (effects {})

canonical戻り型 Option<T>

generic契約 T: Share

std.deque.popFront

公開宣言

```
pub fn popFront<T: Share>(d: &mut Deque<T>) -> Option<T> effects {}
```

効果 なし (effects {})

canonical戻り型 Option<T>

generic契約 T: Share

std.deque.pushBack

公開宣言

```
pub fn pushBack<T: Share>(d: &mut Deque<T>, value: T) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,String>`

generic契約 `T: Share`

std.deque.pushFront

公開宣言

```
pub fn pushFront<T: Share>(d: &mut Deque<T>, value: T) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,String>`

generic契約 `T: Share`

std.disjointSet

集合の併合と同一集合の判定。頂点indexとサイズを検査します。データ構造自身の巻き戻しはrollbackSetを参照してください。

```
import

import std.disjointSet as disjointSet;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct DisjointSet {

}
```

共通型のfield・variant / このmoduleの実装を読む

std.disjointSet.create

公開宣言

```
pub fn create(count: Int) -> Result<DisjointSet,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<DisjointSet,String>**

std.disjointSet.root

公開宣言

```
pub fn root(d: &mut DisjointSet, index: Int) -> Result<Int,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Int,String>**

std.disjointSet.same

公開宣言

```
pub fn same(d: &mut DisjointSet, a: Int, b: Int) -> Result<Bool,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool,String>`

std.disjointSet.size

公開宣言

```
pub fn size(d: &mut DisjointSet, index: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.disjointSet.unite

公開宣言

```
pub fn unite(d: &mut DisjointSet, a: Int, b: Int) -> Result<Bool,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool,String>`

std.distributions

VMのcheckpointed generatorを使うuniform/normal/exponential/Bernoulli等。random効果と入力parameterの定義域に注意します。

```
import

import std.distributions as distributions;
```

共通型のfield・variant / このmoduleの実装を読む

std.distributions.bernoulli

公開宣言

```
pub fn bernoulli(probability: Float) -> Result<Bool,StdError> effects {random}
```

効果 random

canonical戻り型 `Result<Bool,StdError>`

費用契約・原文 $O(1)$; checkpointed uniform draw, probability in $[0,1]$.

失敗・境界契約・原文 StdError NumericDomain/NonFinite; fatal VM execution budget applies.

std.distributions.exponential

公開宣言

```
pub fn exponential(rate: Float) -> Result<Float,StdError> effects {random}
```

効果 random

canonical戻り型 `Result<Float,StdError>`

費用契約・原文 $O(1)$; checkpointed uniform draw, positive finite rate.

失敗・境界契約・原文 StdError NumericDomain/NonFinite; fatal VM execution budget applies.

std.distributions.normal

公開宣言

```
pub fn normal(mean: Float, deviation: Float) -> Result<Float, StdError> effects {random}
```

効果 random

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 O(1); Box-Muller, two checkpointed uniform draws, no hidden cached spare.

失敗・境界契約・原文 StdError NumericDomain/NonFinite; fatal VM execution budget applies.

std.distributions.uniform

公開宣言

```
pub fn uniform() -> Float effects {random}
```

効果 random

canonical戻り型 `Float`

費用契約・原文 O(1); checkpointed generator, 53 random bits, range [0,1).

失敗・境界契約・原文 Fatal VM execution budget only.

std.dp

動的計画法の補助。問題サイズ、整数範囲、保存するtableの容量とworkを確認してください。

```
import
```

```
import std.dp as dp;
```

共通型のfield・variant / このmoduleの実装を読む

std.dp.knapsack

公開宣言

```
pub fn knapsack(weights: &List<Int>, values: &List<Int>, capacity: Int) -> Result<Int,String>
effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.dp.lisLength

公開宣言

```
pub fn lisLength(input: &List<Int>) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.error

application向けError envelopeへの変換。診断の全文・全cause treeを無条件に保存する投影ではありません。元のDiagnosticが必要なら保持します。

```
import  
  
import std.error as error;
```

module固有の公開型

型（公開field型）

```
record Error {  
    code: String  
    message: String  
    source: String  
    line: Int  
    column: Int  
    offset: Int  
    cause: Option<String>  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.error.caused

公開宣言

```
pub fn caused(code: String, message: String, cause: String) -> Error effects {}
```

効果 なし（effects {}）

canonical戻り型

Error

std.error.code

公開宣言

```
pub fn code(code: String) -> Error effects {}
```

効果 なし (effects {})

canonical戻り型 Error

std.error.fromDiagnostic

公開宣言

```
pub fn fromDiagnostic(error: &Diagnostic) -> Error effects {}
```

効果 なし (effects {})

canonical戻り型 Error

費用契約・原文 O(1) envelope projection; complete cause tree remains in the original Diagnostic.

失敗・境界契約・原文 Does not catch a fatal runtime error or grant retry permission; preserves diagnostic code and source location.

std.error.fromFile

公開宣言

```
pub fn fromFile(error: FileError) -> Error effects {}
```

効果 なし (effects {})

canonical戻り型 Error

std.error.fromJson

公開宣言

```
pub fn fromJson(error: JsonError, source: String) -> Error effects {}
```

効果 なし (effects {})

canonical戻り型 **Error**

std.error.fromStd

公開宣言

```
pub fn fromStd(error: StdError, source: String) -> Error effects {}
```

効果 なし (effects {})

canonical戻り型 **Error**

std.external

明示的な外部領域で使う観測の補助。revertで外部作用が消えず、通常領域の観測再利用とfresh/liveを区別します。

```
import
```

```
import std.external as external;
```

共通型のfield・variant / このmoduleの実装を読む

std.external.millis

公開宣言

```
pub fn millis() -> Result<Int, StdError> effects {clock, external}
```

効果 clock, external

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 O(1) clock observation; recorded once per external ledger position.

失敗・境界契約・原文 ExternalFailure/ClockRange; requires an external region.

std.fenwick

更新とprefix/range集約のためのFenwick tree。index区間とchecked整数、容量を検査して使用します。

```
import
```

```
import std.fenwick as fenwick;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Fenwick {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.fenwick.add

公開宣言

```
pub fn add(tree: &mut Fenwick, index: Int, delta: Int) -> Result<Unit,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Unit,String>**

std.fenwick.create

公開宣言

```
pub fn create(length: Int) -> Result<Fenwick,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Fenwick,String>**

std.fenwick.length

公開宣言

```
pub fn length(tree: &Fenwick) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.fenwick.prefix

公開宣言

```
pub fn prefix(tree: &Fenwick, end: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.fenwick.range

公開宣言

```
pub fn range(tree: &Fenwick, start: Int, end: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.fft

複素数/数値配列のFFT関連処理。長さ、transformの向き、scaleとerrorを契約で確認してください。

```
import

import std.fft as fft;
```

module固有の公開型

型（公開field型）

```
record Spectrum {
    real: FloatArray
    imag: FloatArray
}
```

共通型のfield・variant / このmoduleの実装を読む

std.fft.convolve

公開宣言

```
pub fn convolve(left: &FloatArray, right: &FloatArray) -> Result<FloatArray,StdError> effects {}
```

効果	なし（effects {}）
canonical戻り型	<code>Result<FloatArray,StdError></code>
費用契約・原文	$O(n \log n)$ with n the next power of two of the full linear result length; $O(n)$ native temporary storage.
失敗・境界契約・原文	NumericShape/Size/NonFinite; rank-one Float64 inputs, empty input produces an empty result. FFT size at most 2^{20} ; approximate, not exact integer convolution.

std.fft.transform

公開宣言	
<pre>pub fn transform(real: &FloatArray, imag: &FloatArray, inverse: Bool) -> Result<Spectrum,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<Spectrum,StdError>
費用契約・原文	O(n log n) work / O(n) native temporary storage; synchronous, maximum 2^20 samples.
失敗・境界契約・原文	NumericShape/Size/NonFinite; equal rank-one power-of-two inputs, inverse scales by 1/n. Fatal budgets apply.

std.fftAsync

FFT関連の協調型API。同期版の数値契約に加えてTaskError、取消、途中状態の費用を確認します。

```
import

import std.fftAsync as fftAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.fftAsync.transform

公開宣言

```
pub async fn transform(real: FloatArray, imag: FloatArray, inverse: Bool) ->
Result<$import$fft$Spectrum,StdError> effects {tasks}
```

効果	tasks
canonical戻り型	Result<\$import\$fft\$Spectrum,StdError>
費用契約・原文	O(n log n); at most 4096 copy, twiddle, butterfly or scale items per native step, then hands off to ready tasks. O(n) COW storage, cached twiddles and contiguous page writes.
失敗・境界契約・原文	NumericShape/Type/Size/NonFinite; equal rank-one power-of-two inputs, maximum 2^20 samples. Native scratch stays private on failure or cancellation; checkpoint-retained tasks retain their pages. No host worker is started.

std.flow

residual networkの最大流処理。capacityと頂点を検査します。augmentが変更する状態はVMのcheckpoint対象です。

```
import
```

```
import std.flow as flow;
```

module固有の公開型

型（公開field型）

```
record Edge {
  from: Int
  to: Int
  capacity: Int
  flow: Int
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Network {
}
```

共通型のfield・variant / このmoduleの実装を読む

std.flow.add

公開宣言

```
pub fn add(network: &mut Network, from: Int, to: Int, capacity: Int) -> Result<Int,String> effects
{}
```

効果 なし（effects {}）

canonical戻り型 **Result<Int,String>**

費用契約・原文 O(log pages) amortized per List update; stores two residual edges.

失敗・境界契約・原文 Range/Capacity before mutation; fatal work and history budgets apply.

std.flow.augment

公開宣言

```
pub fn augment(network: &mut Network, source: Int, sink: Int, limit: Int) -> Result<Int,String>
effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Int,String>**

費用契約・原文 $O(V^2 E \log \text{pages})$ worst case; $O(V)$ temporary List cells.

失敗・境界契約・原文 Range before mutation; fatal work and history budgets apply during mutation.

std.flow.create

公開宣言

```
pub fn create(vertices: Int, capacity: Int) -> Result<Network,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Network,String>**

費用契約・原文 $O(\text{vertices})$; vertices ≤ 262144 , capacity ≤ 524288 forward edges.

失敗・境界契約・原文 Capacity; fatal work and history budgets apply.

std.flow.edge

公開宣言

```
pub fn edge(network: &Network, id: Int) -> Result<Edge,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Edge,String>**

std.flow.edges

公開宣言

```
pub fn edges(network: &Network) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.flow.reachable

公開宣言

```
pub fn reachable(network: &Network, source: Int) -> Result<List<Bool>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Bool>,String>`

費用契約・原文 $O(V + E \log \text{pages})$; $O(V)$ result and scratch cells.

失敗・境界契約・原文 Range; fatal work and history budgets apply.

std.flow.vertices

公開宣言

```
pub fn vertices(network: &Network) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.geometry

幾何のprimitive。座標範囲、overflow、境界上の点や重複点の扱いを確認します。

```
import
```

```
import std.geometry as geometry;
```

module固有の公開型

型 (公開field型)

```
record Point {  
    x: Int  
    y: Int  
}
```

型 (公開field型)

```
record Vector {  
    x: Float  
    y: Float  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.geometry.convexHull

公開宣言

```
pub fn convexHull(input: &List<Point>, keepCollinear: Bool) -> Result<List<Point>, StdError> effects  
{}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Point>, StdError>**

費用契約・原文 O(n log n) comparisons and O(n) bounded exact predicates, n <=1048576; outputs distinct points
counterclockwise, no repeated first point.

失敗・境界契約・原文 NumericSize; keepCollinear includes boundary points, and an entirely collinear input returns sorted
distinct points.

std.geometry.cross

公開宣言

```
pub fn cross(a: Vector, b: Vector) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

std.geometry.dot

公開宣言

```
pub fn dot(a: Vector, b: Vector) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

std.geometry.intersects

公開宣言

```
pub fn intersects(a: Point, b: Point, c: Point, d: Point) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool, StdError>`

std.geometry.onSegment

公開宣言

```
pub fn onSegment(point: Point, a: Point, b: Point) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool, StdError>`

std.geometry.orientation

公開宣言	
<pre>pub fn orientation(a: Point, b: Point, c: Point) -> Result<Int,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<Int,StdError></code>
費用契約・原文	Fixed-size BigInt intermediates cover the full Int coordinate domain, including a signed determinant wider than 128 bits.

std.geometry.orientationFloat

公開宣言	
<pre>pub fn orientationFloat(a: Vector, b: Vector, c: Vector, tolerance: Float) -> Result<Int,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<Int,StdError></code>
失敗・境界契約・原文	NonFinite intermediates are rejected; negative tolerance returns NumericDomain. tolerance is absolute in determinant (area) units.

std.graph

boundedなgraph、BFS/DFS/Dijkstra、topological、Bellman-Ford、SCC、最小森林、LCA。
BFS未到達は-1、Dijkstra未到達はNoneです。

```
import  
  
import std.graph as graph;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Ancestors {  
  
}
```

型（公開field型）

```
struct Forest {  
    weight: Int  
    components: Int  
    froms: List<Int>  
    tos: List<Int>  
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Graph {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.graph.add

公開宣言

```
pub fn add(graph: &mut Graph, from: Int, to: Int, weight: Int) -> Result<Unit,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 Result<Unit,String>

std.graph.ancestors

公開宣言

```
pub fn ancestors(graph: &Graph, root: Int) -> Result<Ancestors,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Ancestors,String>`

std.graph.bellmanFord

公開宣言

```
pub fn bellmanFord(graph: &Graph, source: Int) -> Result<List<Option<Int>>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Option<Int>>,String>`

std.graph.bfs

公開宣言

```
pub fn bfs(graph: &Graph, source: Int) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.graph.components

公開宣言

```
pub fn components(graph: &Graph) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.graph.create

公開宣言

```
pub fn create(vertices: Int, capacity: Int) -> Result<Graph,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Graph,String>`

std.graph.dfs

公開宣言

```
pub fn dfs(graph: &Graph, source: Int) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.graph.dijkstra

公開宣言

```
pub fn dijkstra(graph: &Graph, source: Int) -> Result<List<Option<Int>>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Option<Int>>,String>`

std.graph.edges

公開宣言

```
pub fn edges(graph: &Graph) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.graph.lca

公開宣言

```
pub fn lca(tree: &Ancestors, a: Int, b: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.graph.minimumForest

公開宣言

```
pub fn minimumForest(graph: &Graph) -> Result<Forest,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Forest,String>`

std.graph.topological

公開宣言

```
pub fn topological(graph: &Graph) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.graph.vertices

公開宣言

```
pub fn vertices(graph: &Graph) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.graphLarge

native IntArrayを使う大きいgraphの構築と探索。頂点/辺は最大1,048,576、予算は別に適用されます。

```
import
```

```
import std.graphLarge as graphLarge;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Ancestors {  
  
}
```

型（公開field型）

```
struct Forest {  
    weight: Int  
    components: Int  
    froms: List<Int>  
    tos: List<Int>  
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Graph {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.graphLarge.add

公開宣言

```
pub fn add(graph: &mut Graph, from: Int, to: Int, weight: Int) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,String>`

費用契約・原文 Four COW point updates; bulk construction is preferred for large edge sets.

std.graphLarge.ancestors

公開宣言

```
pub fn ancestors(graph: &Graph, root: Int) -> Result<Ancestors,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Ancestors,String>`

std.graphLarge.bellmanFord

公開宣言

```
pub fn bellmanFord(graph: &Graph, source: Int) -> Result<List<Option<Int>>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Option<Int>>,String>`

std.graphLarge.bfs

公開宣言

```
pub fn bfs(graph: &Graph, source: Int) -> Result<IntArray,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<IntArray,String>**

費用契約・原文 O(vertices + edges) bounded native traversal, times page lookup; returns native IntArray, -1 for unreachable vertices.

std.graphLarge.components

公開宣言

```
pub fn components(graph: &Graph) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Int>,String>**

std.graphLarge.create

公開宣言

```
pub fn create(vertices: Int, capacity: Int) -> Result<Graph,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Graph,String>**

費用契約・原文 O(1) virtual-zero storage; pages are admitted as touched. Capacity is at most 1048576.

std.graphLarge.dfs

公開宣言

```
pub fn dfs(graph: &Graph, source: Int) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.graphLarge.dijkstra

公開宣言

```
pub fn dijkstra(graph: &Graph, source: Int) -> Result<List<Option<Int>>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Option<Int>>,String>`

std.graphLarge.edges

公開宣言

```
pub fn edges(graph: &Graph) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.graphLarge.fromEdges

公開宣言

```
pub fn fromEdges(vertices: Int, froms: &IntArray, tos: IntArray, weights: IntArray) ->
Result<Graph,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Graph,String>

費用契約・原文 O(edges log pages); native bulk adjacency builder, no per-edge VM callback or coordinate List.

失敗・境界契約・原文 Capacity/Shape or NumericIndex; input arrays are rank-one, equal length and retained immutable views.

std.graphLarge.lca

公開宣言

```
pub fn lca(tree: &Ancestors, a: Int, b: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Int,String>

std.graphLarge.minimumForest

公開宣言

```
pub fn minimumForest(graph: &Graph) -> Result<Forest,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Forest,String>

std.graphLarge.topological

公開宣言

```
pub fn topological(graph: &Graph) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.graphLarge.vertices

公開宣言

```
pub fn vertices(graph: &Graph) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.graphLargeAsync

IntArrayの辺からgraph構築とBFSを協調的に行います。部分状態はcheckpointと取消の対象です。

```
import

import std.graphLargeAsync as graphLargeAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.graphLargeAsync.bfs

公開宣言

```
pub async fn bfs(vertices: Int, froms: IntArray, tos: IntArray, source: Int) ->
Result<IntArray, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 Owns shared immutable edge snapshots; validates/builds/traverses/converts distances in steps of at most 4096 units, yielding between steps. Up to 1048576 vertices/edges; page COW and queue storage are charged to VM budgets.

失敗・境界契約・原文 NumericType/Shape/Size/Index/Domain; validates all endpoints before building adjacency. Returns shortest directed distances, -1 unreachable; no partial result after typed failure or cancellation. Fatal work/memory budgets apply.

std.gui

Viewとwidgetのmodel、scene、dispatch、編集。presentでstageしpublishでOSへ表示します。main taskとgui許可が必要です。

```
import

import std.gui as gui;
```

module固有の公開型

型alias

```
type GuiError = StdError;
```

variantとpayload型

```
enum Action {
    Activate(String),
    Changed(String, Bool),
    Close,
    Key(String, String),
    Resize(Int, Int)
}
```

型（公開field型）

```
record Event {
    kind: String
    x: Int
    y: Int
    key: String
    width: Int
    height: Int
}
```

型（公開field型）

```
record Rect {
    x: Int
    y: Int
    width: Int
    height: Int
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct View {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.gui.arrange

公開宣言

```
pub fn arrange(view: &mut View, ids: &List<String>, bounds: Rect, columns: Int, gap: Int) ->  
Result<Unit, StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 Result<Unit, StdError>

費用契約・原文 O(ids); prevalidates geometry and IDs before mutating the view.

std.gui.background

公開宣言

```
pub fn background(view: &mut View, color: Int) -> Result<Unit, StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 Result<Unit, StdError>

std.gui.bounds

公開宣言

```
pub fn bounds(view: &View) -> Rect effects {}
```

効果 なし（effects {}）

canonical戻り型 Rect

費用契約・原文 O(1); client area geometry for composing bounded overlays.

std.gui.button

公開宣言

```
pub fn button(view: &mut View, id: String, caption: String, x: Int, y: Int, width: Int, height: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.gui.checkbox

公開宣言

```
pub fn checkbox(view: &mut View, id: String, caption: String, checked: Bool, x: Int, y: Int, width: Int, height: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.gui.checked

公開宣言

```
pub fn checked(view: &View, id: String) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool, StdError>`

std.gui.close

公開宣言

```
pub fn close() -> Result<Unit, StdError> effects {gui}
```

効果 gui

canonical戻り型 Result<Unit, StdError>

std.gui.continueInput

公開宣言

```
pub fn continueInput() -> Unit effects {gui}
```

効果 gui

canonical戻り型 Unit

失敗・境界契約・原文 After revert, skips only events already delivered in this execution, including during replay.

std.gui.dispatch

公開宣言

```
pub fn dispatch(view: &mut View, event: Event) -> Option<Action> effects {}
```

効果 なし (effects {})

canonical戻り型 Option<Action>

費用契約・原文 Pointer scans in reverse draw order, at most 2048 widgets; Tab traverses controls.

std.gui.edit

公開宣言

```
pub fn edit(view: &mut View, id: String, event: Event) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.gui.focus

公開宣言

```
pub fn focus(view: &mut View, id: String) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 UnknownId/NotInteractive; retains the widget selection and scroll.

std.gui.focused

公開宣言

```
pub fn focused(view: &View) -> Option<String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<String>`

費用契約・原文 O(1); disabled or removed controls have no focus.

std.gui.graphemeEditing

公開宣言

```
pub fn graphemeEditing(view: &mut View, enabled: Bool) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 Enables cluster editing; existing scalar positions round forward to a boundary. Default is false.

std.gui.graphemeMode

公開宣言

```
pub fn graphemeMode(view: &View) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 `Bool`

std.gui.label

公開宣言

```
pub fn label(view: &mut View, id: String, caption: String, x: Int, y: Int, width: Int, height: Int)
-> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.gui.nextEvent

公開宣言

```
pub fn nextEvent() -> Result<Event, StdError> effects {gui}
```

効果 gui

canonical戻り型 `Result<Event, StdError>`

失敗・境界契約・原文 GUI must be published first; input is journaled, blocking, and replayable.

std.gui.overlay

公開宣言

```
pub fn overlay(base: &View, layer: &View) -> Result<View, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<View, StdError>`

費用契約・原文 O(combined widget count); at most 2048 controls. Copies bounded VM metadata; captions remain shared.

失敗・境界契約・原文 GuiInvalidSize/EditingModeMismatch/WidgetLimit/DuplicateId; input views remain unchanged. Layer controls draw above base controls and take focus when the layer has focus.

std.gui.pollEvent

公開宣言

```
pub fn pollEvent() -> Result<Option<Event>, StdError> effects {gui}
```

効果 gui

canonical戻り型 `Result<Option<Event>, StdError>`

std.gui.present

公開宣言

```
pub fn present(view: &View) -> Result<Unit, StdError> effects {gui}
```

効果 gui

canonical戻り型 Result<Unit, StdError>

失敗・境界契約・原文 Queues a scene; publish may fail with GuiUnavailable or PublishPartiallyApplied.

std.gui.rectangle

公開宣言

```
pub fn rectangle(view: &mut View, id: String, x: Int, y: Int, width: Int, height: Int, color: Int)
-> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

std.gui.remove

公開宣言

```
pub fn remove(view: &mut View, id: String) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

費用契約・原文 O(widgets); rebuilds indices and preserves remaining selection, scroll and focus.

失敗・境界契約・原文 UnknownId; unknown input leaves the view unchanged.

std.gui.resize

公開宣言

```
pub fn resize(view: &mut View, width: Int, height: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.gui.scene

公開宣言

```
pub fn scene(view: &View) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

費用契約・原文 $O(\text{widget count} + \text{caption bytes})$; serialized scene at most 1 MiB.

std.gui.scrollPosition

公開宣言

```
pub fn scrollPosition(view: &View, id: String) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

失敗・境界契約・原文 `UnknownId/NotTextArea`; logical first visible line.

std.gui.selection

公開宣言

```
pub fn selection(view: &View, id: String) -> Result<Tuple<Int,Int>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Tuple<Int,Int>,StdError>`

std.gui.setChecked

公開宣言

```
pub fn setChecked(view: &mut View, id: String, checked: Bool) -> Result<Unit,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,StdError>`

失敗・境界契約・原文 UnknownId/NotCheckbox; updates only the logical view.

std.gui.setEnabled

公開宣言

```
pub fn setEnabled(view: &mut View, id: String, enabled: Bool) -> Result<Unit,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,StdError>`

std.gui.setScrollPosition

公開宣言

```
pub fn setScrollPosition(view: &mut View, id: String, line: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

失敗・境界契約・原文 UnknownId/NotTextArea/InvalidScroll; prevalidates without changing focus or text.

std.gui.setSelection

公開宣言

```
pub fn setSelection(view: &mut View, id: String, cursor: Int, anchor: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

std.gui.setText

公開宣言

```
pub fn setText(view: &mut View, id: String, caption: String) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

std.gui.style

公開宣言

```
pub fn style(view: &mut View, id: String, foreground: Int, background: Int) ->
Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 UnknownId/InvalidColor; retains every other widget property.

std.gui.text

公開宣言

```
pub fn text(view: &View, id: String) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.gui.textArea

公開宣言

```
pub fn textArea(view: &mut View, id: String, caption: String, x: Int, y: Int, width: Int, height:
Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.gui.textBox

公開宣言

```
pub fn textBox(view: &mut View, id: String, caption: String, x: Int, y: Int, width: Int, height: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 Single-line editing with Unicode-scalar selection; 4096 UTF-8 bytes.

std.gui.window

公開宣言

```
pub fn window(title: String, width: Int, height: Int) -> Result<View, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<View, StdError>`

失敗・境界契約・原文 InvalidSize/InvalidTitle; 64..4096 client pixels per axis.

std.guiDialog

native file選択。gui/external/tasksの領域内でTaskを作り外でawaitします。選択したパスへのFile権限は別です。

```
import
```

```
import std.guiDialog as guiDialog;
```

共通型のfield・variant / このmoduleの実装を読む

std.guiDialog.openFile

公開宣言

```
pub fn openFile(title: String, initial: String) -> Task<Result<Option<String>, StdError>> effects
{external, gui, tasks}
```

効果 external, gui, tasks

canonical戻り型 `Task<Result<Option<String>, StdError>>`

費用契約・原文 Nonblocking Task over a bounded native request and response. Revert reuses the recorded operation; replay opens no OS dialog. Returned paths grant no file capability and the API never reads file content.

失敗・境界契約・原文 Requires explicit external region and gui/external/tasks permissions; application task only. One native dialog per process. GuiDialogBusy/Unavailable/Thread/Title/Path/PathLimit/PathEncoding are typed. Cancellation by the user returns None; task cancellation closes the native request. Linux uses system GTK 3; Windows uses Win32 common dialogs.

std.guiDialog.saveFile

公開宣言	
<pre>pub fn saveFile(title: String, initial: String) -> Task<Result<Option<String>,StdError>> effects {external, gui, tasks}</pre>	
効果	external, gui, tasks
canonical戻り型	Task<Result<Option<String>,StdError>>
費用契約・原文	At most 256 UTF-8 title bytes and 4096 path bytes; native UI state and physical worker lifetime are not checkpointed. external live retains only the task/checkpoint-owned result, without complete record/replay support.
失敗・境界契約・原文	Same boundary and lifetime as openFile. Selecting a destination does not create or truncate it. Native overwrite confirmation asks about the selected name; applications still perform and publish their own file operation.

std.guiForm

入力項目と検査、Viewへのrenderの補助。model更新後にrender/present/publishが必要です。

```
import
```

```
import std.guiForm as guiForm;
```

module固有の公開型

型alias

```
type FormError = StdError;
```

variantとpayload型

```
enum Action {
    Close,
    Edited(String),
    Invalid,
    Resize(Int, Int),
    Scrolled(Int),
    Submitted(Json)
}
```

型（公開field型）

```
record Field {
    id: String
    caption: String
    kind: Kind
    required: Bool
    maxBytes: Int
    initial: String
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Form {
}
```


variantとpayload型

```
enum Kind {  
    Check,  
    Integer(Int, Int),  
    Multiline,  
    Real(Float, Float),  
    Text,  
    Timestamp  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.guiForm.create

公開宣言

```
pub fn create(fields: &List<Field>, visibleRows: Int) -> Result<Form, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Form, StdError>**

費用契約・原文 O(fields + schema bytes); at most 64 fields, 1..32 visible rows, maxBytes 1..4096 per value.

失敗・境界契約・原文 Schema/Id/Text; initial numeric input can be incomplete until validate.

std.guiForm.dispatch

公開宣言

```
pub fn dispatch(form: &mut Form, panel: &mut $import$gui$View, event: $import$gui$Event, prefix:  
String) -> Result<Option<Action>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Option<Action>, StdError>**

失敗・境界契約・原文 InvalidText; rejected over-limit edits restore the view text and selection. Submit validates all fields.

std.guiForm.error

公開宣言

```
pub fn error(form: &Form, id: String) -> Result<Option<String>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<String>,StdError>`

std.guiForm.position

公開宣言

```
pub fn position(form: &Form) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.guiForm.render

公開宣言

```
pub fn render(form: &Form, title: String, width: Int, height: Int, rowHeight: Int, prefix: String)
-> Result<$import$gui$View,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<$import$gui$View,StdError>`

費用契約・原文 O(visible fields); new logical view only, preserves model selection/focus and shows validation errors.

失敗・境界契約・原文 Bounds/Id; rowHeight 64..128, caller chooses enough height and publishes the returned view.

std.guiForm.reset

公開宣言

```
pub fn reset(form: &mut Form) -> Unit effects {}
```

効果 なし (effects {})

canonical戻り型 `Unit`

std.guiForm.scroll

公開宣言

```
pub fn scroll(form: &mut Form, delta: Int) -> Unit effects {}
```

効果 なし (effects {})

canonical戻り型 `Unit`

std.guiForm.setValue

公開宣言

```
pub fn setValue(form: &mut Form, id: String, value: String) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 `UnknownId/Text`; validates input bounds before mutation, not its final typed value.

std.guiForm.toJson

公開宣言

```
pub fn toJson(form: &Form) -> Result<Json, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Json, StdError>**

失敗・境界契約・原文 Required/Integer/Real/Range/Timestamp; readonly, returns typed values or the first error with field index.

std.guiForm.validate

公開宣言

```
pub fn validate(form: &mut Form) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 **Bool**

費用契約・原文 O(all fields + value bytes); stores every error, never discards the edited text.

std.guiForm.value

公開宣言

```
pub fn value(form: &Form, id: String) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<String, StdError>**

std.guiMenu

VM内のmenuとshortcut model。表示はsceneへの描画とpublishです。OS全体のmenu状態をrevertするものではありません。

```
import
```

```
import std.guiMenu as guiMenu;
```

module固有の公開型

型（公開field型）

```
record Item {  
  id: String  
  label: String  
  shortcut: String  
  enabled: Bool  
  checkable: Bool  
  checked: Bool  
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Menu {  
  
}
```

型（公開field型）

```
record Outcome {  
  consumed: Bool  
  command: Option<Item>  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.guiMenu.add

公開宣言

```
pub fn add(menu: &mut Menu, item: Item) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 GuiMenuLimit/Id/Text/Shortcut/State; at most 256 unique commands, short captions and explicit shortcuts.

std.guiMenu.checked

公開宣言

```
pub fn checked(menu: &Menu, id: String) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool, StdError>`

std.guiMenu.close

公開宣言

```
pub fn close(menu: &mut Menu) -> Unit effects {}
```

効果 なし (effects {})

canonical戻り型 `Unit`

std.guiMenu.create

公開宣言

```
pub fn create() -> Menu effects {}
```

効果 なし (effects {})

canonical戻り型 **Menu**

std.guiMenu.dispatch

公開宣言

```
pub fn dispatch(menu: &mut Menu, event: $import$gui$Event) -> Outcome effects {}
```

効果 なし (effects {})

canonical戻り型 **Outcome**

費用契約・原文 At most 256 commands for shortcut lookup or enabled navigation. Unconsumed input may be delivered to the underlying view.

std.guiMenu.draw

公開宣言

```
pub fn draw(menu: &Menu, base: &$import$gui$View, prefix: String) ->  
Result<$import$gui$View, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<\$import\$gui\$View, StdError>**

失敗・境界契約・原文 GuiMenuBounds/Id or view composition errors; base and menu remain unchanged, focus/scroll retained in base. Prefix bounds prevent oversized native widget IDs.

std.guiMenu.isOpen

公開宣言

```
pub fn isOpen(menu: &Menu) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 Bool

std.guiMenu.open

公開宣言

```
pub fn open(menu: &mut Menu, x: Int, y: Int, width: Int, bounds: $import$gui$Rect) ->  
Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit, StdError>

失敗・境界契約・原文 GuiMenuEmpty/Bounds; geometry prevalidated before changing state. Long menus use a bounded visible viewport.

std.guiMenu.selected

公開宣言

```
pub fn selected(menu: &Menu) -> Option<String> effects {}
```

効果 なし (effects {})

canonical戻り型 Option<String>

std.guiMenu.setChecked

公開宣言

```
pub fn setChecked(menu: &mut Menu, id: String, checked: Bool) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.guiMenu.setEnabled

公開宣言

```
pub fn setEnabled(menu: &mut Menu, id: String, enabled: Bool) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.guiTable

boundedな表modelとviewportの描画。全データの保持と表示範囲を分けて考えます。

```
import
```

```
import std.guiTable as guiTable;
```

module固有の公開型

型alias

```
type TableError = StdError;
```

variantとpayload型

```
enum Action {
    Close,
    Resize(Int, Int),
    Scrolled(Int),
    Selected(Int)
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Table {
}
```

共通型のfield・variant / このmoduleの実装を読む

std.guiTable.cell

公開宣言

```
pub fn cell(table: &Table, row: Int, column: Int) -> Result<String,StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<String,StdError>**

std.guiTable.columns

公開宣言

```
pub fn columns(table: &Table) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.guiTable.create

公開宣言

```
pub fn create(headers: Frozen<List<String>>, cells: Frozen<List<String>>, visibleRows: Int) -> Result<Table, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Table, StdError>`

費用契約・原文 `O(cells)`; frozen row-major data is shared, not materialized in the scene.

失敗・境界契約・原文 `Shape/Capacity/Text`; 1..16 columns, at most 65536 rows and 1048576 cells, 16MiB text.

std.guiTable.dispatch

公開宣言

```
pub fn dispatch(table: &mut Table, view: &mut $import$gui$View, event: $import$gui$Event, prefix: String) -> Result<Option<Action>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Action>, StdError>`

失敗・境界契約・原文 `Range/Id`; wheel and Up/Down/Home/End move the selected row or viewport.

std.guiTable.position

公開宣言

```
pub fn position(table: &Table) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.guiTable.render

公開宣言

```
pub fn render(table: &Table, title: String, width: Int, height: Int, rowHeight: Int, prefix: String) -> Result<$import$gui$View,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<$import$gui$View,StdError>`

費用契約・原文 $O(\text{visible rows} * \text{columns})$; new view keeps failures away from the caller's published scene.

失敗・境界契約・原文 `Bounds/Id`; equal-width columns, row height 16..128, caller publishes the returned view.

std.guiTable.rows

公開宣言

```
pub fn rows(table: &Table) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.guiTable.scroll

公開宣言

```
pub fn scroll(table: &mut Table, delta: Int) -> Unit effects {}
```

効果 なし (effects {})

canonical戻り型 `Unit`

費用契約・原文 O(1); clamps arbitrary signed deltas before adding them.

std.guiTable.select

公開宣言

```
pub fn select(table: &mut Table, row: Int) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.guiTable.selection

公開宣言

```
pub fn selection(table: &Table) -> Option<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Int>`

std.guiWindows

名前付き複数画面のstage・入力・close。Undo後はcontinueInputと再表示。同期待機とnextEventAnyAsyncの違いを確認します。

```
import

import std.guiWindows as guiWindows;
```

module固有の公開型

型alias

```
type GuiError = StdError;
```

型（公開field型）

```
record WindowEvent {
    window: String
    event: $import$gui$Event
}
```

共通型のfield・variant / このmoduleの実装を読む

std.guiWindows.close

公開宣言

```
pub fn close(id: String) -> Result<Unit,StdError> effects {gui}
```

効果 gui

canonical戻り型 Result<Unit,StdError>

失敗・境界契約・原文 Unknown IDs return GuiWindowNotPublished; publish closes only the named surface.

std.guiWindows.continueInput

公開宣言

```
pub fn continueInput() -> Unit effects {gui}
```

効果 gui

canonical戻り型 Unit

失敗・境界契約・原文 After revert skips only named-window events already delivered in this execution; old gui input has a separate cursor.

std.guiWindows.nextEvent

公開宣言

```
pub fn nextEvent(id: String) -> Result<$import$gui$Event,StdError> effects {gui}
```

効果 gui

canonical戻り型 Result<\$import\$gui\$Event,StdError>

失敗・境界契約・原文 Blocks without scheduling VM tasks; wrong scoped fixture ID returns GuiWindowEventTapeEnd.

std.guiWindows.nextEventAny

公開宣言

```
pub fn nextEventAny() -> Result<WindowEvent,StdError> effects {gui}
```

効果 gui

canonical戻り型 Result<WindowEvent,StdError>

失敗・境界契約・原文 Blocks on native input across active windows; performs no VM task scheduling during the wait. Use pollAny with external-task completion polling; explicit VM task handoff is planned separately.

std.guiWindows.nextEventAnyAsync

公開宣言

```
pub fn nextEventAnyAsync() -> Task<Result<WindowEvent, StdError>> effects {gui, tasks}
```

効果

gui, tasks

canonical戻り型

`Task<Result<WindowEvent, StdError>>`

費用契約・原文

Cold task; nonblocking main-thread polling permits other VM and host tasks to advance. Empty waits use compressed observations.

失敗・境界契約・原文

One active any-window wait; competing pending waits return `GuiWaitBusy`. `GuiClosed`/`GuiWindowNotPublished`/`GuiWindowEventTapeEnd` are typed; cancellation consumes no subsequent event. Factory is called from the application task; staging remains main-task-only.

std.guiWindows.nextEventAnyLiveAsync

公開宣言

```
pub fn nextEventAnyLiveAsync() -> Task<Result<WindowEvent, StdError>> effects {external, gui, live, tasks}
```

効果

external, gui, live, tasks

canonical戻り型

`Task<Result<WindowEvent, StdError>>`

費用契約・原文

Nonblocking main-thread polling; retains only results owned by tasks/checkpoints, no input observation tape.

失敗・境界契約・原文

Requires external live region and gui/tasks/external/live permissions; one shared any-window wait. Restored task reuses its event; new factory consumes new input. Complete recording/replay unsupported; cancellation consumes no event.

std.guiWindows.poll

公開宣言

```
pub fn poll(id: String) -> Result<Option<$import$gui$Event>, StdError> effects {gui}
```

効果 gui

canonical戻り型 `Result<Option<$import$gui$Event>, StdError>`

失敗・境界契約・原文 Requires a published ID; None means no input; scoped fixture polling preserves another window event.

std.guiWindows.pollAny

公開宣言

```
pub fn pollAny() -> Result<Option<WindowEvent>, StdError> effects {gui}
```

効果 gui

canonical戻り型 `Result<Option<WindowEvent>, StdError>`

費用契約・原文 Bounded round-robin polling; does not block; combines native surfaces without merging their IDs.

std.guiWindows.present

公開宣言

```
pub fn present(id: String, view: &$import$gui$View) -> Result<Unit, StdError> effects {gui}
```

効果 gui

canonical戻り型 `Result<Unit, StdError>`

失敗・境界契約・原文 Stages a named surface; publish applies it. IDs are nonempty, <=128 UTF-8 bytes, NUL-free. At most 16 named windows / 16M client pixels including the legacy window.

std.heap

Listを使う最小heap。比較は負/0/正で全操作に一貫して使います。capacity不足はResult、空のpop/peekはNoneです。

```
import
```

```
import std.heap as heap;
```

共通型のfield・variant / このmoduleの実装を読む

std.heap.peek

公開宣言

```
pub fn peek<T: Share>(values: &List<T>) -> Option<T> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<T>`

generic契約 `T: Share`

std.heap.pop

公開宣言

```
pub fn pop<T: Share>(values: &mut List<T>, compare: fn(T,T)->Int!{}~{Send+Share}) -> Option<T> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<T>`

generic契約 `T: Share`

std.heap.push

公開宣言

```
pub fn push<T: Share>(values: &mut List<T>, value: T, capacity: Int, compare: fn(T,T)->Int!{}~{Send+Share}) -> Result<Unit,String> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	Result<Unit,String>
--------------	---------------------

generic契約	T: Share
-----------	----------

std.http

HTTP/HTTPS clientと逐次transfer。external/network/tasks、期限・byte上限・TLS・所有権が契約です。自動redirect/retryはしません。

```
import

import std.http as http;
```

module固有の公開型

型（公開field型）

```
record Request {
  method: String
  url: String
  body: Bytes
  timeoutMillis: Int
  maxBytes: Int
  headers: Frozen<List<HttpHeader>>
  ca: Bytes
  credential: String
}
```

共通型のfield・variant / このmoduleの実装を読む

std.http.close

公開宣言

```
pub fn close(value: &mut HttpDownload) -> Task<Result<Unit,HttpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Unit,HttpError>>`

費用契約・原文 O(pending reads); physical connection never resurrects after revert.

失敗・境界契約・原文 HttpError; cancels pending reads and releases the resource.

std.http.closeUpload

公開宣言

```
pub fn closeUpload(value: &mut HttpUpload) -> Task<Result<Unit,HttpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<Unit,HttpError>>

費用契約・原文 O(pending operations); cancels the driver and queued writes.

失敗・境界契約・原文 HttpError; scope exit closes the resource automatically.

std.http.component

公開宣言

```
pub fn component(value: String) -> Result<String,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<String,StdError>

費用契約・原文 O(UTF-8 bytes); percent-encodes a query value or path component.

失敗・境界契約・原文 Limit when the encoded component would exceed 1 MiB; does not encode an entire URL.

std.http.configured

公開宣言

```
pub fn configured(method: String, url: String, body: Bytes, deadlineMillis: Int, maxBytes: Int,
headers: Frozen<List<HTTPHeader>>, ca: Bytes) -> Task<Result<HttpResponse,HttpError>> effects
{external, network, tasks}
```

効果	external, network, tasks
----	--------------------------

canonical戻り型	<code>Task<Result<HttpResponse,HttpError>></code>
--------------	---

費用契約・原文	O(request bytes + response bytes); bounded 16 origin/CA client cache, native lease allowance and cold-client setup; same-origin connections are reused.
---------	---

失敗・境界契約・原文	HttpError after awaiting; nonempty PEM certificates add trust, hostname/chain verification remains enabled. Client admission can return HttpMemoryLimit/NotSent before physical submission.
------------	---

std.http.credential

公開宣言

```
pub fn credential(alias: String, value: Secret<String>) -> Result<Unit,StdError> effects {external,
network, tasks}
```

効果	external, network, tasks
----	--------------------------

canonical戻り型	<code>Result<Unit,StdError></code>
--------------	--

費用契約・原文	O(secret bytes); retains at most 64 immutable credential aliases.
---------	---

失敗・境界契約・原文	HttpCredential/HttpCredentialImmutable/HttpCredentialLimit; accepts Secret<String>, never journals the private value.
------------	---

std.http.download

公開宣言

```
pub fn download(value: Request) -> Task<Result<HttpDownload,HttpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<HttpDownload,HttpError>>`

費用契約・原文 O(headers); body is consumed incrementally by read, total limit up to 1 GiB.

失敗・境界契約・原文 HttpError; scope-owned connection, deadline includes all subsequent reads.

std.http.finish

公開宣言

```
pub fn finish(value: &mut HttpUpload) -> Task<Result<HttpResponse,HttpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<HttpResponse,HttpError>>`

費用契約・原文 O(response bytes); ends the request body and awaits the bounded response.

失敗・境界契約・原文 HttpError; closing or cancellation does not undo bytes already sent.

std.http.get

公開宣言

```
pub fn get(url: String, deadlineMillis: Int, maxBytes: Int) -> Task<Result<HttpResponse,HttpError>>
effects {external, network, tasks}
```

効果	external, network, tasks
----	--------------------------

canonical戻り型	<code>Task<Result<HttpResponse,HttpError>></code>
--------------	---

費用契約・原文	O(response bytes), with bounded origin/CA client caching and cold-client memory admission; returns a Task.
---------	--

失敗・境界契約・原文	HttpError after awaiting; no automatic redirect or retry.
------------	---

std.http.read

公開宣言

```
pub fn read(value: &mut HttpDownload, maxBytes: Int) -> Task<Result<Option<Bytes>,HttpError>>
effects {external, network, tasks}
```

効果	external, network, tasks
----	--------------------------

canonical戻り型	<code>Task<Result<Option<Bytes>,HttpError>></code>
--------------	--

費用契約・原文	O(chunk bytes); retains a bounded transport buffer, chunks of 1..65536 bytes.
---------	---

失敗・境界契約・原文	HttpError; one outstanding read per connection, EOF returns None.
------------	---

std.http.request

公開宣言

```
pub fn request(method: String, url: String, body: Bytes, deadlineMillis: Int, maxBytes: Int) -> Task<Result<HttpResponse,HttpError>> effects {external, network, tasks}
```

効果	external, network, tasks
----	--------------------------

canonical戻り型	<code>Task<Result<HttpResponse,HttpError>></code>
--------------	---

費用契約・原文	$O(\text{request bytes} + \text{response bytes})$; at most eight native operations run concurrently.
---------	---

失敗・境界契約・原文	HttpError after awaiting; requires external region, deadline 1..120000 ms and body limit 1..4194304 bytes.
------------	--

std.http.send

公開宣言

```
pub fn send(value: Request) -> Task<Result<HttpResponse,HttpError>> effects {external, network, tasks}
```

効果	external, network, tasks
----	--------------------------

canonical戻り型	<code>Task<Result<HttpResponse,HttpError>></code>
--------------	---

費用契約・原文	$O(\text{request bytes} + \text{response bytes})$; public request uses a credential alias.
---------	---

失敗・境界契約・原文	HttpError after awaiting; fingerprint contains the public credential alias, never the private value.
------------	--

std.http.upload

公開宣言

```
pub fn upload(value: Request, maxUploadBytes: Int) -> Task<Result<HttpUpload,HttpError>> effects
{external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<HttpUpload,HttpError>>`

費用契約・原文 $O(\text{headers})$; bounded producer queue, total upload up to 1 GiB.

失敗・境界契約・原文 HttpError; starts a request without holding the complete request body.

std.http.write

公開宣言

```
pub fn write(value: &mut HttpUpload, body: Bytes) -> Task<Result<Unit,HttpError>> effects
{external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Unit,HttpError>>`

費用契約・原文 $O(\text{chunk bytes})$; queues at most one chunk, each at most 65536 bytes.

失敗・境界契約・原文 HttpError; acknowledgement means queued, remote application remains Unknown.

std.httpRouter

method/pathをrouteへ対応させる補助。認証・認可・schema検査を自動完成するframeworkではありません。

```
import
```

```
import std.httpRouter as httpRouter;
```

module固有の公開型

型（公開field型）

```
record Route {  
  method: String  
  path: String  
  handler: String  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.httpRouter.allows

公開宣言

```
pub fn allows(routes: Frozen<List<Route>>, path: String) -> Bool effects {}
```

効果 なし（effects {}）

canonical戻り型 **Bool**

費用契約・原文 $O(\text{route count} * \text{compared path bytes})$.

失敗・境界契約・原文 false when the path has no route; useful to distinguish missing paths from unsupported methods.

std.httpRouter.find

公開宣言

```
pub fn find(routes: Frozen<List<Route>>, method: String, path: String) -> Option<Route> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	Option<Route>
--------------	---------------

費用契約・原文	O(route count * compared text bytes); first matching route wins.
---------	--

失敗・境界契約・原文	None when no exact match; method '*' accepts every method. Paths retain percent encoding; no implicit decoding, prefix match or regex.
------------	--

std.httpServer

boundedなHTTP listener/request/response。外部作用でありrevertで応答を消しません。未応答resourceとlistenerの寿命を管理します。

```
import

import std.httpServer as httpServer;
```

共通型のfield・variant / このmoduleの実装を読む

std.httpServer.bearerCredential

公開宣言

```
pub fn bearerCredential(alias: String, token: Secret<String>) -> Result<Unit,HttpServerError>
effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Result<Unit,HttpServerError>`

費用契約・原文 O(token bytes); at most 16 immutable aliases, 16KiB reserved per configuration.

失敗・境界契約・原文 HttpServerError; external boundary and Secret<String> required; 16..4096 ASCII graphic bytes, never included in operation records.

std.httpServer.close

公開宣言

```
pub fn close(server: &mut HttpServer, deadlineMillis: Int) -> Task<Result<Unit,HttpServerError>>
effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Unit,HttpServerError>>`

費用契約・原文 O(active connections and requests); aborts all physical connections and releases pending requests.

失敗・境界契約・原文 HttpServerError; neither explicit close nor scope exit can be undone by revert.

std.httpServer.closeRequest

公開宣言

```
pub fn closeRequest(request: &mut HttpServerRequest, deadlineMillis: Int) ->
Task<Result<Unit,HttpServerError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<Unit,HttpServerError>>

費用契約・原文 O(1); releases a request without an application reply; an active peer receives 503.

失敗・境界契約・原文 HttpServerError; a consumed/disconnected request cannot be replied to again.

std.httpServer.configured

公開宣言

```
pub fn configured(address: String, port: Int, maxBodyBytes: Int, maxConnections: Int,
lifetimeMillis: Int, deadlineMillis: Int) -> Task<Result<HttpServer,HttpServerError>> effects
{external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<HttpServer,HttpServerError>>

費用契約・原文 O(maxBodyBytes * maxConnections); bounded request/response bodies, headers and queue.

失敗・境界契約・原文 HttpServerError; body 1..1048576 bytes, connections 1..8, whole connection lifetime 1..120000 ms, deadline 1..120000 ms.

std.httpServer.configuredTls

公開宣言

```
pub fn configuredTls(alias: String, address: String, port: Int, maxBodyBytes: Int, maxConnections: Int, lifetimeMillis: Int, deadlineMillis: Int) -> Task<Result<HttpServer,HttpServerError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<HttpServer,HttpServerError>>

費用契約・原文 O(maxBodyBytes * maxConnections); adds 4MiB TLS memory reservation per connection.

失敗・境界契約・原文 HttpServerError; registered alias required, TLS 1.2/1.3 HTTP/1.1; lifetime includes the handshake. Limits match configured.

std.httpServer.configuredTlsAuthenticated

公開宣言

```
pub fn configuredTlsAuthenticated(tlsAlias: String, authAlias: String, address: String, port: Int, maxBodyBytes: Int, maxConnections: Int, lifetimeMillis: Int, deadlineMillis: Int) -> Task<Result<HttpServer,HttpServerError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<HttpServer,HttpServerError>>

費用契約・原文 O(configured connection capacity); includes TLS memory reservations; authentication is checked before VM input capture.

失敗・境界契約・原文 HttpServerError; requires TLS and bearer aliases. Missing/wrong/duplicate authorization receives 401 and closes; accepted authorization is removed from recorded requests.

std.httpServer.listen

公開宣言

```
pub fn listen(address: String, port: Int, deadlineMillis: Int) ->
  Task<Result<HttpServer,HttpServerError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<HttpServer,HttpServerError>>

費用契約・原文 O(configured connection capacity); reserves buffers before binding, at most four listeners.

失敗・境界契約・原文 HttpServerError; numeric bind address, port 0 chooses a free port. Plain HTTP/1.1; physical connections cannot be restored.

std.httpServer.next

公開宣言

```
pub fn next(server: &mut HttpServer, deadlineMillis: Int) ->
  Task<Result<HttpRequest,HttpServerError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<HttpRequest,HttpServerError>>

費用契約・原文 O(request bytes); one outstanding wait per server. Request path/query retain URI encoding.

失敗・境界契約・原文 HttpServerError; cancellation ends this wait, leaving the listener available; closing the listener cancels pending waits.

std.httpServer.respond

公開宣言

```
pub fn respond(request: &mut HttpRequest, status: Int, headers: Frozen<List<HttpHeader>>,
body: Bytes, deadlineMillis: Int) -> Task<Result<Unit,HttpServerError>> effects {external, network,
tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Unit,HttpServerError>>`

費用契約・原文 O(header bytes); body storage is shared with the transport; each request accepts one response.

失敗・境界契約・原文 HttpServerError; success means queued, not peer processing. No automatic retry; restored task receipts never resend. Framing headers are managed by Hyper.

std.httpServer.tlsCredential

公開宣言

```
pub fn tlsCredential(alias: String, certificate: Bytes, key: Secret<String>) ->
Result<Unit,HttpServerError> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Result<Unit,HttpServerError>`

費用契約・原文 O(PEM bytes); at most 16 immutable private configurations, 512KiB reserved per alias.

失敗・境界契約・原文 HttpServerError; external boundary required, PEM certificate/key each at most 64KiB. Only the alias enters operation records.

std.integer

Resultを返すchecked Int算術。通常の+等でfatal overflowになる境界を、回復可能なerrorへするために選びます。

```
import
```

```
import std.integer as integer;
```

共通型のfield・variant / このmoduleの実装を読む

std.integer.add

公開宣言

```
pub fn add(a: Int, b: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Int,String>**

std.integer.combination

公開宣言

```
pub fn combination(n: Int, k: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Int,String>**

std.integer.extendedGcd

公開宣言

```
pub fn extendedGcd(a: Int, b: Int) -> Result<Tuple<Int,Int,Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Tuple<Int,Int,Int>,String>`

std.integer.factors

公開宣言

```
pub fn factors(n: Int) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.integer.gcd

公開宣言

```
pub fn gcd(a: Int, b: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.integer.lcm

公開宣言

```
pub fn lcm(a: Int, b: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.integer.multiply

公開宣言

```
pub fn multiply(a: Int, b: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.integer.primes

公開宣言

```
pub fn primes(limit: Int) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.json

Jsonのparse/stringifyと型別取得。JSONの妥当性とapplication schemaは別に検査します。
Stringを自動でIntに変換しません。

```
import
```

```
import std.json as json;
```

共通型のfield・variant / このmoduleの実装を読む

std.json.array

公開宣言

```
pub fn array(value: Json) -> Option<Frozen<List<Json>>> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Frozen<List<Json>>>`

std.json.boolean

公開宣言

```
pub fn boolean(value: Json) -> Option<Bool> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Bool>`

std.json.floating

公開宣言

```
pub fn floating(value: Json) -> Option<Float> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Float>`

std.json.get

公開宣言

```
pub fn get(value: Json, key: String) -> Option<Json> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Json>`

std.json.integer

公開宣言

```
pub fn integer(value: Json) -> Option<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Int>`

std.json.keys

公開宣言

```
pub fn keys(value: Json) -> List<String> effects {}
```

効果 なし (effects {})

canonical戻り型 `List<String>`

std.json.kind

公開宣言

```
pub fn kind(value: Json) -> String effects {}
```

効果 なし (effects {})

canonical戻り型 `String`

std.json.parse

公開宣言

```
pub fn parse(text: String) -> Result<Json,JsonError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json,JsonError>`

std.json.parseBytes

公開宣言

```
pub fn parseBytes(bytes: Bytes) -> Result<Json,JsonError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json,JsonError>`

std.json.stringify

公開宣言

```
pub fn stringify(value: Json) -> Result<String,JsonError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,JsonError>`

std.json.text

公開宣言

```
pub fn text(value: Json) -> Option<String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<String>`

std.jsonStream

増分JSONをeventとして読む状態。chunk間のUTF-8/tokenと構造を扱い、number tokenのexact変換は必要な型へ明示します。

```
import
```

```
import std.jsonStream as jsonStream;
```

module固有の公開型

型alias

```
type Event = JsonStreamEvent;
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Reader {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.jsonStream.cancel

公開宣言

```
pub fn cancel(reader: &mut Reader) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Unit, StdError>**

std.jsonStream.feed

公開宣言

```
pub fn feed(reader: &mut Reader, chunk: Bytes, finish: Bool) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.jsonStream.finish

公開宣言

```
pub fn finish(reader: &mut Reader) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

std.jsonStream.next

公開宣言

```
pub fn next(reader: &mut Reader) -> Result<Option<JsonStreamEvent>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<JsonStreamEvent>, StdError>`

std.jsonStream.position

公開宣言

```
pub fn position(reader: &Reader) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

std.jsonStream.reader

公開宣言

```
pub fn reader() -> Result<Reader, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Reader, StdError>`

std.lazySegment

lazy propagationを持つ区間data structure。query/updateとcallbackの演算契約を揃え、所有権と失敗時の状態を確認します。

```
import

import std.lazySegment as lazySegment;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct LazySegment<T,U> {

}
```

共通型のfield・variant / このmoduleの実装を読む

std.lazySegment.build

公開宣言

```
pub fn build<T: Share, U: Share>(input: &List<T>, identity: T, combine: fn(T,T)->T!{ }~{Send+Share})
-> Result<LazySegment<T,U>,String> effects {}
```

効果	なし（effects {}）
canonical戻り型	Result<LazySegment<T,U>,String>
generic契約	T: Share, U: Share
費用契約・原文	O(n) combine calls and O(n) storage; n <= 1048576. Callback laws are caller obligations.

std.lazySegment.length

公開宣言

```
pub fn length<T: Share, U: Share>(tree: &LazySegment<T,U>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

generic契約 T: Share, U: Share

std.lazySegment.query

公開宣言

```
pub fn query<T: Share, U: Share>(tree: &mut LazySegment<T,U>, start: Int, end: Int, combine:
fn(T,T)->T!{}~{Send+Share}, apply: fn(T,U,Int)->T!{}~{Send+Share}, compose: fn(U,U)->U!{}~
{Send+Share}) -> Result<T,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<T,String>

generic契約 T: Share, U: Share

費用契約・原文 O(log n) callbacks; maintains left-to-right combine order and pushes pending tags, so requires a mutable borrow.

std.lazySegment.update

公開宣言

```
pub fn update<T: Share, U: Share>(tree: &mut LazySegment<T,U>, start: Int, end: Int, value: U,
  combine: fn(T,T)->T!{}~{Send+Share}, apply: fn(T,U,Int)->T!{}~{Send+Share}, compose: fn(U,U)->U!{}~{Send+Share}) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,String>`

generic契約 `T: Share, U: Share`

費用契約・原文 $O(\log n)$ callback evaluations; persistent List page accesses add their own lookup cost.

失敗・境界契約・原文 Range is checked before mutation; callback panic or fatal budget does not roll back partial mutation. Use checkpoints when recovery is required.

std.map

getOr、contains、Stringキーentries/update。updateはSomeで設定、Noneで削除。すべての関数が任意のキー型を受けるわけではありません。

```
import  
  
import std.map as map;
```

共通型のfield・variant / このmoduleの実装を読む

std.map.contains

```
公開宣言  
  
pub fn contains<K: Ord, V: Share>(values: &Map<K,V>, key: K) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 Bool

generic契約 K: Ord, V: Share

std.map.entries

```
公開宣言  
  
pub fn entries<V: Share>(values: &Map<String,V>) -> List<Tuple<String,V>> effects {}
```

効果 なし (effects {})

canonical戻り型 List<Tuple<String,V>>

generic契約 V: Share

std.map.getOr

公開宣言

```
pub fn getOr<K: Ord, V: Share>(values: &Map<K,V>, key: K, fallback: V) -> V effects {}
```

効果 なし (effects {})

canonical戻り型 V

generic契約 K: Ord, V: Share

std.map.update

公開宣言

```
pub fn update<V: Share>(values: &mut Map<String,V>, key: String, change: fn(Option<V>)->Option<V>!  
{ }~{Send+Share}) -> Unit effects {}
```

効果 なし (effects {})

canonical戻り型 Unit

generic契約 V: Share

std.matching

二部graphの最大matchingと最小vertex cover。左右の頂点数、辺index、容量を検査します。

```
import
```

```
import std.matching as matching;
```

module固有の公開型

型 (公開field型)

```
struct Matching {
    size: Int
    left: List<Option<Int>>
    right: List<Option<Int>>
    leftCover: List<Bool>
    rightCover: List<Bool>
}
```

共通型のfield・variant / このmoduleの実装を読む

std.matching.maximum

公開宣言

```
pub fn maximum(leftVertices: Int, rightVertices: Int, pairs: &List<Tuple<Int,Int>>) ->
Result<Matching,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Matching,String>**

費用契約・原文 Dinic $O(V^2 E \log \text{pages})$ upper bound; $O(V + E)$ VM cells.

失敗・境界契約・原文 Capacity/Range; input is borrowed and never mutated. Fatal work/history budgets apply.

std.math

scalar数学関数の補助。Floatの定義域、NaN/Inf、error型を確認します。

```
import
```

```
import std.math as math;
```

共通型のfield・variant / このmoduleの実装を読む

std.math.clamp

公開宣言

```
pub fn clamp(value: Int, low: Int, high: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.math.max

公開宣言

```
pub fn max(a: Int, b: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.math.min

公開宣言

```
pub fn min(a: Int, b: Int) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.math.mulMod

公開宣言

```
pub fn mulMod(a: Int, b: Int, modulus: Int) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,StdError>`

std.matrix

Listなどで表すmatrix操作の補助。typed native配列の大きな計算にはnumericも選べます。

```
import
```

```
import std.matrix as matrix;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Matrix {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.matrix.columns

公開宣言

```
pub fn columns(matrix: &Matrix) -> Int effects {}
```

効果 なし（effects {}）

canonical戻り型 Int

std.matrix.create

公開宣言

```
pub fn create(rows: Int, columns: Int, input: &List<Int>) -> Result<Matrix,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 Result<Matrix,String>

std.matrix.get

公開宣言

```
pub fn get(matrix: &Matrix, row: Int, column: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Int,String>

std.matrix.multiply

公開宣言

```
pub fn multiply(a: &Matrix, b: &Matrix) -> Result<Matrix,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Matrix,String>

std.matrix.power

公開宣言

```
pub fn power(input: &Matrix, exponent: Int) -> Result<Matrix,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Matrix,String>

std.matrix.rows

公開宣言

```
pub fn rows(matrix: &Matrix) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

std.models

名前付きnative重みのmodel化、encode/decode、file保存。saveはFile/publish境界を持ち、名前・shapeの検査も必要です。

```
import
```

```
import std.models as models;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Model {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.models.create

公開宣言

```
pub fn create(weights: Map<String,FloatArray>) -> Result<Model,StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Model,StdError>**

費用契約・原文 O(parameters + elements) finite validation, shared native pages; <=128 names, each 1..128 UTF-8 bytes, encoded model <=32 MiB.

失敗・境界契約・原文 NumericSize/Domain/Type/NonFinite; sorted names, no NUL; fatal work/memory budgets apply. No per-cell VM objects.

std.models.decode

公開宣言

```
pub fn decode(bytes: Bytes) -> Result<Model, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Model, StdError>`

失敗・境界契約・原文 NumericSize/Domain/Shape/NonFinite; validates complete container, lengths, shapes, names, values and checksum before creating numeric pages. No code execution or external I/O.

std.models.encode

公開宣言

```
pub fn encode(model: &Model) -> Result<Bytes, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bytes, StdError>`

費用契約・原文 O(elements + bytes) native serialization; dimensions and exact finite IEEE bits, canonical names, versioned format with SHA-256 integrity (not an authenticity signature).

失敗・境界契約・原文 NumericSize/Domain/NonFinite; fatal memory/work budgets apply. Serialization materializes a bounded Bytes value, not a streaming container.

std.models.get

公開宣言

```
pub fn get(model: &Model, name: String) -> Option<FloatArray> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<FloatArray>`

std.models.load

公開宣言

```
pub fn load(path: String) -> Result<Model, StdError> effects {fileRead}
```

効果 fileRead

canonical戻り型 Result<Model, StdError>

失敗・境界契約・原文 File:<code> or codec errors; observed file bytes follow existing journal/replay contract, work/history/file budgets apply.

std.models.save

公開宣言

```
pub fn save(model: &Model, path: String) -> Result<Unit, StdError> effects {fileWrite}
```

効果 fileWrite

canonical戻り型 Result<Unit, StdError>

失敗・境界契約・原文 numeric codec errors; file writes and publish use existing fatal VM failures; deferred VM file write, physically visible only after publish, then not undone by revert.

std.models.weights

公開宣言

```
pub fn weights(model: &Model) -> Map<String, FloatArray> effects {}
```

効果 なし (effects {})

canonical戻り型 Map<String, FloatArray>

費用契約・原文 O(parameters) metadata; COW numeric storage versions are shared.

std.modular

剰余算術の補助。modulusの定義域、負数の正規化、逆元が存在する条件を確認します。

```
import
```

```
import std.modular as modular;
```

共通型のfield・variant / このmoduleの実装を読む

std.modular.add

公開宣言

```
pub fn add(a: Int, b: Int, modulus: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.modular.inverse

公開宣言

```
pub fn inverse(value: Int, modulus: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.modular.multiply

公開宣言

```
pub fn multiply(a: Int, b: Int, modulus: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.modular.normalize

公開宣言

```
pub fn normalize(value: Int, modulus: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.modular.power

公開宣言

```
pub fn power(base: Int, exponent: Int, modulus: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.number

数値のparse/format、整数関連の補助。表記と範囲の失敗はResultを処理します。

```
import
```

```
import std.number as number;
```

共通型のfield・variant / このmoduleの実装を読む

std.number.decimal

公開宣言

```
pub fn decimal(value: String) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,StdError>`

std.number.floating

公開宣言

```
pub fn floating(value: String) -> Result<Float,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float,StdError>`

std.number.formatFloat

公開宣言

```
pub fn formatFloat(value: Float) -> Result<String,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,StdError>`

std.number.formatInt

公開宣言

```
pub fn formatInt(value: Int, radix: Int) -> Result<String,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,StdError>`

std.number.integer

公開宣言

```
pub fn integer(value: String, radix: Int) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,StdError>`

std.numeric

shape/strideを持つIntArray/FloatArray、view、COW更新、linear algebra、統計。
values/shapeのList返却はv2.0.0の可変ownerです。

```
import

import std.numeric as numeric;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct OnlineMoments {

}
```

共通型のfield・variant / このmoduleの実装を読む

std.numeric.activation

公開宣言

```
pub fn activation(operation: String, array: &FloatArray) -> Result<FloatArray,StdError> effects {}
```

効果	なし（effects {}）
canonical戻り型	<code>Result<FloatArray,StdError></code>
費用契約・原文	O(elements) native work/storage; relu and stable sigmoid, no per-cell callbacks.
失敗・境界契約・原文	NumericDomain/Type/NonFinite; fatal work/memory budgets apply.

std.numeric.affine

公開宣言	
<pre>pub fn affine(array: &FloatArray, scale: Float, offset: Float) -> Result<FloatArray, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	$O(\text{elements})$ native work/storage; shape preserved, two finite scalars, no fused rounding guarantee.
失敗・境界契約・原文	NumericType/NonFinite/Overflow; fatal work/memory budgets apply.

std.numeric.broadcastFloat

公開宣言	
<pre>pub fn broadcastFloat(array: &FloatArray, shape: &List<Int>) -> Result<FloatArray, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	$O(\text{rank} + \log \text{pages})$; views share the storage version
失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.broadcastInt

公開宣言

```
pub fn broadcastInt(array: &IntArray, shape: &List<Int>) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 $O(\text{rank} + \log \text{pages})$; views share the storage version

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.correlation

公開宣言

```
pub fn correlation(left: &FloatArray, right: &FloatArray) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(n)$; constant inputs fail with `NumericDomain`

失敗・境界契約・原文 `StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence`; fatal work and memory budgets apply.

std.numeric.covariance

公開宣言

```
pub fn covariance(left: &FloatArray, right: &FloatArray, ddof: Int) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(n)$; paired online moments with explicit ddof

失敗・境界契約・原文 `StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence`; fatal work and memory budgets apply.

std.numeric.dot

公開宣言

```
pub fn dot(left: &FloatArray, right: &FloatArray) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(\text{elements times rank and log pages})$

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.eigenSymmetric

公開宣言

```
pub fn eigenSymmetric(matrix: &FloatArray, tolerance: Float, maxSweeps: Int) -> Result<EigenResult, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<EigenResult, StdError>`

費用契約・原文 $O(\text{maxSweeps} * n^3)$; bounded cyclic Jacobi, sorted eigenvalues

失敗・境界契約・原文 StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence; fatal work and memory budgets apply.

std.numeric.fromFloat

公開宣言

```
pub fn fromFloat(shape: &List<Int>, values: &List<Float>) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(\text{elements})$; typed pages, without per-element VM objects

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.fromInt

公開宣言

```
pub fn fromInt(shape: &List<Int>, values: &List<Int>) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 O(elements); typed pages, without per-element VM objects

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.getFloat

公開宣言

```
pub fn getFloat(array: &FloatArray, indices: &List<Int>) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 O(rank + log pages); views share the storage version

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.getInt

公開宣言

```
pub fn getInt(array: &IntArray, indices: &List<Int>) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 O(rank + log pages); views share the storage version

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.histogram

公開宣言

```
pub fn histogram(array: &FloatArray, edges: &FloatArray) -> Result<HistogramResult, StdError>
effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<HistogramResult, StdError>**

費用契約・原文 $O(n \log \text{edges} + \text{edges})$; outlier counts are explicit

失敗・境界契約・原文 StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence; fatal work and memory budgets apply.

std.numeric.leastSquares

公開宣言

```
pub fn leastSquares(matrix: &FloatArray, right: &FloatArray, tolerance: Float) ->
Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<FloatArray, StdError>**

費用契約・原文 $O(m*n^2)$; requires $m \geq n$ and full column rank

失敗・境界契約・原文 StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence; fatal work and memory budgets apply.

std.numeric.mapFloat

公開宣言

```
pub fn mapFloat(operation: String, array: &FloatArray) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(\text{elements times rank and log pages})$

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.materializeFloat

公開宣言

```
pub fn materializeFloat(array: &FloatArray) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(\text{elements times rank and log pages})$

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.materializeInt

公開宣言

```
pub fn materializeInt(array: &IntArray) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 $O(\text{elements times rank and log pages})$

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.math

公開宣言

```
pub fn math(operation: String, value: Float) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(1)$; bounded native scalar math

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.matmul

公開宣言

```
pub fn matmul(left: &FloatArray, right: &FloatArray) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(m*n*k*\log \text{ pages})$; native kernel, without VM callbacks

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.mean

公開宣言

```
pub fn mean(array: &FloatArray) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(\text{elements times rank and log pages})$

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.mergeMoments

公開宣言

```
pub fn mergeMoments(target: &mut OnlineMoments, source: &OnlineMoments) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit, StdError>`

費用契約・原文 $O(1)$; combines independent owners without retaining their samples.

失敗・境界契約・原文 `StdError NumericSize/NonFinite`; fatal VM execution and allocation budgets apply.

std.numeric.norm2

公開宣言

```
pub fn norm2(array: &FloatArray) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(\text{elements})$ native work / $O(\text{rank})$ iterator state; scaled Euclidean accumulation avoids intermediate square overflow and underflow.

失敗・境界契約・原文 `NumericType/NonFinite/Overflow`; `Float64` array, empty norm is zero, nonrepresentable final norm fails. Fatal budgets apply.

std.numeric.onlineCount

公開宣言

```
pub fn onlineCount(moments: &OnlineMoments) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

費用契約・原文 $O(1)$.

失敗・境界契約・原文 No recoverable errors; fatal VM execution budget applies.

std.numeric.onlineMean

公開宣言

```
pub fn onlineMean(moments: &OnlineMoments) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(1)$.

失敗・境界契約・原文 `StdError NumericEmpty`; fatal VM execution budget applies.

std.numeric.onlineMoments

公開宣言

```
pub fn onlineMoments() -> OnlineMoments effects {}
```

効果 なし (effects {})

canonical戻り型 `OnlineMoments`

費用契約・原文 $O(1)$; checkpointed mutable owner with no retained sample list.

失敗・境界契約・原文 Fatal VM allocation budget only.

std.numeric.onlineVariance

公開宣言

```
pub fn onlineVariance(moments: &OnlineMoments, ddof: Int) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(1)$; divisor is count - ddof.

失敗・境界契約・原文 `StdError NumericDomain/Empty`; fatal VM execution budget applies.

std.numeric.pushMoment

公開宣言

```
pub fn pushMoment(moments: &mut OnlineMoments, value: Float) -> Result<Unit, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Unit, StdError>**

費用契約・原文 $O(1)$; count bounded to 2^{53} , errors leave previous statistics unchanged.

失敗・境界契約・原文 StdError NumericSize/NonFinite; fatal VM execution and allocation budgets apply.

std.numeric.qr

公開宣言

```
pub fn qr(array: &FloatArray, tolerance: Float) -> Result<QrResult, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<QrResult, StdError>**

費用契約・原文 $O(m*n*\min(m,n))$; column-pivoted economy Householder QR

失敗・境界契約・原文 StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence; fatal work and memory budgets apply.

std.numeric.quantile

公開宣言

```
pub fn quantile(array: &FloatArray, probability: Float) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Float, StdError>**

費用契約・原文 $O(n \log n)$; explicit sorted copy and linear interpolation

失敗・境界契約・原文 StdError NumericShape/Domain/NonFinite/Overflow/Empty/Singular/Convergence; fatal work and memory budgets apply.

std.numeric.reshapeFloat

公開宣言

```
pub fn reshapeFloat(array: &FloatArray, shape: &List<Int>) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(\text{rank} + \log \text{pages})$; views share the storage version

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.reshapeInt

公開宣言

```
pub fn reshapeInt(array: &IntArray, shape: &List<Int>) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 $O(\text{rank} + \log \text{pages})$; views share the storage version

失敗・境界契約・原文 `StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly`; fatal execution and memory budgets apply.

std.numeric.scale

公開宣言

```
pub fn scale(array: &FloatArray, factor: Float) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<FloatArray, StdError>**

費用契約・原文 O(elements) native work / output storage; preserves array shape, no per-cell VM objects.

失敗・境界契約・原文 NumericType/NonFinite/Overflow; finite scalar and Float64 array, fatal work and memory budgets apply.

std.numeric.shapeFloat

公開宣言

```
pub fn shapeFloat(array: &FloatArray) -> Result<List<Int>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Int>, StdError>**

費用契約・原文 O(rank + log pages); views share the storage version

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.shapeInt

公開宣言

```
pub fn shapeInt(array: &IntArray) -> Result<List<Int>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Int>, StdError>**

費用契約・原文 O(rank + log pages); views share the storage version

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.sliceFloat

公開宣言

```
pub fn sliceFloat(array: &FloatArray, axis: Int, start: Int, length: Int, step: Int) ->
Result<FloatArray, StdError> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	Result<FloatArray, StdError>
--------------	------------------------------

費用契約・原文	O(rank + log pages); views share the storage version
---------	--

失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.
------------	--

std.numeric.sliceInt

公開宣言

```
pub fn sliceInt(array: &IntArray, axis: Int, start: Int, length: Int, step: Int) ->
Result<IntArray, StdError> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	Result<IntArray, StdError>
--------------	----------------------------

費用契約・原文	O(rank + log pages); views share the storage version
---------	--

失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.
------------	--

std.numeric.solve

公開宣言

```
pub fn solve(matrix: &FloatArray, right: &FloatArray, tolerance: Float) ->
Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<FloatArray, StdError>**

費用契約・原文 $O(n^3)$; pivoted LU with $O(n^2)$ temporary storage

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.stridesFloat

公開宣言

```
pub fn stridesFloat(array: &FloatArray) -> Result<List<Int>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Int>, StdError>**

費用契約・原文 $O(\text{rank} + \log \text{pages})$; views share the storage version

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.stridesInt

公開宣言	
<pre>pub fn stridesInt(array: &IntArray) -> Result<List<Int>, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<List<Int>, StdError></code>
費用契約・原文	$O(\text{rank} + \log \text{pages})$; views share the storage version
失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.sum

公開宣言	
<pre>pub fn sum(array: &FloatArray) -> Result<Float, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<Float, StdError></code>
費用契約・原文	$O(\text{elements times rank and log pages})$
失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.sumToShape

公開宣言	
<pre>pub fn sumToShape(array: &FloatArray, shape: &List<Int>) -> Result<FloatArray, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	$O(\text{elements} * \text{rank} + \text{output elements})$, $O(\text{output elements})$ native scratch; Neumaier accumulation, scalar/singleton dimensions retained as requested.
失敗・境界契約・原文	NumericShape/Size/Type/NonFinite/Overflow; target must broadcast to input shape; empty reductions are zero, fatal budgets apply.

std.numeric.transposeFloat

公開宣言	
<pre>pub fn transposeFloat(array: &FloatArray, axes: &List<Int>) -> Result<FloatArray, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	$O(\text{rank} + \log \text{pages})$; views share the storage version
失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.transposeInt

公開宣言

```
pub fn transposeInt(array: &IntArray, axes: &List<Int>) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<IntArray, StdError>**

費用契約・原文 O(rank + log pages); views share the storage version

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.valuesFloat

公開宣言

```
pub fn valuesFloat(array: &FloatArray) -> Result<List<Float>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Float>, StdError>**

費用契約・原文 O(elements times 64); explicit VM List allocation

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.valuesInt

公開宣言

```
pub fn valuesInt(array: &IntArray) -> Result<List<Int>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Int>, StdError>**

費用契約・原文 O(elements times 64); explicit VM List allocation

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.variance

公開宣言

```
pub fn variance(array: &FloatArray, ddof: Int) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Float, StdError>**

費用契約・原文 O(elements times rank and log pages)

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.withFloat

公開宣言

```
pub fn withFloat(array: &FloatArray, indices: &List<Int>, value: Float) ->  
Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<FloatArray, StdError>**

費用契約・原文 O(256 + log pages); copies one numeric page and its tree path

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.withInt

公開宣言

```
pub fn withInt(array: &IntArray, indices: &List<Int>, value: Int) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 $O(256 + \log \text{pages})$; copies one numeric page and its tree path

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.zerosFloat

公開宣言

```
pub fn zerosFloat(shape: &List<Int>) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(\text{elements})$; typed pages, without per-element VM objects

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.zerosInt

公開宣言

```
pub fn zerosInt(shape: &List<Int>) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 O(elements); typed pages, without per-element VM objects

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.zipFloat

公開宣言

```
pub fn zipFloat(operation: String, left: &FloatArray, right: &FloatArray) ->  
Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 O(elements times rank and log pages)

失敗・境界契約・原文 StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.

std.numeric.zipInt

公開宣言

```
pub fn zipInt(operation: String, left: &IntArray, right: &IntArray) -> Result<IntArray, StdError>
effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	<code>Result<IntArray, StdError></code>
--------------	---

費用契約・原文	$O(\text{elements times rank and log pages})$
---------	---

失敗・境界契約・原文	StdError NumericShape/Index/Size/Type/Overflow/NonFinite/Domain/Singular/Empty/ReadOnly; fatal execution and memory budgets apply.
------------	--

std.numericAsync

重いnative数値計算の協調API。同期契約に加えてTaskError、途中状態、work/memory/取消を扱います。

```
import

import std.numericAsync as numericAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericAsync.dot

```
公開宣言

pub async fn dot(left: FloatArray, right: FloatArray) -> Result<Float,StdError> effects {tasks}
```

効果	tasks
canonical戻り型	Result<Float,StdError>
費用契約・原文	Native chunks of at most 4096 products, preserving Neumaier ordering; yields between chunks. Inputs share native storage.
失敗・境界契約・原文	NumericShape/Type/NonFinite/Overflow; task cancellation and fatal budgets apply. No host worker is started.

std.numericAsync.eigenSymmetric

公開宣言

```
pub async fn eigenSymmetric(matrix: FloatArray, tolerance: Float, maxSweeps: Int) ->
Result<EigenResult, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<EigenResult, StdError>`

費用契約・原文 Cyclic Jacobi bounded to maxSweeps; copy, symmetry check, scans, rotations, stable sorting and outputs execute at most 4096 scalar units per step with shared-page COW work, cached storage digests and bounded page-range rotation buffers.

失敗・境界契約・原文 NumericShape/Type/Domain/NonFinite/Overflow/Convergence/Size; finite symmetric square Float64 matrix, finite nonnegative tolerance and maxSweeps 0..10000. Same rotation/arithmetic order and stable eigenvalue order as numeric.eigenSymmetric, no partial result on error/cancel; fatal memory/work budgets apply.

std.numericAsync.leastSquares

公開宣言

```
pub async fn leastSquares(matrix: FloatArray, right: FloatArray, tolerance: Float) ->
Result<FloatArray, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 Full-column-rank QR $O(m \cdot n^2)$; bounded rhs validation, QR, compensated projection, upper substitution and permutation, with at most 4096 scalar units per step. Inputs and private numeric pages are shared; no full rhs/Q/R materialization or second factorization.

失敗・境界契約・原文 NumericShape/Type/Domain/NonFinite/Overflow/Singular/Size; $m \geq n$, rank-one rhs and finite nonnegative relative rank tolerance. Same arithmetic order as numeric.leastSquares. No partial output on error/cancel; fatal work/memory budgets apply.

std.numericAsync.matmul

公開宣言	
<pre>pub async fn matmul(left: FloatArray, right: FloatArray) -> Result<FloatArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	Native chunks of at most 4096 products and page-range COW writes; yields between chunks. Output initialization shares logarithmic zero storage.
失敗・境界契約・原文	NumericShape/Type/Size/NonFinite/Overflow; task cancellation and fatal budgets apply. No partial output escapes a failed task.

std.numericAsync.norm2

公開宣言	
<pre>pub async fn norm2(array: FloatArray) -> Result<Float, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<Float, StdError></code>
費用契約・原文	At most 4096 values per native step; scaled accumulation preserves logical order without materializing input or squaring large magnitudes.
失敗・境界契約・原文	NumericType/NonFinite/Overflow; arbitrary ranks and views; same scaled result as numeric.norm2, task cancellation and fatal budgets apply.

std.numericAsync.qr

公開宣言	
<pre>pub async fn qr(matrix: FloatArray, tolerance: Float) -> Result<QrArrayResult, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<QrArrayResult, StdError></code>
費用契約・原文	Column-pivoted Householder $O(m \cdot n \cdot \min(m, n))$; bounded native steps of at most 4096 scalar units, private shared-page COW work and task handoff between steps.
失敗・境界契約・原文	NumericShape/Type/Domain/NonFinite/Overflow/Size; economy Q and R, permutation as native IntArray; same arithmetic order as numeric.qr. No partial result on failure or cancellation, fatal work/memory budgets apply.

std.numericAsync.scale

公開宣言	
<pre>pub async fn scale(array: FloatArray, factor: Float) -> Result<FloatArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	Shared logarithmic zero initialization; at most 4096 values per native step, bounded scratch and page-range COW writes.
失敗・境界契約・原文	NumericType/NonFinite/Overflow; arbitrary ranks and strided input; finite factor, no partial result on failure or cancellation.

std.numericAsync.solve

公開宣言	
<pre>pub async fn solve(matrix: FloatArray, right: FloatArray, tolerance: Float) -> Result<FloatArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	Partial-pivot LU $O(n^3)$ arithmetic with $O(n^2)$ shared-page COW private storage; at most 4096 copies, pivot checks, swap pairs or update products per native step, including input initialization and back substitution.
失敗・境界契約・原文	NumericShape/Type/NonFinite/Singular/Size; square Float64 matrix, rank-one rhs, finite nonnegative relative pivot tolerance. Same arithmetic order as numeric.solve, no partial output on error/cancel. Fatal work/memory budgets apply; conservative per-step COW admission remains.

std.numericAsync.zipFloat

公開宣言	
<pre>pub async fn zipFloat(operation: String, left: FloatArray, right: FloatArray) -> Result<FloatArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	<code>Result<FloatArray, StdError></code>
費用契約・原文	At most 4096 element pairs per native step; shared inputs and logarithmic zero initialization, bounded scratch and page-range COW output.
失敗・境界契約・原文	NumericShape/Type/Domain/NonFinite; add/sub/mul/div and matching shapes; division by zero fails. No partial result escapes.

std.numericIdentityAsync

identity配列生成等の協調処理。shapeと有限予算を検査し、完了後に結果を取り出します。

```
import  
  
import std.numericIdentityAsync as numericIdentityAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericIdentityAsync.checkFinite

公開宣言

```
pub async fn checkFinite(array: FloatArray) -> Result<Unit, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Unit, StdError>`

費用契約・原文 at most 4096 logical cells per native call, no prefix scan or copied pages; yields between chunks.

失敗・境界契約・原文 NumericType/NonFinite; cancellation exposes no partial validation result, fatal work/memory limits apply.

std.numericIdentityAsync.key

公開宣言

```
pub async fn key(array: FloatArray, tag: String, index: Int, left: Bytes, right: Bytes) ->  
Result<Bytes, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Bytes, StdError>`

費用契約・原文 O(backing storage cells + rank + tag bytes), bounded prepare chunks followed by O(rank + tag bytes) key finalization; identity matches synchronous std.NumericTensorKey, including entire backing storage of a view.

失敗・境界契約・原文 NumericDomain for tag >256 bytes, negative index or parent keys other than 0/32 bytes; child diagnostic/budget codes are preserved, cancellation and fatal limits apply.

std.numericIdentityAsync.prepare

公開宣言	
<pre>pub async fn prepare(array: FloatArray) -> Result<TensorIdentityWork, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	Result<TensorIdentityWork, StdError>
費用契約・原文	bounded postorder chunks of at most 4096 scalar/node units with O(log(storage pages)) traversal state and shared input pages; cache-independent handoff points.
失敗・境界契約・原文	NumericType; owned opaque work survives checkpoints, cancellation releases the unretained work. No external observations or host worker.

std.numericIndex

viewのlogical行優先のflat indexでアクセス・COW更新します。transposeや逆順sliceもlogical順です。

```
import

import std.numericIndex as numericIndex;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericIndex.getFloat

公開宣言

```
pub fn getFloat(array: &FloatArray, index: Int) -> Result<Float, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 $O(\text{rank} + \log \text{pages})$; no coordinate List.

失敗・境界契約・原文 `StdError NumericIndex`; fatal VM work and memory budgets apply.

std.numericIndex.getInt

公開宣言

```
pub fn getInt(array: &IntArray, index: Int) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 $O(\text{rank} + \log \text{pages})$; no coordinate List.

失敗・境界契約・原文 `StdError NumericIndex`; fatal VM work and memory budgets apply.

std.numericIndex.lengthFloat

公開宣言

```
pub fn lengthFloat(array: &FloatArray) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 $O(\text{rank})$; no coordinate List or payload copy.

失敗・境界契約・原文 Fatal VM work and memory budgets apply.

std.numericIndex.lengthInt

公開宣言

```
pub fn lengthInt(array: &IntArray) -> Result<Int, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int, StdError>`

費用契約・原文 $O(\text{rank})$; no coordinate List or payload copy.

失敗・境界契約・原文 Fatal VM work and memory budgets apply.

std.numericIndex.withFloat

公開宣言

```
pub fn withFloat(array: &FloatArray, index: Int, value: Float) -> Result<FloatArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 $O(\text{rank} + 256 + \log \text{pages})$; COW copies one 256-element page and tree path.

失敗・境界契約・原文 StdError NumericIndex/ReadOnly; fatal VM work and memory budgets apply.

std.numericIndex.withInt

公開宣言	
<pre>pub fn withInt(array: &IntArray, index: Int, value: Int) -> Result<IntArray, StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<IntArray, StdError>
費用契約・原文	$O(\text{rank} + 256 + \log \text{pages})$; COW copies one 256-element page and tree path.
失敗・境界契約・原文	StdError NumericIndex/ReadOnly; fatal VM work and memory budgets apply.

std.numericInputAsync

Listを所有するtaskとしてnative配列を作ります。shape/valuesをmoveし、コピーを4096セル以下へ分割します。

```
import
```

```
import std.numericInputAsync as numericInputAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericInputAsync.fromFloat

公開宣言

```
pub async fn fromFloat(shape: List<Int>, values: List<Float>) -> Result<FloatArray, StdError>
effects {tasks}
```

効果 tasks

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 O(elements times log pages), at most 4096 raw cells per step; input List pages shared, bounded output COW and handoff between chunks.

失敗・境界契約・原文 NumericShape/Type/Index/ReadOnly/Size; preserves raw float bits including NaN and signed zero like synchronous numeric.fromFloat; no partial result on cancellation, fatal budgets apply. Existing synchronous API is unchanged.

std.numericInputAsync.fromInt

公開宣言	
<pre>pub async fn fromInt(shape: List<Int>, values: List<Int>) -> Result<IntArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	Result<IntArray, StdError>
費用契約・原文	O(elements times log pages), at most 4096 raw cells per step; shared input pages and bounded COW output, yields between steps.
失敗・境界契約・原文	NumericShape/Type/Index/ReadOnly/Size; exact Int bits and synchronous result, no partial result on cancellation, retained checkpoints keep independent work versions, fatal budgets apply.

std.numericRange

checked Int64の等差native配列。start/step/lengthのoverflowと容量を検査します。

```
import
```

```
import std.numericRange as numericRange;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericRange.integers

公開宣言

```
pub fn integers(start: Int, step: Int, length: Int) -> Result<IntArray, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<IntArray, StdError>`

費用契約・原文 O(length) native page construction without per-element VM objects.

失敗・境界契約・原文 NumericSize/Index/Overflow; length 0..16777216, checked Int64 endpoints; VM work/memory budgets apply.

std.numericReduceAsync

sum/mean/varianceの協調処理。4096論理要素以下でhandoffし、既存計算順序と失敗契約を保持します。

```
import

import std.numericReduceAsync as numericReduceAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericReduceAsync.mean

公開宣言

```
pub async fn mean(array: FloatArray) -> Result<Float, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 At most 4096 logical values per native step, Welford order as numeric.mean; shared strided/broadcast inputs, constant scalar state, task handoff.

失敗・境界契約・原文 NumericType/Empty/NonFinite/Overflow; same finite mean-and-variance checks as numeric.mean; no partial result on failure/cancel, fatal budgets apply.

std.numericReduceAsync.sum

公開宣言

```
pub async fn sum(array: FloatArray) -> Result<Float, StdError> effects {tasks}
```

効果 tasks

canonical戻り型 `Result<Float, StdError>`

費用契約・原文 At most 4096 logical values per native step, shared input pages and constant scalar state; yields between steps, same Neumaier order as numeric.sum.

失敗・境界契約・原文 NumericType/NonFinite/Overflow; empty input sums to 0, arbitrary rank/views, no partial result on typed failure or cancellation. Fatal memory/work budgets apply.

std.numericReduceAsync.variance

公開宣言	
pub async fn variance(array: FloatArray, ddof: Int) -> Result<Float,StdError> effects {tasks}	
効果	tasks
canonical戻り型	Result<Float,StdError>
費用契約・原文	At most 4096 logical values per native step, same Welford order and count-ddof divisor as numeric.variance; shared input and constant scalar state.
失敗・境界契約・原文	NumericIndex for negative ddof, NumericEmpty for count <= ddof, NumericType/NonFinite/Overflow; no partial result on failure/cancel, fatal budgets apply.

std.numericShapeAsync

logical reshapeとbroadcast軸の合計を協調処理。shape Listはtaskへ所有権を渡します。

```
import
```

```
import std.numericShapeAsync as numericShapeAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericShapeAsync.reshapeLogical

公開宣言

```
pub async fn reshapeLogical(array: FloatArray, shape: List<Int>) -> Result<FloatArray, StdError>
effects {tasks}
```

効果 tasks

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 O(rank) for an already-contiguous input; otherwise at most 4096 raw logical cells per step, shared inputs and bounded page writes, yields between steps.

失敗・境界契約・原文 NumericShape/Type/Index/ReadOnly/Domain; preserves raw NaN payloads and signed zero without arithmetic finite checks, no partial output on failure/cancel. Fatal budgets apply.

std.numericShapeAsync.sumToShape

公開宣言	
<pre>pub async fn sumToShape(array: FloatArray, shape: List<Int>) -> Result<FloatArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	Result<FloatArray, StdError>
費用契約・原文	At most 4096 input/final-output cells per step, Neumaier order per output cell, two shared native page accumulators, fixed-size page-footprint admission and task handoff.
失敗・境界契約・原文	NumericShape/Type/Index/ReadOnly/Domain/NonFinite/Overflow; same broadcast contraction and empty/scalar result as synchronous SumToShape, no partial result on failure/cancel. Fatal budgets apply.

std.numericTransformAsync

単項数値変換とactivationの協調処理。4096論理要素以下のstep、部分結果と取消を扱います。

```
import
```

```
import std.numericTransformAsync as numericTransformAsync;
```

共通型のfield・variant / このmoduleの実装を読む

std.numericTransformAsync.activation

公開宣言

```
pub async fn activation(operation: String, array: FloatArray) -> Result<FloatArray, StdError>
effects {tasks}
```

効果 tasks

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 At most 4096 values per step, bounded scratch and task handoff; strided/broadcast input without full materialization.

失敗・境界契約・原文 relu/reluGrad/sigmoid/sigmoidGrad/tanhGrad/reciprocal with the existing tensor activation contract. Grad operations consume forward outputs. No partial result on failure/cancel.

std.numericTransformAsync.mapFloat

公開宣言

```
pub async fn mapFloat(operation: String, array: FloatArray) -> Result<FloatArray, StdError> effects
{tasks}
```

効果 tasks

canonical戻り型 `Result<FloatArray, StdError>`

費用契約・原文 At most 4096 logical values per native step; shared inputs, logarithmic zero initialization, page-range COW output, handoff between steps.

失敗・境界契約・原文 Same operations/domain/finite/rounding contract as numeric.mapFloat; no partial result on typed failure or cancellation. Fatal memory/work budgets apply.

std.optimize

SGD/Adam等のparameter更新とoptimizer状態。重みだけでなくmoment/stepもcheckpointで保持したい状態に含めます。

```
import
```

```
import std.optimize as optimize;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Adam {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.optimize.adam

公開宣言

```
pub fn adam(weights: &Map<String,FloatArray>, beta1: Float, beta2: Float, epsilon: Float) ->  
Result<Adam,StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Adam,StdError>**

費用契約・原文 $O(\text{elements})$ moment initialization; β_1/β_2 in $[0,1)$, finite $\epsilon > 0$; moments use native pages and support checkpoint/revert.

失敗・境界契約・原文 OptimizerConfiguration or numeric validation failures; ≤ 128 named finite Float64 parameters / ≤ 32 MiB model.

std.optimize.adamStep

公開宣言

```
pub fn adamStep(state: &mut Adam, weights: &Map<String,FloatArray>, gradients:
&Map<String,FloatArray>, rate: Float) -> Result<Map<String,FloatArray>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Map<String,FloatArray>,StdError>`

費用契約・原文 $O(\text{parameters} + \text{elements})$, several bounded native buffers; synchronous, maximum $2^{53}-1$ steps. Keeps two moment arrays per parameter.

失敗・境界契約・原文 OptimizerRate/Parameters/Iterations or NumericShape/Size/Domain/NonFinite/Overflow; typed failures leave state and input weights unchanged. Fatal VM failures require checkpoint recovery. Bias corrected Adam, epsilon outside sqrt.

std.optimize.sgd

公開宣言

```
pub fn sgd(weights: &Map<String,FloatArray>, gradients: &Map<String,FloatArray>, rate: Float) ->
Result<Map<String,FloatArray>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Map<String,FloatArray>,StdError>`

費用契約・原文 $O(\text{parameters} + \text{elements})$ work/storage; native buffers, exact name/shape match, positive finite rate.

失敗・境界契約・原文 OptimizerRate/Parameters, NumericShape/Size/Domain/NonFinite/Overflow; fatal budgets apply. Inputs unchanged on error and success.

std.optimize.steps

公開宣言

```
pub fn steps(state: &Adam) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.option

Optionの操作補助。Noneと0/空Stringなどの正常値を区別して使用します。

```
import
```

```
import std.option as option;
```

共通型のfield・variant / このmoduleの実装を読む

std.option.map

公開宣言

```
pub fn map<T: Share, U: Share>(value: Option<T>, transform: fn(T)->U!{}~{Send+Share}) -> Option<U>
effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<U>`

generic契約 `T: Share, U: Share`

std.option.valueOr

公開宣言

```
pub fn valueOr<T: Share>(value: Option<T>, fallback: T) -> T effects {}
```

効果 なし (effects {})

canonical戻り型 `T`

generic契約 `T: Share`

std.path

パス表現の操作補助。Fileのroot外アクセス許可を与えるものではありません。

```
import
```

```
import std.path as path;
```

共通型のfield・variant / このmoduleの実装を読む

std.path.join

公開宣言

```
pub fn join(base: String, child: String) -> Result<String,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,String>`

std.path.relative

公開宣言

```
pub fn relative(value: String) -> Result<String,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,String>`

std.range

区間の操作補助。端点の包含、半開区間、空区間を関数ごとに確認します。

```
import
```

```
import std.range as range;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct SparseMin {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.range.length

公開宣言

```
pub fn length(table: &SparseMin) -> Int effects {}
```

効果 なし（effects {}）

canonical戻り型 `Int`

std.range.minimum

公開宣言

```
pub fn minimum(table: &SparseMin, start: Int, end: Int) -> Result<Int,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 `Result<Int,String>`

std.range.sparseMin

公開宣言

```
pub fn sparseMin(input: &List<Int>) -> Result<SparseMin,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<SparseMin,String>`

std.regex

boundedな正規表現compile/search。spanはUTF-8 byte offset、capture不参加はNone。他言語の全regex拡張との互換を仮定しません。

```
import
```

```
import std.regex as regex;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Match {  
  
}
```

型（公開field型）

```
record Options {  
    caseInsensitive: Bool  
    multiLine: Bool  
    dotAll: Bool  
}
```

型（公開field型）

```
record Span {  
    start: Int  
    end: Int  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.regex.bytes

公開宣言

```
pub fn bytes(input: Bytes, span: Span) -> Result<Bytes, StdError> effects {}
```

効果

なし (effects {})

canonical戻り型

`Result<Bytes, StdError>`

std.regex.compile

公開宣言

```
pub fn compile(pattern: String) -> Result<Regex, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Regex, StdError>`

std.regex.compileBytes

公開宣言

```
pub fn compileBytes(pattern: String) -> Result<Regex, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Regex, StdError>`

std.regex.compileBytesOptions

公開宣言

```
pub fn compileBytesOptions(pattern: String, options: Options) -> Result<Regex, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Regex, StdError>`

std.regex.compileOptions

公開宣言

```
pub fn compileOptions(pattern: String, options: Options) -> Result<Regex, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Regex, StdError>`

std.regex.engineVersion

公開宣言

```
pub fn engineVersion() -> String effects {}
```

効果 なし (effects {})

canonical戻り型 `String`

std.regex.find

公開宣言

```
pub fn find(pattern: &Regex, input: &String) -> Result<Option<Match>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Match>, StdError>`

std.regex.findBytes

公開宣言

```
pub fn findBytes(pattern: &Regex, input: Bytes) -> Result<Option<Match>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Match>, StdError>`

std.regex.findBytesFrom

公開宣言

```
pub fn findBytesFrom(pattern: &Regex, input: Bytes, start: Int) -> Result<Option<Match>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Match>, StdError>`

std.regex.findFrom

公開宣言

```
pub fn findFrom(pattern: &Regex, input: &String, start: Int) -> Result<Option<Match>, StdError>
effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Match>, StdError>`

失敗・境界契約・原文 RegexIndex on non-boundary text offset; preserves anchors and word-boundary context before start.

std.regex.fromJson

公開宣言

```
pub fn fromJson(value: Json) -> Result<Regex, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Regex, StdError>`

std.regex.group

公開宣言

```
pub fn group(value: &Match, index: Int) -> Result<Option<Span>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Span>, StdError>`

std.regex.groupCount

公開宣言

```
pub fn groupCount(value: &Match) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 **Int**

std.regex.groupNames

公開宣言

```
pub fn groupNames(pattern: &Regex) -> Result<List<Option<String>>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Option<String>>, StdError>**

std.regex.isMatch

公開宣言

```
pub fn isMatch(pattern: &Regex, input: &String) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Bool, StdError>**

std.regex.isMatchBytes

公開宣言

```
pub fn isMatchBytes(pattern: &Regex, input: Bytes) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Bool, StdError>**

std.regex.isText

公開宣言

```
pub fn isText(pattern: &Regex) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool, StdError>`

std.regex.next

公開宣言

```
pub fn next(value: &Match) -> Option<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Int>`

std.regex.options

公開宣言

```
pub fn options(pattern: &Regex) -> Result<Options, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Options, StdError>`

std.regex.source

公開宣言

```
pub fn source(pattern: &Regex) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.regex.span

公開宣言

```
pub fn span(value: &Match) -> Span effects {}
```

効果 なし (effects {})

canonical戻り型 `Span`

std.regex.text

公開宣言

```
pub fn text(input: &String, span: Span) -> Result<String,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,StdError>`

std.regex.toJson

公開宣言

```
pub fn toJson(pattern: &Regex) -> Result<Json,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Json,StdError>`

std.result

Resultの成功・失敗の変換と合成。error型とcallbackの効果/所有権を揃えます。

```
import
```

```
import std.result as result;
```

共通型のfield・variant / このmoduleの実装を読む

std.result.map

公開宣言

```
pub fn map<T: Share, U: Share, E: Share>(value: Result<T,E>, transform: fn(T)->U!{}~{Send+Share}) -> Result<U,E> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<U,E>**

generic契約 **T: Share, U: Share, E: Share**

std.result.mapError

公開宣言

```
pub fn mapError<T: Share, E: Share, F: Share>(value: Result<T,E>, transform: fn(E)->F!{}~{Send+Share}) -> Result<T,F> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<T,F>**

generic契約 **T: Share, E: Share, F: Share**

std.result.value0r

公開宣言	
pub fn value0r<T: Share, E: Share>(value: Result<T,E>, fallback: T) -> T effects {}	
効果	なし (effects {})
canonical戻り型	T
generic契約	T: Share, E: Share

std.rollbackSet

データ構造自身のundo履歴を持つ集合併合。VMのcommit/revertとは別のAPIであることを区別します。

```
import

import std.rollbackSet as rollbackSet;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct RollbackSet {

}
```

共通型のfield・variant / このmoduleの実装を読む

std.rollbackSet.create

公開宣言

```
pub fn create(count: Int) -> Result<RollbackSet,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<RollbackSet,String>**

std.rollbackSet.mark

公開宣言

```
pub fn mark(set: &RollbackSet) -> Int effects {}
```

効果 なし（effects {}）

canonical戻り型 **Int**

std.rollbackSet.rollback

公開宣言

```
pub fn rollback(set: &mut RollbackSet, mark: Int) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,String>`

std.rollbackSet.root

公開宣言

```
pub fn root(set: &RollbackSet, index: Int) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Int,String>`

std.rollbackSet.same

公開宣言

```
pub fn same(set: &RollbackSet, a: Int, b: Int) -> Result<Bool,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool,String>`

std.rollbackSet.unite

公開宣言

```
pub fn unite(set: &mut RollbackSet, a: Int, b: Int) -> Result<Bool,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bool,String>`

std.scanner

1MiB以内のBytesからASCII空白区切りtoken/Intを読む可変parser。EOFはNone、parse失敗でcursorが自動復元する保証はありません。

```
import
```

```
import std.scanner as scanner;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Scanner {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.scanner.fromBytes

公開宣言

```
pub fn fromBytes(input: Bytes) -> Result<Scanner,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 `Result<Scanner,String>`

std.scanner.nextInt

公開宣言

```
pub fn nextInt(scanner: &mut Scanner) -> Result<Option<Int>,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 `Result<Option<Int>,String>`

std.scanner.nextToken

公開宣言

```
pub fn nextToken(scanner: &mut Scanner) -> Result<Option<Bytes>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Bytes>,String>`

std.scanner.position

公開宣言

```
pub fn position(scanner: &Scanner) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.search

順序付けされたcollection等の検索。sorted前提と比較規則、境界indexを確認します。

```
import
```

```
import std.search as search;
```

共通型のfield・variant / このmoduleの実装を読む

std.search.binary

公開宣言

```
pub fn binary<T: Share>(values: &List<T>, key: T, compare: fn(T,T)->Int!{}~{Send+Share}) ->
Option<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Int>`

generic契約 `T: Share`

std.search.lowerBound

公開宣言

```
pub fn lowerBound<T: Share>(values: &List<T>, key: T, compare: fn(T,T)->Int!{}~{Send+Share}) -> Int
effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

generic契約 `T: Share`

std.search.upperBound

公開宣言

```
pub fn upperBound<T: Share>(values: &List<T>, key: T, compare: fn(T,T)->Int!{}~{Send+Share}) -> Int
effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	Int
--------------	-----

generic契約	T: Share
-----------	----------

std.segment

区間集約用tree。callback、初期値、区間、更新と返却値の所有権を確認します。tryUpdatedは失敗時の入力保持を意図したAPIです。

```
import
```

```
import std.segment as segment;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct Segment<T> {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.segment.build

公開宣言

```
pub fn build<T: Share>(input: &List<T>, identity: T, combine: fn(T,T)->T!{}~{Send+Share}) ->  
Result<Segment<T>,String> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Segment<T>,String>**

generic契約 **T: Share**

std.segment.length

公開宣言

```
pub fn length<T: Share>(tree: &Segment<T>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

generic契約 T: Share

std.segment.query

公開宣言

```
pub fn query<T: Share>(tree: &Segment<T>, start: Int, end: Int, combine: fn(T,T)->T!{}~  
{Send+Share}) -> Result<T,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<T,String>

generic契約 T: Share

std.segment.set

公開宣言

```
pub fn set<T: Share>(tree: &mut Segment<T>, index: Int, value: T, combine: fn(T,T)->T!{}~  
{Send+Share}) -> Result<Unit,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Unit,String>

generic契約 T: Share

std.segment.tryUpdated

公開宣言

```
pub fn tryUpdated<T: Share, E: Share>(tree: &Segment<T>, index: Int, value: T, rangeError: E,
  combine: fn(T,T)->Result<T,E>!{}~{Send+Share}) -> Result<Segment<T>,E> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	<code>Result<Segment<T>,E></code>
--------------	---

generic契約	<code>T: Share, E: Share</code>
-----------	---------------------------------

費用契約・原文	$O(n)$ copy plus $O(\log n)$ callback evaluations; $n \leq 32768$.
---------	---

失敗・境界契約・原文	Range returns rangeError; first callback Err(E) is returned; borrowed tree remains unchanged on all failures.
------------	---

std.sequence

列の変換・集約等の補助。入力を借用するか新しいListを作るかをsignatureで確認します。

```
import
```

```
import std.sequence as sequence;
```

共通型のfield・variant / このmoduleの実装を読む

std.sequence.compressed

公開宣言

```
pub fn compressed(values: &List<Int>) -> List<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 `List<Int>`

std.sequence.nextPermutation

公開宣言

```
pub fn nextPermutation(values: &mut List<Int>) -> Bool effects {}
```

効果 なし (effects {})

canonical戻り型 `Bool`

std.sequence.prefix

公開宣言

```
pub fn prefix(values: &List<Int>) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.sequence.reverse

公開宣言

```
pub fn reverse<T: Share>(values: &mut List<T>) -> Unit effects {}
```

効果 なし (effects {})

canonical戻り型 Unit

generic契約 T: Share

std.sequence.rotate

公開宣言

```
pub fn rotate<T: Share>(values: &List<T>, offset: Int) -> List<T> effects {}
```

効果 なし (effects {})

canonical戻り型 List<T>

generic契約 T: Share

std.sequence.unique

公開宣言

```
pub fn unique<T: Share>(values: &List<T>, compare: fn(T,T)->Int!{}~{Send+Share}) -> List<T> effects {}
```

効果 なし (effects {})

canonical戻り型 List<T>

generic契約 T: Share

std.sort

安定merge sortとin-place版、fallible comparator版。負/0/正の一貫した比較を使います。
tryStableは借用入力を保持します。

```
import
```

```
import std.sort as sort;
```

共通型のfield・variant / このmoduleの実装を読む

std.sort.inPlace

公開宣言

```
pub fn inPlace<T: Share>(input: &mut List<T>, compare: fn(T,T)->Int!{})~{Send+Share}) -> Unit  
effects {}
```

効果 なし (effects {})

canonical戻り型 **Unit**

generic契約 **T: Share**

std.sort.integers

公開宣言

```
pub fn integers(input: &List<Int>) -> List<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 **List<Int>**

std.sort.stable

公開宣言

```
pub fn stable<T: Share>(input: &List<T>, compare: fn(T,T)->Int!{}~{Send+Share}) -> List<T> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	List<T>
--------------	---------

generic契約	T: Share
-----------	----------

std.sort.tryStable

公開宣言

```
pub fn tryStable<T: Share, E: Share>(input: &List<T>, compare: fn(T,T)->Result<Int,E>!{}~{Send+Share}) -> Result<List<T>,E> effects {}
```

効果	なし (effects {})
----	-----------------

canonical戻り型	Result<List<T>,E>
--------------	-------------------

generic契約	T: Share, E: Share
-----------	--------------------

費用契約・原文	O(n log n) comparisons and temporary elements; caller work budget applies.
---------	--

失敗・境界契約・原文	First callback Err(E) is returned; borrowed input is unchanged, including on fatal budget failure.
------------	--

std.sparse

native疎行列の構築・計算。storage変換、一時領域、shapeとindexの検査を含めて費用を見積もります。

```
import
```

```
import std.sparse as sparse;
```

module固有の公開型

型（公開field型）

```
record Matrix {
  rows: Int
  cols: Int
  offsets: IntArray
  indices: IntArray
  values: FloatArray
}
```

共通型のfield・variant / このmoduleの実装を読む

std.sparse.create

公開宣言

```
pub fn create(rows: Int, cols: Int, row: &IntArray, col: &IntArray, values: &FloatArray) ->
Result<Matrix,StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 **Result<Matrix,StdError>**

費用契約・原文 O(nnz log nnz + rows) work / O(nnz + rows) native buffers; at most 2^20 rows, columns and input entries.

失敗・境界契約・原文 NumericShape/Type/Index/Size/NonFinite; rank-one COO vectors, stable duplicate summation, sorted unique columns, exact zeros dropped.

std.sparse.matvec

公開宣言	
pub fn matvec(matrix: &Matrix, right: &FloatArray) -> Result<FloatArray,StdError> effects {}	
効果	なし (effects {})
canonical戻り型	Result<FloatArray,StdError>
費用契約・原文	O(rows + cols + nnz) work / O(rows + cols) temporary storage; borrowed CSR page scans, native Float64 accumulation without per-cell VM objects.
失敗・境界契約・原文	NumericShape/Type/Index/Size/NonFinite; validates all CSR offsets, sorted unique columns, finite values and rank-one right vector, including user-constructed records. Synchronous; fatal budgets apply.

std.sparseAsync

疎行列の協調計算。native workとTaskError、取消、部分状態の寿命を確認します。

```
import

import std.sparseAsync as sparseAsync;
```

module固有の公開型

型（公開field型）

```
record SolveResult {
  solution: FloatArray
  iterations: Int
  residual: Float
  converged: Bool
}
```

共通型のfield・variant / このmoduleの実装を読む

std.sparseAsync.conjugateGradient

公開宣言

```
pub async fn conjugateGradient(matrix: $import$sparse$Matrix, right: FloatArray, tolerance: Float,
maxIterations: Int) -> Result<SolveResult,StdError> effects {tasks}
```

効果	tasks
canonical戻り型	Result<SolveResult,StdError>
費用契約・原文	$O(\text{iterations} * (\text{rows} + \text{cols} + \text{nnz}) * \log(\max(\text{rows}, \text{cols}, \text{nnz})))$; CSR validation/products and vector/norm/dot kernels yield after at most 4096 logical items; helpers share the current Task and native input pages.
失敗・境界契約・原文	NumericShape/Domain/NonFinite; positive tolerance, 1..100000 iterations; NumericDomain also reports underflow of the squared search residual, square symmetric positive-definite input required. No dense materialization; zero initial guess, residual is $\ b-Ax\ $, convergence target tolerance * $\max(1, \ b\)$. Returns converged=false on iteration exhaustion; fatal budgets/cancellation apply.

std.sparseAsync.matvec

公開宣言	
<pre>pub async fn matvec(matrix: \$import\$sparse\$Matrix, right: FloatArray) -> Result<FloatArray, StdError> effects {tasks}</pre>	
効果	tasks
canonical戻り型	Result<FloatArray, StdError>
費用契約・原文	O(cols log(cols) + (rows + nnz) log(max(rows,cols,nnz))) work; bounded 4096 input checks, entries or row completions per step. Shared CSR/right pages, O(rows) COW output; no nnz-sized temporary copy or host worker.
失敗・境界契約・原文	NumericShape/Type/Index/Size/NonFinite; canonical rank-one CSR and finite right values (including unused cells), compensated sums, same 2^20 size bound as sparse.matvec. Cancellation discards private partial output; retained task checkpoints preserve it.

std.stream

chunk/token/UTF-8/writerの補助。chunk境界は文字・token境界と一致しないため状態を保持します。

```
import
```

```
import std.stream as stream;
```

module固有の公開型

型（公開field型）

```
record StreamError {
    code: String
    position: Int
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Tokens {
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Utf8 {
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct Writer {
}
```

共通型のfield・variant / このmoduleの実装を読む

std.stream.decode

公開宣言

```
pub fn decode(decoder: &mut Utf8, chunk: Bytes, finish: Bool) -> Result<String,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String,StreamError>`

std.stream.drain

公開宣言

```
pub fn drain(writer: &mut Writer) -> Result<Bytes,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Bytes,StreamError>`

std.stream.feed

公開宣言

```
pub fn feed(tokens: &mut Tokens, chunk: Bytes, finish: Bool) -> Result<Unit,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,StreamError>`

std.stream.next

公開宣言

```
pub fn next(tokens: &mut Tokens) -> Option<Bytes> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Bytes>`

std.stream.nextInt

公開宣言

```
pub fn nextInt(tokens: &mut Tokens) -> Result<Option<Int>,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Option<Int>,StreamError>`

std.stream.tokens

公開宣言

```
pub fn tokens(limit: Int) -> Result<Tokens,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Tokens,StreamError>`

std.stream.utf8

公開宣言

```
pub fn utf8() -> Utf8 effects {}
```

効果 なし (effects {})

canonical戻り型 `Utf8`

std.stream.write

公開宣言

```
pub fn write(writer: &mut Writer, chunk: Bytes) -> Result<Unit,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Unit,StreamError>`

std.stream.writer

公開宣言

```
pub fn writer(capacity: Int) -> Result<Writer,StreamError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Writer,StreamError>`

std.stringSearch

KMP/Z等の文字列検索primitive。indexの単位と空patternの扱いを実装で確認します。

```
import  
  
import std.stringSearch as stringSearch;
```

共通型のfield・variant / このmoduleの実装を読む

std.stringSearch.kmp

公開宣言

```
pub fn kmp(input: Bytes, pattern: Bytes) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.stringSearch.prefix

公開宣言

```
pub fn prefix(input: Bytes) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.stringSearch.z

公開宣言

```
pub fn z(input: Bytes) -> Result<List<Int>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<Int>,String>`

std.suffix

suffix関連の構築・検索。入力容量、列挙順、indexの単位とworkを確認します。

```
import
```

```
import std.suffix as suffix;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct SuffixIndex {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.suffix.build

公開宣言

```
pub fn build(input: Bytes) -> Result<SuffixIndex, StdError> effects {}
```

効果 なし（effects {}）

canonical戻り型 `Result<SuffixIndex, StdError>`

費用契約・原文 $O(n \log n)$ native radix doubling, linear LCP; $O(n)$ scratch, two paged IntArray results sharing across checkpoints.

失敗・境界契約・原文 NumericSize for inputs above 1048576 bytes; fatal VM memory/work budgets preflight the kernel.

std.suffix.find

公開宣言

```
pub fn find(index: &SuffixIndex, pattern: Bytes) -> Result<Tuple<Int,Int>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Tuple<Int,Int>,StdError>`

費用契約・原文 $O(\text{pattern bytes} * \log n)$ comparisons; half-open interval in suffix order. Empty pattern matches n nonempty suffixes, empty input has none.

std.suffix.lcp

公開宣言

```
pub fn lcp(index: &SuffixIndex) -> IntArray effects {}
```

効果 なし (effects {})

canonical戻り型 `IntArray`

費用契約・原文 $O(1)$ shared handle; element 0 is zero, element i compares `order[i-1]` and `order[i]`.

std.suffix.length

公開宣言

```
pub fn length(index: &SuffixIndex) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.suffix.order

公開宣言

```
pub fn order(index: &SuffixIndex) -> IntArray effects {}
```

効果 なし (effects {})

canonical戻り型 IntArray

std.task

yieldNowとselectReady。handoffは外部待機ではありません。selectReadyは完了indexを返し、選択されたTaskを別にawaitします。

```
import  
  
import std.task as task;
```

共通型のfield・variant / このmoduleの実装を読む

std.task.selectReady

```
公開宣言  
  
pub fn selectReady<A, B>(left: Task<A>, right: Task<B>) -> Task<Int> effects {tasks}
```

効果	tasks
canonical戻り型	Task<Int>
generic契約	A, B
費用契約・原文	A cold scheduler action; starts both inputs when started. Constant metadata and no result copy.
失敗・境界契約・原文	Reports completion (including failure or cancellation) with index 0/1; left wins an observed tie. Await the selected input to obtain its result. Cancelling this waiter leaves both inputs running.

std.task.yieldNow

```
公開宣言  
  
pub fn yieldNow() -> Unit effects {tasks}
```

効果	tasks
canonical戻り型	Unit
費用契約・原文	One scheduler handoff. Default choices rotate among ready tasks of the best priority; explicit recorded choices take precedence.
失敗・境界契約・原文	Only VM internal regions; forbidden in external regions and cleanup / guard execution. Does not wait for host input or allocate a new Task.

std.taskError

TaskErrorを元診断・typed budget・wait graphを持つFailureへ変換。asStdはcodeと行だけの投影です。

```
import
```

```
import std.taskError as taskError;
```

module固有の公開型

型（公開field型）

```
struct Failure {  
  code: String  
  diagnostic: Option<Diagnostic>  
  budget: Option<BudgetKind>  
  waitGraph: Option<WaitGraph>  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.taskError.asStd

公開宣言

```
pub fn asStd(error: TaskError) -> StdError effects {}
```

効果 なし（effects {}）

canonical戻り型 **StdError**

費用契約・原文 O(1) projection; returns the original diagnostic code and line or a classified task/budget code.

失敗・境界契約・原文 Does not retain the full diagnostic tree; use describe before projecting when messages, source, causes or typed budget details are required. No partial calculation result is returned.

std.taskError.describe

公開宣言

```
pub fn describe(error: TaskError) -> Failure effects {}
```

効果 なし (effects {})

canonical戻り型 **Failure**

費用契約・原文 O(1) envelope creation; diagnostic causes and wait-graph storage remain shared with the original task error.

失敗・境界契約・原文 Pure conversion, no external effects, retries or recovery of a cancelled task; preserves Failed diagnostics and typed budget/deadlock details.

std.tcp

byte stream接続・送受信。external/network/tasksを使い、readをmessage境界と仮定しません。acceptedBytesと結果不明を区別します。

```
import

import std.tcp as tcp;
```

共通型のfield・variant / このmoduleの実装を読む

std.tcp.close

公開宣言

```
pub fn close(socket: &mut TcpSocket, deadlineMillis: Int) -> Task<Result<Unit,TcpError>> effects
{external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<Unit,TcpError>>

費用契約・原文 O(pending operations); closes both halves and cancels pending operations.

失敗・境界契約・原文 TcpError; scope exit also closes the socket. A restored VM handle does not recreate the connection.

std.tcp.configuredTls

公開宣言

```
pub fn configuredTls(host: String, port: Int, ca: Bytes, deadlineMillis: Int) ->
Task<Result<TcpSocket,TcpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 Task<Result<TcpSocket,TcpError>>

費用契約・原文 O(host bytes + PEM trust roots); a nonempty PEM bundle supplies explicit trust roots, up to 65536 bytes.

失敗・境界契約・原文 TcpError; no hostname/certificate bypass. Empty PEM uses system roots. Client certificate authentication is not provided.

std.tcp.connect

公開宣言

```
pub fn connect(host: String, port: Int, deadlineMillis: Int) -> Task<Result<TcpSocket,TcpError>>
effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<TcpSocket,TcpError>>`

費用契約・原文 `O(host bytes)`; at most eight sockets and eight pending operations.

失敗・境界契約・原文 `TcpError`; plain TCP, no automatic application retry, deadline 1..120000 ms.

std.tcp.connectTls

公開宣言

```
pub fn connectTls(host: String, port: Int, deadlineMillis: Int) -> Task<Result<TcpSocket,TcpError>>
effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<TcpSocket,TcpError>>`

費用契約・原文 `O(host bytes + trust roots)`; verifies hostname and certificate with the system trust store.

失敗・境界契約・原文 `TcpError`; TLS 1.2/1.3, one deadline includes connect and handshake, no automatic application retry.

std.tcp.read

公開宣言

```
pub fn read(socket: &mut TcpSocket, maxBytes: Int, deadlineMillis: Int) ->
Task<Result<Option<Bytes>,TcpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Option<Bytes>,TcpError>>`

費用契約・原文 `O(chunk bytes)`; one outstanding read, 1..65536 bytes, EOF returns None.

失敗・境界契約・原文 `TcpError`; timeout/cancellation closes the physical socket; a checkpoint retains completed receipts.

std.tcp.shutdownWrite

公開宣言

```
pub fn shutdownWrite(socket: &mut TcpSocket, deadlineMillis: Int) -> Task<Result<Unit,TcpError>>
effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Unit,TcpError>>`

費用契約・原文 O(1); sends EOF while keeping the read half available.

失敗・境界契約・原文 TcpError; physical half-close cannot be undone by revert.

std.tcp.write

公開宣言

```
pub fn write(socket: &mut TcpSocket, body: Bytes, deadlineMillis: Int) ->
Task<Result<Int,TcpError>> effects {external, network, tasks}
```

効果 external, network, tasks

canonical戻り型 `Task<Result<Int,TcpError>>`

費用契約・原文 O(body bytes); one outstanding write, at most 65536 bytes per operation.

失敗・境界契約・原文 TcpError; acceptedBytes counts locally accepted bytes, not peer processing. Unknown phase can include additional in-flight bytes.

std.text

Unicode scalar単位のtext操作。組み込みStringのbyte offset、unicodeのgraphemeとは異なる単位があります。

```
import
```

```
import std.text as text;
```

共通型のfield・variant / このmoduleの実装を読む

std.text.find

公開宣言

```
pub fn find(value: String, needle: String) -> Option<Int> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<Int>`

std.text.length

公開宣言

```
pub fn length(value: String) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

std.text.slice

公開宣言

```
pub fn slice(value: String, start: Int, end: Int) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.text.split

公開宣言

```
pub fn split(value: String, delimiter: String) -> Result<List<String>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<String>, StdError>`

std.text.tokens

公開宣言

```
pub fn tokens(value: String) -> Result<List<String>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<String>, StdError>`

std.text.trim

公開宣言

```
pub fn trim(value: String) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.training

sample数によるbatch meanとglobal-norm clipping等。同期native autodiff/optimizerとの合成です。

```
import
```

```
import std.training as training;
```

共通型のfield・variant / このmoduleの実装を読む

std.training.clipGlobalNorm

公開宣言

```
pub fn clipGlobalNorm(gradients: &Map<String,FloatArray>, maximum: Float) ->
Result<Map<String,FloatArray>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Map<String,FloatArray>,StdError>`

費用契約・原文 O(elements) scaled native norms, then O(elements) scaling only when clipping is needed; unchanged arrays share storage. Model parameter bounds apply. Synchronous.

失敗・境界契約・原文 TrainingNorm or numeric/model validation errors; positive finite maximum, finite per-array norm required. Global combination avoids squaring large norms or materializing all gradient cells in a VM List. Inputs unchanged.

std.training.gradients

公開宣言

```
pub fn gradients(tape: &$import$autodiff$Tape, loss: $import$autodiff$Node, parameters:
&Map<String,$import$autodiff$Node>) -> Result<Map<String,FloatArray>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<Map<String,FloatArray>,StdError>`

費用契約・原文 One backward pass plus named metadata and model validation of selected cells; <=128 selected nodes, native pages shared. Returned map retains selected gradients only; intermediate gradients become collectible when the local owner is dropped.

失敗・境界契約・原文 TrainingParameters/Unused, AutodiffNode/Loss or numeric errors; names follow the model contract, tape and selected nodes unchanged. Selecting constants or unreachable nodes is an error, not an invented zero gradient.

std.training.meanSquaredError

公開宣言

```
pub fn meanSquaredError(tape: &mut $import$autodiff$Tape, predicted: $import$autodiff$Node, target:
$import$autodiff$Node) -> Result<$import$autodiff$Node,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<$import$autodiff$Node,StdError>`

費用契約・原文 O(elements) native forward kernels; same shape required, mean is over every element. Synchronous and subject to native work/memory budgets.

失敗・境界契約・原文 AutodiffNode/Size or numeric validation errors; intermediate successful tape operations remain on a later failure.

std.training.mergeMean

公開宣言	
<pre>pub fn mergeMean(left: &Map<String,FloatArray>, leftSamples: Int, right: &Map<String,FloatArray>, rightSamples: Int) -> Result<Map<String,FloatArray>,StdError> effects {}</pre>	
効果	なし (effects {})
canonical戻り型	Result<Map<String,FloatArray>,StdError>
費用契約・原文	O(parameters + elements), native scale/add outputs; weights batch means by their positive sample counts. <=128 named parameters /32 MiB model; exact names and shapes required, inputs unchanged.
失敗・境界契約・原文	TrainingSamples/Parameters, NumericShape/NonFinite/Overflow/Size; total count <=2^53-1 for exact conversion. Fatal budgets apply, no partial map escapes.

std.trie

prefix検索のdata structure。入力単位、capacity、更新・返却ownerを確認します。

```
import
```

```
import std.trie as trie;
```

module固有の公開型

型（公開fieldなし。専用APIで生成・操作）

```
struct BytesTrie<V> {  
  
}
```

型（公開fieldなし。専用APIで生成・操作）

```
struct TextTrie<V> {  
  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.trie.create

公開宣言

```
pub fn create<V: Share>(capacity: Int) -> Result<BytesTrie<V>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<BytesTrie<V>,String>**

generic契約 **V: Share**

失敗・境界契約・原文 Capacity requires 1..1048576 nodes including the root; insert prevalidates capacity before mutation.

std.trie.createText

公開宣言

```
pub fn createText<V: Share>(capacity: Int) -> Result<TextTrie<V>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<TextTrie<V>,String>

generic契約 V: Share

std.trie.get

公開宣言

```
pub fn get<V: Share>(tree: &BytesTrie<V>, key: Bytes) -> Option<V> effects {}
```

効果 なし (effects {})

canonical戻り型 Option<V>

generic契約 V: Share

std.trie.insert

公開宣言

```
pub fn insert<V: Share>(tree: &mut BytesTrie<V>, key: Bytes, value: V) -> Result<Option<V>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Option<V>,String>

generic契約 V: Share

費用契約・原文 O(key bytes * log(node capacity)) map/list accesses; existing value is returned. Keys are at most 1048576 bytes.

std.trie.nodes

公開宣言

```
pub fn nodes<V: Share>(tree: &BytesTrie<V>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

generic契約 `V: Share`

std.trie.prefixCount

公開宣言

```
pub fn prefixCount<V: Share>(tree: &BytesTrie<V>, prefix: Bytes) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 `Int`

generic契約 `V: Share`

std.trie.remove

公開宣言

```
pub fn remove<V: Share>(tree: &mut BytesTrie<V>, key: Bytes) -> Option<V> effects {}
```

効果 なし (effects {})

canonical戻り型 `Option<V>`

generic契約 `V: Share`

費用契約・原文 $O(\text{key bytes} * \log(\text{node capacity}))$; prunes zero-count branches and recycles their node IDs.

std.trie.size

公開宣言

```
pub fn size<V: Share>(tree: &BytesTrie<V>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 **Int**

generic契約 **V: Share**

std.trie.textGet

公開宣言

```
pub fn textGet<V: Share>(tree: &TextTrie<V>, key: String) -> Result<Option<V>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Option<V>,String>**

generic契約 **V: Share**

std.trie.textInsert

公開宣言

```
pub fn textInsert<V: Share>(tree: &mut TextTrie<V>, key: String, value: V) ->  
Result<Option<V>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Option<V>,String>**

generic契約 **V: Share**

std.trie.textNodes

公開宣言

```
pub fn textNodes<V: Share>(tree: &TextTrie<V>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

generic契約 V: Share

std.trie.textPrefixCount

公開宣言

```
pub fn textPrefixCount<V: Share>(tree: &TextTrie<V>, prefix: String) -> Result<Int,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Int,String>

generic契約 V: Share

std.trie.textRemove

公開宣言

```
pub fn textRemove<V: Share>(tree: &mut TextTrie<V>, key: String) -> Result<Option<V>,String> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Option<V>,String>

generic契約 V: Share

std.trie.textSize

公開宣言

```
pub fn textSize<V: Share>(tree: &TextTrie<V>) -> Int effects {}
```

効果 なし (effects {})

canonical戻り型 Int

generic契約 V: Share

std.unicode

grapheme、明示正規化、full default casing。自動正規化はしません。offsetsはUTF-8 bytesです。データ版はversionsで確認します。

```
import
```

```
import std.unicode as unicode;
```

module固有の公開型

variantとpayload型

```
enum Form {  
    NFC,  
    NFD,  
    NFKC,  
    NFKD  
}
```

共通型のfield・variant / このmoduleの実装を読む

std.unicode.graphemeCount

公開宣言

```
pub fn graphemeCount(text: &String) -> Result<Int,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<Int,StdError>**

std.unicode.graphemeOffsets

公開宣言

```
pub fn graphemeOffsets(text: &String) -> Result<List<Int>,StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 **Result<List<Int>,StdError>**

std.unicode.graphemeSlice

公開宣言

```
pub fn graphemeSlice(text: &String, start: Int, end: Int) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<String, StdError>

失敗・境界契約・原文 UnicodeIndex; zero-based cluster boundaries, start inclusive/end exclusive.

std.unicode.graphemes

公開宣言

```
pub fn graphemes(text: &String) -> Result<List<String>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<List<String>, StdError>

std.unicode.isNormalized

公開宣言

```
pub fn isNormalized(text: &String, form: Form) -> Result<Bool, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 Result<Bool, StdError>

std.unicode.lower

公開宣言

```
pub fn lower(text: &String) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.unicode.normalize

公開宣言

```
pub fn normalize(text: &String, form: Form) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

費用契約・原文 bounded canonical/compatibility decomposition, ordering and composition; native work and scratch precharged.

失敗・境界契約・原文 UnicodeInputLimit/OutputLimit; input <=1MiB, output <=8MiB; fatal VM budgets apply before expansion.

std.unicode.sentences

公開宣言

```
pub fn sentences(text: &String) -> Result<List<String>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<String>, StdError>`

std.unicode.upper

公開宣言

```
pub fn upper(text: &String) -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.unicode.versions

公開宣言

```
pub fn versions() -> Result<String, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<String, StdError>`

std.unicode.wordBounds

公開宣言

```
pub fn wordBounds(text: &String) -> Result<List<String>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<String>, StdError>`

std.unicode.words

公開宣言

```
pub fn words(text: &String) -> Result<List<String>, StdError> effects {}
```

効果 なし (effects {})

canonical戻り型 `Result<List<String>, StdError>`

共通型付録

SDKのAPI schemaが持つ共通型のpayload・公開fieldです。native資源は専用factoryを使います。以下のfield型は任意のphysical接続をconstructorで作れる保証ではありません。

BigInt

型（公開fieldなし。専用APIで生成・操作）

```
record BigInt {  
  
}
```

CsvStreamState

型（公開fieldなし。専用APIで生成・操作）

```
record CsvStreamState {  
  
}
```

DbBatch

型（公開field型）

```
record DbBatch {  
  rows: Frozen<List<DbRow>>  
  done: Bool  
}
```

DbConnection

型（公開field型）

```
record DbConnection {  
  backend: String  
}
```

DbCursor

型（公開field型）

```
record DbCursor {  
    columns: Frozen<List<String>>  
}
```

DbError

型（公開field型）

```
record DbError {  
    code: String  
    phase: String  
    sqlCode: Option<Int>  
    sqlState: Option<String>  
}
```

DbRow

型（公開field型）

```
record DbRow {  
    values: Frozen<List<DbValue>>  
}
```

DbStatement

型（公開field型）

```
record DbStatement {  
    columns: Frozen<List<String>>  
    parameters: Int  
}
```

DbValue

variantとpayload型

```
enum DbValue {  
    Bool(Bool),  
    Bytes(Bytes),  
    Float(Float),  
    Int(Int),  
    Null,  
    Private(String),  
    Text(String)  
}
```

Decimal

型（公開fieldなし。専用APIで生成・操作）

```
record Decimal {  
  
}
```

DecimalRounding

variantとpayload型

```
enum DecimalRounding {  
    AwayFromZero,  
    Ceiling,  
    Exact,  
    Floor,  
    HalfDown,  
    HalfEven,  
    HalfUp,  
    TowardZero  
}
```

Duration

型（公開fieldなし。専用APIで生成・操作）

```
record Duration {  
  
}
```

EigenResult

型（公開field型）

```
record EigenResult {  
    values: FloatArray  
    vectors: FloatArray  
    sweeps: Int  
}
```

EigenWork

型（公開fieldなし。専用APIで生成・操作）

```
record EigenWork {  
  
}
```

FftWork

型（公開fieldなし。専用APIで生成・操作）

```
record FftWork {  
  
}
```

FloatArray

型（公開fieldなし。専用APIで生成・操作）

```
record FloatArray {  
  
}
```

GraphBfsWork

型（公開fieldなし。専用APIで生成・操作）

```
record GraphBfsWork {  
  
}
```

HistogramResult

型（公開field型）

```
record HistogramResult {  
    counts: IntArray  
    underflow: Int  
    overflow: Int  
}
```

HttpDownload

型（公開field型）

```
record HttpDownload {  
    status: Int  
    headers: Frozen<List<HttpHeader>>  
}
```

HttpError

型（公開field型）

```
record HttpError {  
    code: String  
    phase: String  
    status: Int  
}
```

HttpHeader

型（公開field型）

```
record HttpHeader {  
    name: String  
    value: Bytes  
}
```

HttpResponse

型（公開field型）

```
record HttpResponse {  
    status: Int  
    headers: Frozen<List<HttpHeader>>  
    body: Bytes  
}
```

HttpServer

型（公開field型）

```
record HttpServer {  
    address: String  
    port: Int  
    maxBytes: Int  
}
```

HttpServerError

型（公開field型）

```
record HttpServerError {  
    code: String  
    phase: String  
    status: Int  
}
```

HttpRequest

型（公開field型）

```
record HttpRequest {  
    method: String  
    target: String  
    path: String  
    query: String  
    headers: Frozen<List<HttpHeader>>  
    body: Bytes  
}
```


HttpUpload

型（公開field型）

```
record HttpUpload {  
    maxBytes: Int  
}
```

Instant

型（公開fieldなし。専用APIで生成・操作）

```
record Instant {  
  
}
```

IntArray

型（公開fieldなし。専用APIで生成・操作）

```
record IntArray {  
  
}
```

Json

variantとpayload型

```
enum Json {  
    Array(Frozen<List<Json>>),  
    Bool(Bool),  
    Float(Float),  
    Int(Int),  
    Null,  
    Object(Frozen<Map<String,Json>>),  
    Text(String)  
}
```

JsonError

型（公開field型）

```
record JsonError {  
    code: String  
    line: Int  
    column: Int  
    offset: Int  
}
```

JsonStreamEvent

variantとpayload型

```
enum JsonStreamEvent {  
    ArrayEnd(Int),  
    ArrayStart(Int),  
    Bool(Bool, Int),  
    Key(String, Int),  
    Null(Int),  
    Number(String, Int),  
    ObjectEnd(Int),  
    ObjectStart(Int),  
    Text(String, Int)  
}
```

JsonStreamState

型（公開fieldなし。専用APIで生成・操作）

```
record JsonStreamState {  
  
}
```

LeastSquaresWork

型（公開fieldなし。専用APIで生成・操作）

```
record LeastSquaresWork {  
  
}
```

QrArrayResult

型（公開field型）

```
record QrArrayResult {  
    q: FloatArray  
    r: FloatArray  
    permutation: IntArray  
    rank: Int  
}
```

QrResult

型（公開field型）

```
record QrResult {  
    q: FloatArray  
    r: FloatArray  
    permutation: Frozen<List<Int>>  
    rank: Int  
}
```

QrWork

型（公開fieldなし。専用APIで生成・操作）

```
record QrWork {  
  
}
```

Regex

型（公開fieldなし。専用APIで生成・操作）

```
record Regex {  
  
}
```

SolveWork

型（公開fieldなし。専用APIで生成・操作）

```
record SolveWork {  
  
}
```

SparseWork

型（公開fieldなし。専用APIで生成・操作）

```
record SparseWork {  
  
}
```

StdError

型（公開field型）

```
record StdError {  
    code: String  
    offset: Int  
}
```

TcpError

型（公開field型）

```
record TcpError {  
    code: String  
    phase: String  
    acceptedBytes: Int  
}
```

TcpSocket

型（公開field型）

```
record TcpSocket {  
    peer: String  
}
```

TensorIdentityWork

型（公開fieldなし。専用APIで生成・操作）

```
record TensorIdentityWork {  
  
}
```

診断・Task・propertyの組込み型

次はSDK文書の組込み宣言です。enumのvariant名一覧はpayloadを省略しています。TaskErrorの詳しい診断はtaskError.describeのFailureを参照してください。CLIのframes/hints追加情報は診断JSONの契約です。

SDK組込み型宣言

```
pub enum BudgetKind { Compiler, DependencyResolution, EffectInference, ExecutionSteps, History, HistoryMemory, HistoryStorage, IteratorItems, Monomorphization, NativeWork, Other, SchedulerObjects, SchedulerStorage, TaskSteps, TransferDepth, TypeExpansion }
```

SDK組込み型宣言

```
pub enum TaskError { BudgetExceeded, Cancelled, ChannelClosed, Deadlock, Failed, TimedOut }
```

SDK組込み型宣言

```
pub enum WaitTarget { Channel, Group, Task }
```

SDK組込み型宣言

```
pub struct Diagnostic { code: String, message: String, source: String, line: Int, column: Int, taskId: Option<Int>, causes: List<Diagnostic>, waitGraph: WaitGraph }
```

SDK組込み型宣言

```
pub struct PropertyCase<T> { seed: Int, case: Int, input: T, shrinks: Int }
```

SDK組込み型宣言

```
pub struct PropertyFailure { seed: Int, case: Int, input: Int, shrinks: Int }
```

SDK組込み型宣言

```
pub struct WaitEdge { task: Int, target: WaitTarget }
```

SDK組込み型宣言

```
pub struct WaitGraph { description: String, edges: List<WaitEdge> }
```

TaskErrorとWaitTargetのpayload

SDKのvariant名一覧で省略されているpayloadを、v2.0.0のsrc/v2/v05.rsと照合した補足です。

```
enum TaskError {  
    BudgetExceeded(BudgetKind),  
    Cancelled,  
    ChannelClosed,  
    Deadlock(WaitGraph),  
    Failed(Diagnostic),  
    TimedOut  
}  
enum WaitTarget {  
    Task(Int),  
    Channel(Int),  
    Group(Int)  
}
```

出典と検証

本文はREWIND 2.0.0の実装・言語リファレンス・各機能の契約を照合して編集しました。全60実行例について、source実行とsourceを削除したartifact実行の出力を照合しました。うち8例はartifactのcompact recordを、sourceとGUI入力fixtureに依存せずreplayしました。GUI実行例はfixture検証です。今回の実行例検証環境はLinux x86_64です。

対象compiler / language	2.0.0
対象実装commit	d0f3ee6684769d3f92dfb0e21c55defc00dc181
参照SDK SHA-256	a9320aea142fed6a6461c0257e2c2c69610b5f770ea8afb0eec93fcb90608ccf

APIは公開Linux SDK内のstd-api.json、86組のmodule.api.json / module.mdから取得しています。標準moduleのsourceはSDKとrepositoryをbyte単位で一致確認し、HTML生成時にもSHA-256を照合しています。歴史的な草案を現在の未実装項目として再掲していません。

再生成用の本文、snapshot、例、期待出力、検証scriptはrepositoryのdocs/bookとscriptsにあります。PDFはこのHTMLの印刷用表示から生成できます。HTML/PDFの編集日: 2026-10-10。

対象Release・対象実装の固定版